

DOCUMENTACIÓN TÉCNICA

NexusMQ

Broker de mensajería distribuido en C++23
shared-nothing thread-per-core · Raft por partición

Andrés Ojeda Rodríguez · 2026

Tabla de contenidos

Documentación técnica

Prefacio

1. Resumen ejecutivo

2. Contexto y motivación

3. Estado del arte

4. Glosario

5. Vista de conjunto

6. Principios de diseño

7. Concurrencia

8. Modelo de I/O

9. Almacenamiento

10. Replicación y consenso

11. Ingress

12. Observabilidad

13. Protocolo binario nativo

14. Subconjunto Kafka

15. API REST de administración

16. Modelo de errores y códigos de wire

17. Mapa de módulos

18. Catálogo por subsistema

19. Arranque y composition root

20. Herramientas y bindings

21. Estrategia de pruebas

22. Puerta de calidad y CI/CD

23. Rendimiento y benchmarks

24. Portabilidad

25. Despliegue

26. Configuración y operación

27. Seguridad

28. Registro de decisiones (ADR)

29. Historia de desarrollo

30. Limitaciones y trabajo futuro

Apéndice A. Bibliografía

Apéndice — Catálogo de diagramas

Diagrama 1: Contexto del sistema (C4 nivel 1)

Diagrama 2: Contenedores de un nodo (C4 nivel 2)

Diagrama 3: Grafo de dependencias de targets

Diagrama 4: Mapa fase → targets

Diagrama 5: Topología thread-per-core (shared-nothing)

Diagrama 6: Sharding del plano de datos por núcleo

Diagrama 7: Secuencia de una operación de E/S sobre el proactor

Diagrama 8: Layout del log de una partición

Diagrama 9: Retención y compactación del log

Diagrama 10: Estados y transiciones de una réplica Raft

Diagrama 11: Replicación y confirmación a quórum (`acks=quorum`)

Diagrama 12: Failover y elección de líder

Diagrama 13: Espacios de coordenadas (índice de entrada Raft vs offset por record)

Diagrama 14: Compactación por snapshot e `InstallSnapshot`

Diagrama 15: Planos de red (cliente vs inter-nodo Raft)

Diagrama 16: Layout de la cabecera del frame nativo

Diagrama 17: Petición/respuesta del subconjunto Kafka

Diagrama 18: Flujo REST admin con JWT

Diagrama 19: Ingress en dos modos — nativo directo vs proxy

Diagrama 20: Puente TLS — OpenSSL síncrono sobre el proactor asíncrono

Diagrama 21: Despliegue local con Docker Compose

Diagrama 22: Despliegue en Kubernetes — StatefulSet + Service headless

Diagrama 23: Pipeline de observabilidad — métricas y logs

Apéndice — Registro de decisiones (ADR)

Registro de decisiones de arquitectura (ADR)

ADR-0001: Plataforma de desarrollo y `*target*` (Linux primero, Windows después)

ADR-0002: Modelo de I/O asíncrona (proactor; `io_uring` primero, IOCP después)

ADR-0003: Modelo de replicación (Raft por partición)

ADR-0004: Protocolo del plano de datos (binario propio + gateway REST)

ADR-0005: Arquitectura de concurrencia (*shared-nothing thread-per-core* con reactor propio)

ADR-0006: Rol del *ingress* (dos modos: nativo directo + proxy/REST)

ADR-0007: Postura de consistencia (CP / PACELC PC-EC)

ADR-0008: Viabilidad de coste cero (desarrollo y test gratuitos; cloud como *target* opcional)

ADR-0009: Política de manejo de errores por capa (excepciones / `std::expected` / códigos de wire)

ADR-0010: Entorno de desarrollo (VS Code sobre WSL; reemplaza el IDE de ADR-0001)

ADR-0011: Estándar C++23 + libc++ en Clang (para `std::expected`)

ADR-0012: Backend `io_uring` directo sobre el uapi del kernel (sin liburing)

ADR-0013: Capa `nexus-wire` para el framing sobre conexión (`FrameReader` / `FrameWriter`)

ADR-0014: Modelo del log de Raft (índice = ordinal de entrada; término en sidecar)

ADR-0015: `RaftNode` como máquina de estados síncrona sin E/S (entradas → salidas)

ADR-0016: `ReplicatedPartition` como tipo paralelo a `Partition` (composición de la pila Raft)

ADR-0017: Target `nexus-telemetry` para observabilidad (métricas/logs) bajo el broker

ADR-0018: REST admin por puerto/adaptador (`AdminService` en ingress, `AdminApi` en server)

ADR-0019: TLS opcional vía OpenSSL con puente de BIOs de memoria sobre el `Proactor`

ADR-0020: *Binding* de Python vía **ABI C estable** (`nexus-ffi` + `ctypes`) en lugar de `pybind11`

ADR-0021: Backend **IOCP** (Windows) — diseño fijado, **implementación diferida**

ADR-0022: Backend **IOCP** (Windows) — **implementado** y compile-verificado con MinGW (reemplaza ADR-0021)

ADR-0023: Backend **IOCP** (Windows) — **verificado en runtime** sobre Windows con MSVC (reemplaza ADR-0022)

ADR-0024: Compactación del log de Raft por **snapshot** (base de snapshot en `RaftLog`)

ADR-0025: Activación en producción del stack Raft + multi-reactor con transporte inter-nodo real

ADR-0026: Sharding por núcleo del plano de datos — partición → núcleo y coordinación de grupos por hash

ADR-0027: Cableado del modo proxy en el plano de datos — **pool** de conexiones aguas arriba por reactor

ADR-0028: Port **completo** de `nexusrd` a Windows — backend, afinidad y señales por plataforma

ADR-0029: Adaptador Kafka **asíncrono cross-core** sobre el broker vivo + codec por versión (interop `kcat`)

Prefacio

Este documento es la **documentación técnica final** de NexusMQ: la referencia autoritativa del *qué*, el *porqué* y el *cómo* del sistema, organizada para leerse de principio a fin o consultarse por partes.

Qué es este documento

NexusMQ es un *message broker* de mensajería distribuido de alto rendimiento, escrito en **C++23**, con arquitectura **shared-nothing thread-per-core** (un reactor por núcleo físico, *pinned*) y **Raft por partición** como mecanismo único de replicación y consenso. Es un **proyecto de aprendizaje y portfolio**: su objetivo nº1 es demostrar profundidad técnica en un sistema coherente y criterio de ingeniería, no acumular funcionalidades.

Esta documentación recoge ese sistema **tal como está construido** (*as-built*): la visión y el contexto, la arquitectura y sus principios, los contratos de red, la implementación por subsistemas, la estrategia de calidad, la operación y, por último, el registro de decisiones de arquitectura (ADR) y la historia de su evolución por fases.

A quién va dirigido

- **Al autor**, como mapa de construcción y memoria del diseño.
- **A revisores técnicos** (entrevistadores, colaboradores) que evalúen el proyecto como muestra de competencia en sistemas e infraestructura, C++ de bajo nivel y *backend* distribuido. Lo que se pretende evidenciar es **profundidad dentro de una sola tesis arquitectónica** —correctitud distribuida, latencia y funcionamiento *end-to-end*— y **decisiones justificadas y medidas**, no una lista de características.

Se asume familiaridad con conceptos de sistemas (E/S, concurrencia, memoria) y de datos distribuidos (replicación, consenso). Los términos de dominio se definen en el [Glosario](#).

Cómo está organizado

La documentación se divide en **siete partes** más **apéndices**:

Parte	Contenido
I — Introducción y visión	Resumen ejecutivo, contexto y motivación, estado del arte y glosario.
II — Arquitectura	Vista de conjunto, principios de diseño, concurrencia, modelo de I/O, almacenamiento, replicación y consenso, <i>ingress</i> y observabilidad.
III — Contratos	Protocolo binario nativo, subconjunto Kafka, API REST de administración y modelo de errores.
IV — Implementación	Mapa de módulos, catálogo por subsistema, arranque y <i>composition root</i> , herramientas y <i>bindings</i> .
V — Calidad	Estrategia de pruebas, puerta de calidad y CI/CD, rendimiento y <i>benchmarks</i> , portabilidad.
VI — Operación y despliegue	Despliegue, configuración y operación, seguridad.
VII — Decisiones y evolución	Registro de decisiones (ADR), historia de desarrollo y trabajo futuro.

Los **apéndices** recogen la bibliografía, el índice de diagramas y una guía rápida de construcción y ejecución.

Cada capítulo vive en su propio fichero Markdown numerado bajo `docs/tecnica/`. Los **diagramas** (Mermaid, tablas y *layouts* ASCII) viven en `docs/diagramas/` y se referencian desde la prosa.

Relación con los contratos as-built y con los ADR

Esta documentación distingue, siguiendo **Diátaxis**, dos tipos de texto:

- **Referencia** — los **contratos as-built** en `docs/`: el protocolo binario nativo (`protocol.md`), el subconjunto Kafka (`kafka.md`), la API REST de administración (`openapi.yaml`) y la metodología y cifras de rendimiento (`benchmarks.md`). Son la **única fuente de verdad** de los detalles precisos (byte-layouts, ApiKeys y versiones, *endpoints*, números). Se consultan.
- **Explicación** — los capítulos narrativos de `docs/tecnica/`. Se leen en orden. Los capítulos de contratos (Parte III) **explican y enlazan** —dan *rationale*, flujos, ejemplos y diagramas— pero **no duplican** el contrato: remiten a él para evitar divergencia (*drift*).

Las **decisiones de arquitectura** se registran como ADR, uno por archivo, en `docs/adr/` (ADR-0001 a ADR-0029). Un ADR aceptado **no se edita**: si una decisión cambia, se **reemplaza** por un ADR nuevo que marca al anterior como reemplazado. El registro narrado vive en el capítulo 28; los capítulos los referencian por su número (p. ej. `ADR-0005`).

Alcance y fuentes

Esta documentación técnica es la **fuentes de verdad** del proyecto: reparte y desarrolla la visión, la arquitectura y las decisiones por las siete partes, **extrae** los ADR a `docs/adr/` y se apoya en los **contratos as-built** de `docs/` (`protocol.md` , `kafka.md` , `openapi.yaml` , `benchmarks.md`). El código fuente en `src/` y la suite de pruebas son la referencia última cuando el detalle de implementación importa.

1. Resumen ejecutivo

Qué es NexusMQ, qué demuestra, cómo es su arquitectura en un vistazo y en qué estado se encuentra. Una página para situarse antes de entrar en el detalle.

Qué es

NexusMQ es un **broker de mensajería distribuido de alto rendimiento** escrito en **C++23**. Adopta el modelo de **log particionado** estilo Kafka: los productores **añaden** registros al final de un log *append-only* y los consumidores **leen** desde el *offset* que elijan y avanzan a su ritmo —el broker no borra al entregar, retiene por tiempo o tamaño—. Un *topic* es un flujo lógico dividido en **particiones**; cada partición es una secuencia ordenada e inmutable de registros, cada uno identificado por un *offset* monótono.

No pretende competir con Kafka en producción. Su propósito es **demostrar**, en una base de código propia y comprensible, las técnicas que hacen viable un sistema así. La arquitectura es la **frontera actual** del diseño de brokers (la que usa Redpanda), no la de hace una década.

Qué demuestra

El proyecto persigue **profundidad dentro de una sola tesis arquitectónica**, no amplitud. Lo que busca evidenciar es:

- **Profundidad técnica en un sistema coherente** — correctitud distribuida (consenso y replicación), latencia (camino caliente sin contención) y funcionamiento *end-to-end* (de la conexión del cliente al *fsync* en disco).
- **Criterio de ingeniería** — cada técnica avanzada se introduce **donde un *profiling* lo justifica** ("medido, no *checklist*"), y cada decisión de arquitectura queda registrada y argumentada como un [ADR](#).

Arquitectura en un vistazo

- **Shared-nothing thread-per-core**. Un **reactor por núcleo físico**, *pinned* a su CPU, dueño exclusivo de su anillo `io_uring`, su *allocator* y su subconjunto de particiones. **No hay estado mutable compartido** entre reactores: se comunican por **paso de mensajes** (colas SPSC *cross-core*). Elimina la clase de bug más cara —las carreras sobre estado compartido—. Ver [ADR-0005](#) y el diagrama [05 — topología thread-per-core](#).
- **Raft por partición**. Cada partición es su propio **grupo Raft**: su log replicado es el log de la partición (el WAL). El *high-watermark* es el `commitIndex` de Raft, y el modelo de `acks (0 / 1 / quorum)` se asienta sobre su *commit*. Postura **CP** (consistencia sobre disponibilidad; PACELC PC/EC). Ver [ADR-0003](#) y [ADR-0007](#).
- **I/O por *completions* (proactor)**. Abstracción común a `io_uring` en Linux —usado **directamente sobre el uapi del kernel**, sin `liburing` ([ADR-0012](#))— e **IOCP** en Windows. Sobre el proactor se asientan las **corutinas de C++23**. Ver [ADR-0002](#) y el diagrama [07 — secuencia proactor](#).
- **Protocolo binario propio** (framing + *multiplexing* por *correlation ID* + versionado) con **gateway REST** para administración y un **subconjunto Kafka-compatible** ya implementado (interop con `kcat`). Ver [ADR-0004](#) y [ADR-0029](#).

- **Capa de *ingress* en dos modos:** cliente nativo directo al líder (primario) + proxy/REST (*opt-in*). Ver [ADR-0006](#).

La solución son **15 librerías** `nexus-*` (common, io, wire, protocol, storage, reactor, consensus, cluster, broker, kafka, ingress, telemetry, server, client, ffi) más los ejecutables (`nexusd` , `nexus-cli` , `nexus-bench` , `nexus-loadgen`) y herramientas de soporte. La vista de conjunto está en el diagrama [02 — contenedores C4](#) y el grafo de dependencias en [03 — grafo de dependencias](#).

Estado

NexusMQ está construido **por fases**, cada una autocontenida y demoable. Las **fases 1 a 4 están implementadas**:

Fase	Nombre	Entregable
1	Storage engine	Motor de log monopartición con cifras de rendimiento. I/O bloqueante, sin reactor.
1b	Reactor + broker monolítico	Broker de un nodo <i>thread-per-core</i> : publicar y consumir con cliente nativo.
2	Distribución	Cluster tolerante a fallos (Raft por partición, <i>failover</i>).
3	Ingress + operación	Plataforma operable y observable (TLS, REST admin, CLI, métricas Prometheus).
4	<i>Stretch</i>	Productor idempotente, DLQ, compactación, LZ4/Zstd, <i>direct I/O</i> , subconjunto Kafka, IOCP (Windows).

El objetivo primario es **Linux x86-64**; el árbol completo compila y pasa la suite con GCC/libstdc++ y Clang/libc++, y el port a **Windows** está **verificado en runtime con MSVC** (ADR-0023/0028). El mapa fase → targets está en el diagrama [04 — mapa fase-targets](#).

Dónde están las cifras

Este documento **no fabrica números de rendimiento**. La metodología de medición —percentiles p50/p99/p999, generación de carga *open-loop* para evitar *coordinated omission*— y todos los resultados reproducibles viven en el contrato as-built `benchmarks.md` , explicado en el capítulo [Rendimiento y benchmarks](#).

2. Contexto y motivación

El problema que NexusMQ aborda, por qué se construye, qué objetivos persigue y —tan importante como lo anterior— qué queda explícitamente fuera de alcance.

2.1 El problema: la mensajería como columna vertebral

Los sistemas distribuidos modernos se construyen, cada vez más, alrededor de una **columna vertebral de mensajería asíncrona**. En lugar de que los servicios se llamen entre sí de forma síncrona y acoplada, **publican** y **consumen** eventos a través de un *broker* que los desacopla en el tiempo y en el espacio. Apache Kafka popularizó el modelo del **log distribuido append-only particionado** como sustrato común para *event streaming*, *event sourcing*, integración de datos y comunicación entre microservicios.

El modelo de log particionado resuelve varios problemas a la vez:

- **Desacoplo temporal** — productor y consumidor no necesitan estar vivos a la vez.
- **Re-lectura (*replay*)** — varios consumidores independientes pueden leer el mismo flujo, y rebobinar a un *offset* histórico.
- **Escalado horizontal** — por particiones, cada una unidad de paralelismo y de replicación.

Construir un broker de esta clase **desde cero** obliga a enfrentarse, sin atajos, a los problemas centrales de la ingeniería de sistemas: **durabilidad** ante fallos, **consenso y replicación**, **control de concurrencia** de alto rendimiento, **gestión explícita de memoria**, **I/O asíncrona** eficiente y **diseño de protocolos** de red. Es, por tanto, un vehículo idóneo para demostrar dominio de C++ moderno y de los fundamentos de los sistemas distribuidos.

2.2 Objetivos de aprendizaje y portfolio

NexusMQ es, ante todo, un **proyecto de aprendizaje y de portfolio**. No busca competir con Kafka en producción, sino **demostrar**, en una base de código propia y comprensible, las técnicas que hacen viable un sistema así.

La propuesta de valor del proyecto se sostiene en cuatro diferenciadores:

- **Arquitectura *shared-nothing thread-per-core* con reactor propio**. No hay *thread-pool* con estado compartido bajo *locks*: cada núcleo es un reactor independiente (su *event loop* *io_uring*, su *allocator*, su subconjunto de particiones) y se comunican por **paso de mensajes** explícito. Es la frontera actual del diseño de brokers, no la de hace más de una década (ver [estado del arte](#)).
- **Capa de *ingress* integrada** —no como pieza externa (HAProxy/Nginx) sino como parte del producto— y con **dos modos** explícitos (cliente nativo directo al líder + proxy/REST), demostrando criterio en vez de "un *ingress* que lo hace todo".
- **C++ de alto rendimiento con criterio**. Cada técnica avanzada (lock-free, SIMD, *allocators* por núcleo, *io_uring*) se introduce **donde un *profiling* lo justifica** y se acompaña de su *benchmark* antes/después, con metodología anti-*coordinated-omission*.
- **Portabilidad por diseño**. Linux primero, con una abstracción de I/O (*proactor*) preparada para un backend Windows (IOCP) que más tarde se completó y verificó en runtime.

El proyecto busca evidenciar **profundidad técnica en un sistema coherente** (correctitud distribuida + latencia + funcionamiento *end-to-end*) y **criterio** (decisiones justificadas y medidas), por encima de la mera acumulación de funcionalidades. La ambición se persigue como **profundidad dentro de una sola tesis arquitectónica**, nunca como amplitud: dos arquitecturas en paralelo sería *scope sprawl*, no ambición.

Casos de uso objetivo

El diseño se valida contra cuatro patrones de uso representativos:

- **Streaming de eventos** entre microservicios (publicación/suscripción durable).
- **Cola de trabajo** con grupos de consumidores y reparto de carga.
- **Event sourcing / auditoría** — log inmutable re-leíble desde cualquier *offset*.
- **Buffer de absorción de picos** (*backpressure*) entre sistemas de ritmos distintos.

2.3 Objetivos específicos (OE-1..OE-7)

El objetivo general es diseñar e implementar, por fases, un broker de mensajería distribuido de alto rendimiento en C++23, con arquitectura *shared-nothing thread-per-core*, persistencia durable, replicación con consenso (Raft por partición) y una capa de *ingress* inteligente, acompañado de herramientas de operación y observabilidad. Se desglosa en siete objetivos específicos:

- **OE-1.** Construir un *storage engine* de log *append-only* con segmentación, índice disperso, *checksums* (CRC32C) y política de durabilidad configurable.
- **OE-2.** Definir e implementar un **protocolo binario propio** con *framing*, *multiplexing* y versionado, una librería cliente nativa en C++ y un **gateway REST** para interoperabilidad.
- **OE-3.** Implementar el núcleo del broker sobre un **reactor thread-per-core propio**: *topics*, particiones (cada una anclada a un núcleo), grupos de consumidores y semánticas de entrega.
- **OE-4.** Dotar al sistema de **replicación con consenso mediante Raft por partición** (elección de líder y *failover* automático) para tolerancia a fallos.
- **OE-5.** Construir la **capa de ingress** en dos modos (cliente nativo directo al líder + proxy/ REST), con terminación TLS, *rate limiting*, *circuit breaker*, *health checks* y *load balancing*.
- **OE-6.** Proporcionar **observabilidad** de grado producción (métricas Prometheus, logs estructurados, *tracing*) y herramientas de administración (API REST + CLI).
- **OE-7.** Demostrar, con **benchmarks reproducibles y metodológicamente rigurosos**, el impacto de las técnicas de alto rendimiento aplicadas (throughput y latencia con percentiles, sin *coordinated omission*).

2.4 Alcance funcional

El sistema se organiza en seis bloques funcionales:

1. **Ingress Layer** — proxy/gateway en dos modos: TLS 1.3, *rate limiting* (*token bucket*), *circuit breaker*, *health checks*, *load balancing*, *retry* con *backoff* + *jitter*, y gateway REST.
2. **Core Broker** — *topics*, particiones (ancladas a núcleo), grupos de consumidores, gestión de *offsets*, semánticas de entrega, DLQ, TTL, *batching*/compresión, *backpressure* por créditos.
3. **Consenso y replicación** — Raft por partición: elección de líder, replicación de log, *failover* automático, *high-watermark* derivado del `commitIndex` de Raft.
4. **Protocolo de red** — *framing*, *multiplexing*, versionado; librería cliente C++ nativa + gateway REST.
5. **Storage engine** — WAL, segmentos, índice disperso, *mmap*/io_uring, retención y compactación.

Requisitos funcionales (selección)

- **RF-01.** Un productor publica mensajes en un *topic*/partición y recibe confirmación con el *offset* asignado y la política de *ack* solicitada.
- **RF-02.** Un consumidor lee desde un *offset* arbitrario y avanza (*commit* manual y automático).
- **RF-03.** Los mensajes persisten de forma durable y sobreviven a un reinicio del broker.
- **RF-04.** Los grupos de consumidores reparten particiones entre miembros con rebalanceo.
- **RF-05.** El sistema replica cada partición (grupo Raft) y elige un nuevo líder automáticamente ante la caída del actual, **sin pérdida de datos *committed***.
- **RF-06.** La capa de *ingress* aplica *rate limiting* y *circuit breaking* por cliente/*topic*, y ofrece un gateway REST.
- **RF-07.** Operación vía API REST y CLI; métricas expuestas en formato Prometheus.

Requisitos no funcionales (cualitativos)

Las cifras objetivo (throughput, latencias por percentil, retención) son orientativas y se fijan con la metodología del capítulo [Rendimiento y benchmarks](#); las cifras reales viven en [benchmarks.md](#). Los requisitos cualitativos son:

- **Durabilidad / ACK.** Modelo de *ack* configurable por petición: `acks=0` (sin esperar, *at-most-once*), `acks=1` (escrito en el líder) y `acks=quorum` (replicado y *committed* por la mayoría del grupo Raft; **por defecto**). Ningún mensaje confirmado con `acks=quorum` se pierde mientras sobreviva la mayoría del grupo. La política de `fsync` (cada N mensajes / N ms / por *commit*) es ortogonal y combinable.
- **Consistencia / Disponibilidad.** Postura **CP** ([ADR-0007](#)): ante partición de red, una partición sin quórum **deja de aceptar escrituras** antes que arriesgar divergencia. *Failover* automático sin pérdida de datos *committed* por el quórum.
- **Seguridad.** TLS 1.3 en el borde (terminación en *ingress*) y opción de mTLS intra-cluster; autenticación de cliente y autorización básica por *topic*.
- **Observabilidad.** Métricas, logs estructurados y *tracing* con *correlation IDs*.
- **Portabilidad.** Núcleo independiente de plataforma tras la abstracción de I/O (*proactor*); Linux primero, Windows después.
- **Mantenibilidad.** Separación estricta plano de datos / plano de control; cobertura de pruebas en la lógica de dominio.

2.5 Fuera de alcance (explícito)

Para acotar el proyecto y evitar la dispersión —el riesgo número uno de un proyecto "infinito"—, **quedan fuera** de forma deliberada:

- **Compatibilidad total con el protocolo de Kafka.** Se aborda solo un **subconjunto** (`ApiVersions` / `Metadata` / `Produce` / `Fetch`) como *stretch* de Fase 4, suficiente para que `kcat` hable con el broker; sin grupos de consumidores Kafka (`JoinGroup` / `Heartbeat`) ni transacciones. Ver [ADR-0004](#) y [ADR-0029](#).
- **Exactly-once transaccional entre particiones.** Sí se contempla **productor idempotente** (*effectively-once* por partición).
- **Tiered storage** a almacenamiento de objetos (idea futura, inspirada en Pulsar).

- **Multi-tenancy y ACLs avanzadas; cifrado en reposo.**
- **Dashboard gráfico propio.** La observabilidad se expone vía Prometheus/CLI; un panel Grafana es opcional y *self-hosted*.
- **Despliegue de pago en cloud.** El proyecto se **orienta** a Azure/AWS por diseño (imagen Docker, configuración 12-factor), pero su desarrollo, prueba y *benchmark* **no requieren** gasto en cloud: el cluster de 3 nodos se levanta en local con Docker Compose. Ver [ADR-0008](#).

El detalle de las limitaciones aceptadas y la evolución posible se trata en el capítulo [Limitaciones y trabajo futuro](#).

3. Estado del arte

Dónde se sitúa NexusMQ en el mapa de los *brokers* de mensajería: qué hereda del modelo clásico de log particionado y por qué adopta la **frontera actual** (*thread-per-core* + Raft por partición) en lugar del diseño de hace más de una década.

3.1 El modelo de log particionado

Apache Kafka popularizó el **log distribuido append-only particionado** como sustrato común de *event streaming*, *event sourcing*, integración de datos y comunicación entre microservicios. Ese modelo —*topics* divididos en **particiones**, cada una un log inmutable de registros identificados por **offset** monótono, con consumidores que leen desde el offset que eligen— es la base que NexusMQ **hereda** (ver el [capítulo 9, Almacenamiento](#)). De Kafka se toman también el **high-watermark**, los **grupos de consumidores**, la **retención** y el **formato de record batch v2**.

3.2 Dos generaciones de arquitectura

La diferencia entre NexusMQ y un Kafka clásico no está en *qué* es (un log particionado), sino en *cómo* se ejecuta por dentro:

Dimensión	Kafka clásico (≈2011)	Frontera actual (Redpanda)	NexusMQ
Concurrencia	<i>thread-pool</i> + estado compartido bajo <i>locks</i>	<i>shared-nothing thread-per-core</i>	<i>thread-per-core</i> (reactor propio)
Replicación	ISR + <i>controller</i> (consenso solo para metadatos)	Raft por partición	Raft por partición
I/O	JVM + <i>page cache</i> del SO	<i>io_uring</i> (Seastar)	proactor <i>io_uring</i> /IOCP + corutinas C++23
Lenguaje	Scala/Java (JVM, GC)	C++ (Seastar)	C++23 (reactor mínimo propio)

El salto de generación es deliberado: NexusMQ no se posiciona en la frontera de 2011 (*thread-pool* + ISR clásico), sino en la **frontera actual**, la que usa **Redpanda** para batir a Kafka en latencia. Es el diseño más ambicioso, el más coherente con el objetivo de *correctitud demostrable* y el que mejor compone consigo mismo: un grupo Raft anclado a un núcleo, un anillo *io_uring* por núcleo, sin estado mutable compartido.

3.3 Análisis de referentes

Sistema	Lenguaje	Modelo	Qué se toma como referencia
Apache Kafka	Scala/Java	Log particionado + ISR; KRaft para metadatos	Modelo de log, particiones, grupos de consumidores, <i>high-watermark</i> , retención, <i>record batch</i> .
Redpanda	C++ (Seastar)	Kafka-compatible; Raft por partición ; <i>thread-per-core</i> / <i>shared-nothing</i>	Validación directa del diseño adoptado: un broker <i>thread-per-core</i> + Raft por partición es viable y <i>state-of-the-art</i> en C++.
RabbitMQ	Erlang	Colas AMQP, <i>routing</i> por <i>exchange</i>	Semánticas de entrega, <i>dead letter queues</i> , ACK/NACK.
NATS / JetStream	Go	Mensajería ligera + <i>streams</i> durables	Simplicidad del protocolo, baja latencia, <i>at-least-once</i> .
Apache Pulsar	Java	<i>Compute/storage</i> separados (BookKeeper)	Separación de capas, <i>tiered storage</i> (idea para <i>stretch</i>).

3.4 Posicionamiento de NexusMQ

- **Redpanda como validación de diseño, no como dependencia.** Redpanda demuestra que la arquitectura *thread-per-core* + Raft por partición es construible en C++ con rendimiento de primera línea. NexusMQ construye su **propio reactor mínimo** sobre `io_uring` + corutinas; **no usa Seastar** (dependencia pesada, *Linux-only*): "construir" demuestra más que "usar" y encaja con la política de hacer el núcleo a mano (ver [ADR-0005](#)).
- **No compite con Kafka en producción.** El objetivo es **demostrar**, en una base de código propia y comprensible, las técnicas que hacen viable un sistema así —profundidad técnica y criterio—, no acumular funcionalidades ni superar a Kafka en *features*.
- **Compatibilidad Kafka como subconjunto recuperable.** En vez de atarse a la especificación completa de Kafka, se implementa un **subconjunto** Kafka-compatible como *stretch* de Fase 4 (interop con `kcat`), preservando el control del protocolo binario propio (ver [ADR-0004](#) y el [capítulo 14](#)).

4. Glosario

Define los términos de dominio que el resto de la documentación usa sin más aclaración. Identificadores y términos técnicos asentados van en inglés (es la convención del ecosistema); la definición, en español.

Modelo de log y mensajería

- **Topic.** Flujo lógico de mensajes, dividido en una o más **particiones**.
- **Partición.** Unidad de paralelismo y de replicación: un **log append-only** ordenado e inmutable. En NexusMQ, además, **un grupo Raft** y el dueño de un núcleo concreto.
- **Offset.** Identificador monótono de un registro dentro de una partición; da el orden (no el *timestamp*).
- **Record / RecordBatch.** Un registro es la unidad de mensaje (clave, valor, cabeceras, *timestamp*); el **RecordBatch** (formato v2, estilo Kafka) agrupa varios registros con un *checksum* y metadatos comunes. Ver [contrato](#).
- **Segmento.** Tramo físico del log en disco: un fichero `.log` (registros) + un `.index` (índice disperso offset → posición). El log de una partición es una secuencia de segmentos.
- **Índice disperso.** Índice que mapea algunos offsets a posiciones de fichero (no todos), para acotar su tamaño; la búsqueda fina se completa con un escaneo corto.
- **WAL (Write-Ahead Log).** Log de escritura previa que garantiza durabilidad. En NexusMQ el **log de Raft es el WAL** de la partición.
- **High-watermark (HWM).** Offset hasta el cual los datos están *committed* (replicados por el quórum) y, por tanto, visibles a los consumidores. Se deriva del `commitIndex` de Raft.
- **acks.** Política de confirmación de una escritura: `acks=0` (sin esperar, *at-most-once*), `acks=1` (escrito en el líder), `acks=quorum` (replicado y *committed* por la mayoría del grupo Raft; valor por defecto).
- **Grupo de consumidores.** Conjunto de consumidores que se reparten las particiones de un *topic* con **rebalanceo**, para escalar el consumo.
- **DLQ (Dead Letter Queue).** Destino de los mensajes no procesables tras N intentos.
- **Productor idempotente.** Productor con *producer-id* + *sequence* que evita duplicados por reintento (*effectively-once* por partición).
- **Retención / compactación.** Borrado de datos antiguos por tiempo/tamaño (retención) o conservación del último valor por clave (compactación).

Consenso y distribución

- **Raft.** Algoritmo de consenso (Ongaro & Ousterhout) para replicar un log de forma tolerante a fallos con un líder elegido por mayoría.
- **commitIndex.** Índice del log de Raft hasta el cual las entradas están *committed*.
- **Quórum.** Mayoría de un grupo Raft ($\lfloor n/2 \rfloor + 1$), necesaria para *commit* y para elegir líder.
- **Leader epoch.** Número que identifica el mandato de un líder; detecta y descarta líderes obsoletos.
- **Snapshot (Raft).** Imagen/base compactada del estado que permite truncar el prefijo del log replicado y acotar su tamaño (ver [ADR-0024](#)).

- **PACELC.** Marco que extiende CAP: ante **P**artición se elige **A** o **C**; en operación normal (**E**lse), **L**atencia o **C**onsistencia. NexusMQ es **PC/EC**.
- **Failover.** Sustitución automática de un líder caído por un nuevo líder elegido.

Concurrencia, memoria y sistemas

- **Shared-nothing.** Diseño sin estado mutable compartido entre unidades de ejecución; se comunican por **paso de mensajes**.
- **Thread-per-core.** Un hilo (reactor) por núcleo físico, *pinned* a su CPU.
- **Reactor / Proactor.** Modelos de I/O asíncrona: *reactor* basado en *readiness* (epoll); *proactor* basado en *completions* (io_uring/IOCP). NexusMQ usa **proactor**.
- **io_uring / IOCP.** Interfaces de I/O por *completions* de Linux y Windows. NexusMQ usa io_uring **directo sobre el uapi del kernel**, sin liburing ([ADR-0012](#)).
- **Cross-core message passing.** Comunicación entre reactores por colas **SPSC** con despertar del destino (estilo `submit_to`).
- **SPSC.** *Single-producer single-consumer*: cola con un único productor y un único consumidor (caso simple y rápido).
- **Backpressure (por créditos).** Control de flujo en el que el receptor concede *créditos*; sin créditos, el emisor se frena (colas acotadas).
- **Afinidad (pinning).** Fijar un hilo a un núcleo concreto para localidad de caché y NUMA.
- **NUMA.** *Non-Uniform Memory Access*: la memoria local al nodo del núcleo es más rápida.
- **False sharing.** Degradación por compartir una línea de caché entre hilos que escriben; se mitiga con `alignas(std::hardware_destructive_interference_size)`.
- **CRC32C.** Variante de CRC de 32 bits con instrucción hardware (SSE4.2), usada como *checksum*.
- **Direct I/O.** I/O que evita el *page cache* del SO (`O_DIRECT`), gestionando caché/alineación propias.
- **Reloj monotónico.** Reloj que nunca retrocede; se usa para *timeouts*/heartbeats/elección Raft (nunca el *wall-clock*).

Modelo de errores y observabilidad

- `expected<T>`. `std::expected<T, Error>` (C++23): resultado-o-error como valor, usado en el núcleo/hot-path ([ADR-0009](#)).
- **errorCode (wire).** Código de error numérico (`i16`) del protocolo; es **contrato**, se traduce al modelo interno en el borde.
- **RFC 7807 (ProblemDetail).** Formato estándar de error para la API REST de administración.
- **Liveness / Readiness.** Estar vivo (el proceso responde) vs estar listo para tráfico (*checks* de disco/Raft/lag).
- **Coordinated omission.** Sesgo de medición de latencia de los clientes *closed-loop*; se evita con un generador *open-loop* (ver [capítulo 23](#)).

Otros patrones

- **Circuit breaker.** Patrón que "abre" el circuito ante fallos para no insistir contra un destino caído.
- **Token bucket.** Algoritmo de *rate limiting* basado en una cubeta de *tokens* que se rellena a tasa fija.
- **Decompression bomb.** *Batch* comprimido que se expande desmesuradamente al descomprimir; se mitiga con límites de ratio/tamaño.

5. Vista de conjunto

El sistema a vista de pájaro: cómo se descompone NexusMQ en librerías, cómo fluyen sus dependencias y cómo se mapea al modelo C4. Es el mapa que sitúa los capítulos siguientes.

5.1 Un nodo, N reactores

Cada **nodo** del broker es un proceso (imagen Docker) que ejecuta **N reactores** (uno por núcleo físico, *pinned*) y mantiene un subconjunto de **réplicas de partición** —líder de unas, seguidor de otras—, donde **cada partición es su propio grupo Raft**. El cluster de referencia son **3 nodos** (tolera la caída de uno con quórum de 2). En desarrollo, los 3 nodos se levantan con Docker Compose en una sola máquina (ver [capítulo 25](#)). El contexto y los contenedores se ilustran en los diagramas [1](#) y [2](#).

5.2 Las 15 librerías `nexus-*`

La solución es un único árbol CMake con **15 librerías** `nexus-*`, más ejecutables y herramientas. Por capas (de la base hacia arriba):

Capa	Librerías	Responsabilidad
Base	<code>nexus-common</code>	Tipos, <code>bytes</code> , <code>varint</code> , <code>crc32c</code> , <code>record</code> , <code>error</code> , <code>task</code> (corutinas), compresión.
Plataforma	<code>nexus-io</code> , <code>nexus-wire</code>	Proactor (<code>io_uring</code> / <code>IOCP</code>), <code>File</code> / <code>Socket</code> ; framing sobre conexión.
Dominio E/S	<code>nexus-protocol</code> , <code>nexus-storage</code>	Protocolo binario (codec, frames, versionado); motor de log (segmentos, índice).
Ejecución	<code>nexus-reactor</code>	Reactor por núcleo, colas SPSC cross-core, <code>PartitionRouter</code> .
Consenso	<code>nexus-consensus</code> , <code>nexus-cluster</code>	Raft (FSM, log, RPC); transporte inter-nodo.
Broker	<code>nexus-broker</code> , <code>nexus-kafka</code>	Topics, particiones, grupos, offsets; subset Kafka.
Bordes	<code>nexus-ingress</code> , <code>nexus-telemetry</code>	Dos modos de ingress, TLS, REST admin; métricas/logs.
Cima	<code>nexus-server</code> , <code>nexus-client</code> , <code>nexus-ffi</code>	Composición de <code>nexusrd</code> ; cliente nativo; ABI C para bindings.

Ejecutables y *tools*: `nexusrd` (el servidor, `src/server/main.cpp`), `nexus-cli`, `nexus-bench`, `nexus-loadgen` y `wincheck` (arnés Windows-only). El detalle por subsistema está en el [capítulo 18](#).

5.3 Regla de dependencia

Las dependencias **apuntan hacia el núcleo**: una librería nunca depende de otra de su misma capa o superior. `nexus-common` no depende de nada interno; `nexus-server` está arriba y compone casi todo. Esta

disciplina (clean architecture aplicada a sistemas) mantiene el núcleo —storage, broker, consenso— **independiente del detalle de I/O y de protocolo**, que entran por *puertos* y *adaptadores* (DIP). El grafo completo está en el [diagrama 3](#) y se detalla en el [capítulo 17](#).

5.4 Modelo C4 (resumen)

- **Contexto:** productores y consumidores ↔ NexusMQ ↔ operadores (CLI/REST) y Prometheus; otros nodos del cluster por el plano Raft.
- **Contenedores:** ingress (dos modos), nodos de broker (N reactores *per-core* c/u), almacén en disco, plano de administración.
- **Componentes (por reactor):** *event loop* del proactor → *handlers* de protocolo (corutinas) → broker (particiones del núcleo) → storage engine → módulo Raft de esas particiones; colas SPSC *cross-core* hacia otros reactores.

5.5 Planos del sistema

NexusMQ separa explícitamente tres planos, lo que se refleja en su despliegue y en sus puertos (ver [diagrama 15](#)):

- **Plano de datos / cliente:** *produce/fetch* por el protocolo binario (o subset Kafka) contra el líder de cada partición. Es el camino caliente.
- **Plano de consenso / inter-nodo:** los RPC de Raft (`AppendEntries` / `RequestVote`) entre réplicas, por un puerto separado.
- **Plano de control / administración:** API REST (`/api/v1`), salud y métricas.

La separación plano de datos/plano de control es un principio rector: el camino caliente se mantiene libre de lógica de administración (ver [capítulo 6](#)).

6. Principios de diseño

Los principios que vertebran NexusMQ y que explican por qué el código está como está. Especializan al dominio de sistemas los principios generales (SOLID, clean architecture) sin duplicarlos.

6.1 Principios rectores

- **Separación plano de datos / plano de control.** El camino caliente (*produce/fetch*) se mantiene libre de lógica de administración; la gestión (crear *topics*, rebalanceos) vive en un plano aparte (REST admin).
- **Shared-nothing.** El estado mutable es **propiedad de un único reactor**; nada se comparte entre núcleos salvo por paso de mensajes. Es el principio que hace el sistema razonable y testeable (ver [capítulo 7](#)).
- **Hexagonal aplicada a sistemas.** El núcleo (storage, broker, consenso) no depende del detalle de I/O ni de protocolo: estos entran por **puertos** (interfaces) y **adaptadores** (p. ej. `io_uring` vs IOCP es un adaptador del puerto de I/O). Materializa **DIP** y permite romper ciclos de capas ([ADR-0017](#), [ADR-0018](#)).
- **Mecanismo vs política.** El storage ofrece *mecanismo* (append, read, fsync); la *política* (retención, durabilidad, `acks`) es configurable y externa.

6.2 Las decisiones de arquitectura, en una frase cada una

Los principios anteriores se concretan en decisiones registradas como ADR (ver [capítulo 28](#)). Las que más condicionan el diseño:

- **Concurrencia *shared-nothing thread-per-core*** con reactor propio ([ADR-0005](#)): elimina la clase de bug más cara (carreras sobre estado compartido) y los *locks* del camino caliente.
- **Raft por partición** ([ADR-0003](#)): un único mecanismo de ordenación, replicación y elección por partición; el log de Raft es el WAL; el *high-watermark* es el `commitIndex`.
- **Proactor / completions** ([ADR-0002](#), [ADR-0012](#)): I/O modelada como *completions* (`io_uring`/IOCP), base portable sobre la que se asientan las corutinas C++23.
- **Postura CP / PACELC PC-EC** ([ADR-0007](#)): ante partición se elige consistencia; en operación normal también consistencia sobre latencia.
- **Modelo de errores por capa** ([ADR-0009](#)): `expected<T>` en el núcleo, excepciones en el plano de control, códigos de wire en el protocolo (ver [capítulo 16](#)).

6.3 Patrones arquitectónicos y de almacenamiento

- **Reactor/Proactor** (I/O por *completions*, un reactor por núcleo).
- **Leader-Follower vía Raft**: cada partición tiene un líder que ordena las escrituras y *followers* que replican.
- **Pipeline**: recepción → decodificación → *append* → replicación → *ack*, como etapas conectadas por colas.
- **WAL + log-structured storage**: escritura secuencial append-only; **el log de Raft es el WAL**. **Segmentación + índice disperso** para *seek* y retención por segmentos enteros; **compactación por clave** como política alternativa.

6.4 Patrones distribuidos y de resiliencia

- **Consenso (Raft) por partición**, con extensiones contempladas: *pre-vote*, *leadership transfer* y réplicas *learner*.
- **Productor idempotente** (*producer-id + sequence*) para *effectively-once* por partición; **DLQ** para mensajes no procesables.
- **Resiliencia del ingress**: *Circuit Breaker* (closed/open/half-open), *Bulkhead*, *retry* con *backoff + jitter*, *rate limiting* (token bucket), **backpressure por créditos** (colas acotadas *end-to-end*; sin créditos, el productor se frena, no se descarta).

6.5 Disciplina de C++ y calidad

Transversal a todo el código: **RAII estricto** (sin `new / delete` crudos; recursos del SO envueltos en tipos), semántica de valor, `const` -correcto, `enum class`; corutinas que **nunca bloquean** el reactor; y una **puerta de calidad** (dos compiladores, sanitizers, `clang-tidy`, `clang-format`) obligatoria antes de cada *commit* (ver [capítulo 22](#)). El detalle de convenciones lo fija la `BibliotecaDocumentacion`, autoridad de estilo del proyecto.

7. Concurrency

El modelo de ejecución de NexusMQ: por qué no hay *locks* en el camino caliente y cómo se reparte el trabajo entre núcleos sin estado compartido. Decisiones de fondo: [ADR-0005](#) y [ADR-0026](#).

7.1 Un reactor por núcleo

NexusMQ ejecuta **un reactor por núcleo físico**, cada uno *pinned* a su CPU. Un reactor es dueño **exclusivo** de: su **anillo io_uring** (o IOCP en Windows), su **allocator** (arena/pool local, NUMA-aware) y un **subconjunto de réplicas de partición**. **No hay estado mutable compartido entre reactores**. Esto elimina la clase de bug más cara —las carreras sobre estado compartido— y, con ella, los *locks* del camino caliente: cada partición es una **unidad de serialización** (estilo *actor*) anclada a un núcleo. La topología se ilustra en el [diagrama 5](#).

7.2 Sharding: qué núcleo es dueño de qué

El reparto de carga no se hace por *work-stealing* sobre una cola compartida, sino por **asignación de particiones a núcleos** ([ADR-0026](#)):

- **Partición → núcleo:** `partition % N` (N = número de reactores). El reactor resultante es el **único dueño** de esa réplica.
- **Grupo de consumidores → núcleo:** `hash(group_id) % N` (FNV-1a), para que la coordinación de cada grupo viva siempre en el mismo reactor.

Como cada *shard* tiene **un único dueño**, las operaciones sobre él son **linealizables sin locks**: las sirve el reactor propietario, en orden, sin sincronización. Ver el [diagrama 6](#).

7.3 Paso de mensajes cross-core

Cuando una petición llega al núcleo A pero su partición vive en B, **no** se accede a estado de B: la petición se **reenvía** a B por una cola **SPSC** (*single-producer single-consumer*), con despertar del destino (estilo `submit_to` de Seastar). El resultado vuelve por el mismo mecanismo. SPSC es el caso simple y rápido (un *ring buffer* basta); el diseño minimiza productores/consumidores por cola precisamente para evitar la complejidad (CAS, ABA) de las colas MPMC.

- **Lock-free solo con motivo y medición.** Donde hay colas, se usa CAS con el `memory_order` mínimo correcto (`acquire / release` en productor-consumidor); en las MPMC se resuelve el problema **ABA** (contadores de versión / *hazard pointers* / *epoch-based reclamation*) y se valida con **ThreadSanitizer + estrés aleatorio** en CI.
- **False sharing.** Los campos escritos por hilos distintos (p. ej. *head/tail* de cada cola) se separan a líneas de caché distintas con `alignas(std::hardware_destructive_interference_size)`.

7.4 Backpressure por créditos

Las colas son **acotadas end-to-end**. El receptor concede **créditos** al emisor; cuando se agotan, el emisor se **frena** en lugar de descartar mensajes o dejar crecer los *buffers* sin límite. Esto evita el colapso bajo sobrecarga y mantiene las latencias de cola acotadas (`ConnectionState.credits` , ver [capítulo 8](#)).

7.5 Nada bloqueante en el reactor

La contrapartida del modelo es **disciplina asíncrona total**: un `fsync` síncrono, un `malloc` que entre al kernel o un *lock* contendido **congelan el núcleo entero** y con él todas las tareas que multiplexa. Por eso **todo** I/O (incluido `fsync`) pasa por el proactor y se usan *allocators* por núcleo para no entrar al kernel en el camino caliente. Las corutinas del reactor nunca hacen `co_await` sobre operaciones bloqueantes (ver [capítulo 8](#)).

7.6 Reloj para la concurrencia distribuida

Todo lo que mide **intervalos** y no puede retroceder —*timeouts*, *heartbeats* y el *election timeout* de Raft, *backoff*— usa el **reloj monotónico** (`steady_clock`), nunca el *wall-clock*: un salto de NTP hacia atrás dispararía elecciones espurias. El *wall-clock* se reserva para *timestamps* de `Record` y retención por tiempo. El orden dentro de una partición lo da el **offset**, no la marca de tiempo.

8. Modelo de I/O

Cómo NexusMQ habla con disco y red sin bloquear nunca un núcleo. El plano de datos es, en esencia, **I/O de red y disco a gran escala**; el modelo elegido es un **proactor** (*completions*) sobre el que se asientan las corutinas C++23.

8.1 Proactor, no reactor

Se adopta una abstracción de **proactor** (modelo de *completions*: se envía una operación asíncrona y se recibe su finalización con el resultado), no un *reactor* *epoll* (*readiness*). El motivo es la **portabilidad**: *completions* es la forma común a los dos backends objetivo, **io_uring** (Linux) e **IOCP** (Windows) ([ADR-0002](#)). El puerto `Proactor` y los tipos portables `NativeHandle / IoResult` viven en `nexus-io`; los backends concretos son adaptadores (`io_uring_backend`, `iocp_backend`). La secuencia *submit* → *completion* → *reanudar corutina* está en el [diagrama 7](#).

8.2 io_uring directo, sin liburing

En Linux, el backend usa **io_uring directamente sobre el uapi del kernel, sin liburing** ([ADR-0012](#)): el proyecto gestiona sus propios anillos de *submission/completion*. Bajo *thread-per-core* hay **un anillo por núcleo**, sin compartición entre reactores, lo que encaja con el modelo *shared-nothing* y amortiza syscalls por *batching* (coherente con "no una syscall por byte"). En Windows, el backend IOCP se construye sobre `GetQueuedCompletionStatusEx / AcceptEx` y está verificado en runtime con MSVC (ver [capítulo 24](#)).

8.3 Corutinas sobre el proactor

Las **corutinas de C++23** se asientan de forma natural sobre el proactor: un `co_await` de una operación de I/O suspende la corutina y la reanuda en su *completion*, con el resultado. El código del camino caliente queda **secuencial y legible** sobre I/O por *completions*. Las operaciones del núcleo devuelven `task<expected<T>>`: combinan la asincronía (`task`) con el modelo de errores como valor (`expected`, ver [capítulo 16](#)). Hay que cuidar el *lifetime* de los argumentos a través de la suspensión: una corutina puede reanudarse en otro momento, así que no se hace `co_await` sobre temporales colgantes.

8.4 Nada bloqueante en el reactor

La regla de oro: **un solo `fsync`, `malloc` que entre al kernel o `lock` contendido congela el núcleo entero**. Por eso:

- **Todo** I/O de disco —incluido `fsync / fdatasync`— pasa por `io_uring` asíncrono (`IORING_OP_FSYNC`), no por llamadas bloqueantes.
- Se usan **allocators por núcleo** (arena/pool) para no entrar al kernel a pedir memoria en el camino caliente.
- El control de flujo es por **créditos** (colas acotadas), no por *buffers* ilimitados (ver [capítulo 7](#)).

8.5 Durabilidad y checksums

- **write no persiste.** La durabilidad se fuerza con `fsync / fdatasync` ; como su coste es alto, se **agrupa** (por N mensajes / N ms / por *commit*), nunca por escritura. La política de `fsync` es configurable y ortogonal al modelo de `acks` .
- **Checksums.** Cada `RecordBatch` lleva un **CRC32C** que cubre el batch completo; al leer y al recuperar tras *crash* se verifica, detectando corrupción silenciosa (*bit rot, torn writes*) que el WAL por sí solo no ve. CRC32C usa la instrucción hardware de SSE4.2 con *fallback* software (ver [capítulo 9](#)).
- **Recuperación.** Al arrancar se valida el CRC de la cola del log y se **trunca la cola torn** (escritura a medias por un *crash*), dejando el log en un estado consistente.

8.6 Evolución escalonada de la I/O de storage

La I/O de disco se introdujo por fases (decisión deliberada de [ADR-0002](#)): la **Fase 1** arrancó con I/O de fichero **bloqueante** (`pread / pwrite + fsync`) detrás de la abstracción `File` , para validar correctitud y durabilidad antes que rendimiento; a partir de la **Fase 1b**, bajo el reactor, el disco pasó a `io_uring` asíncrono. **Direct I/O** (`O_DIRECT`) con caché y *readahead* propios quedó como **profundidad opcional** de Fase 4, solo si el *profiling* lo justifica (ver [capítulo 23](#)).

9. Almacenamiento

El *storage engine*: un log particionado append-only sobre segmentos, con índice disperso, *checksums* y retención. Es la Fase 1 del proyecto y la base sobre la que se monta Raft. El layout se ilustra en el [diagrama 8](#).

9.1 Log particionado

Cada partición es un **log append-only**: una secuencia ordenada e inmutable de registros, cada uno con un **offset** monótono. Los productores añaden al final; los consumidores leen desde el offset que elijan y avanzan a su ritmo (el broker no borra al entregar: retiene por tiempo/tamaño). Una partición es físicamente una **secuencia de segmentos**:

```
Partición 0 → [Seg 0 (closed)][Seg 1 (closed)][Seg 2 (active)]    cada Seg: .log + .index
```

`PartitionLog` reúne los segmentos, el segmento activo, `logStartOffset` / `logEndOffset` y el `recoveryPoint` (último `fsync`). Es **REACTOR-LOCAL**: pertenece al reactor dueño de la partición y no es *thread-safe* (su invariante es que `logEndOffset` es monótono).

9.2 Segmentos e índice disperso

Cada **segmento** (`Segment`) es un par de ficheros: `.log` (los `RecordBatch`) y `.index` (el índice disperso). El **índice disperso** mapea, cada N bytes, `relativeOffset` → posición en fichero (`IndexEntry { relativeOffset:u32, filePosition:u32 }`), lo que permite un `seek` eficiente sin recorrer todo el segmento: se localiza la entrada de índice más cercana por debajo del offset buscado y se escanea hacia delante un tramo corto.

Ciclo de vida de un segmento: `active` (recibe *appends*) → `closed` (alcanza `segment.bytes` / `segment.ms`; se sella y se finaliza su `.index`) → `eligible` (supera la retención) → `deleted`. La retención **nunca** borra el segmento activo.

9.3 RecordBatch v2

La unidad de escritura y replicación **no** es el registro suelto, sino el **RecordBatch** (estilo Kafka v2), que amortiza cabeceras, CRC y compresión sobre muchos registros. Sus campos (en disco, *little-endian*; *deltas* en `varint` / `zigzag`):

Campo	Tipo	Descripción
<code>baseOffset</code>	i64	Offset del primer registro del batch.
<code>length</code>	i32	Longitud del batch.
<code>crc32c</code>	u32	Checksum que cubre el batch completo.
<code>attrs</code>	u16	Codec de compresión (none/LZ4/Zstd), bits de txn/idempotencia.
<code>producerId</code> / <code>producerEpoch</code> / <code>baseSequence</code>	i64/i16/i32	Productor idempotente.
<code>recordCount</code>	i32	Número de registros.
<code>records[]</code>	—	Cada uno con <code>offsetDelta</code> / <code>timestampDelta</code> (varint/zigzag), <code>key</code> , <code>value</code> , <code>headers</code> .

El **offset** lógico de cada registro es `baseOffset + offsetDelta`. El formato exacto es contrato: ver [docs/protocol.md](#). La **compresión** LZ4/Zstd es opcional (se compila solo si `find_package` la encuentra) y se aplica por batch.

9.4 Durabilidad y recuperación

- **fsync agrupado** (por N mensajes / N ms / por *commit*): durabilidad real sin pagar un `fsync` por escritura (ver [capítulo 8](#)).
- **CRC32C por batch**, verificado al leer y al recuperar: detecta corrupción silenciosa.
- **Recuperación al arrancar**: se valida el CRC de la cola del log y se **trunca la cola torn** (escritura a medias por un *crash*), dejando el log consistente y listo en pocos segundos. Es la durabilidad que sostiene la garantía `acks=quorum`.

9.5 Retención y compactación

- **Retención por tiempo/tamaño** (`retention.ms` / `retention.bytes`): se borran **segmentos sellados enteros** cuando se supera el umbral (nunca registros sueltos ni el segmento activo). Ver el [diagrama 9](#).
- **Compactación por clave** (`LogCompactor`): política alternativa que conserva el último valor por clave, útil para *topics* de estado tipo *changelog*.

El storage ofrece **mecanismo** (append, read, fsync); la **política** (retención, durabilidad, compresión) es configurable y externa (`LogConfig`). Bajo Raft, este mismo log es el **WAL** de la partición y lo envuelve `RaftLog` (ver [capítulo 10](#)).

10. Replicación y consenso

Cómo NexusMQ tolera fallos sin perder datos *committed*: **Raft por partición**, con el log de Raft como WAL. Es la Fase 2 del proyecto y el corazón de su *correctitud demostrable*. Decisiones: [ADR-0003](#), [0014](#), [0015](#), [0016](#), [0024](#), [0025](#).

10.1 Un grupo Raft por partición

Cada partición es **su propio grupo Raft**: un único mecanismo de ordenación, replicación y elección. Frente al ISR clásico de Kafka (consenso solo para metadatos + primario-backup para datos), Raft por partición es **conceptualmente uniforme, formalmente especificado y testeable de forma determinista**, y compone con shared-nothing (un grupo anclado a un núcleo). El precio es el **quórum en cada escritura** (latencia de mayoría) en el camino caliente, asumido por la postura CP. Los estados de un nodo (`follower` → `candidate` → `leader`) están en el [diagrama 10](#).

10.2 El log de Raft es el WAL

No hay un WAL separado del log replicado: **el log de Raft es el log de la partición**. De ahí dos consecuencias clave:

- **High-watermark** = `commitIndex`. El offset visible a los consumidores es el `commitIndex` del grupo Raft; no existe un mecanismo de ISR aparte. El modelo de `acks` se asienta sobre el *commit*: `acks=0` responde sin esperar (*at-most-once*); `acks=1`, tras el *append* local del líder; `acks=quorum` (por defecto), tras el *commit* de Raft. Ver el flujo de replicación en el [diagrama 11](#).
- **Dos espacios de coordenadas** ([ADR-0014](#)): el **índice de entrada Raft** (ordinal de entrada; **una entrada = un `RecordBatch`**) es distinto del **offset por record** (dentro del batch). `RaftLog` envuelve el `PartitionLog` y persiste `term`/offsets por entrada en un **sidecar** de tamaño fijo, dejando el `RecordBatch` y `nexus-storage` **sin cambios**. La correspondencia se ilustra en el [diagrama 13](#).

10.3 RaftNode: una máquina de estados sin E/S

El núcleo de Raft, `RaftNode`, es una **máquina de estados síncrona sin E/S** ([ADR-0015](#)): recibe **entradas** (`tick(now)`, `on_append_entries`, `on_request_vote`, `on_propose` ...) con el reloj **inyectado** y produce una **cola de mensajes de salida** drenable; no hace ni red ni disco por sí mismo. Esta separación núcleo/E-S es lo que habilita la **simulación determinista** (reloj y red virtuales): se reproducen *timing*, elecciones de líder y particiones de forma **repetible**, cumpliendo FIRST sin *flakiness* (ver [capítulo 21](#)). El estado persistente (`currentTerm`, `votedFor`, `log[]`) se guarda **antes** de enviar nada por la red.

10.4 ReplicatedPartition y el transporte inter-nodo

`ReplicatedPartition` es un **tipo paralelo** a `Partition` ([ADR-0016](#)) que compone la pila Raft sobre el log de la partición. En producción, los RPC de Raft viajan por un **plano inter-nodo separado** del plano de cliente ([ADR-0025](#)):

- Un **sobre de wire** (`topic | partition | from | to | type | payload` , longitud-prefijo) rutea cada `RaftMessage` a la réplica (`topic, partition`) destino, reutilizando el `encode / decode` por RPC.
- Un **portador por partición** (`RaftCarrier`), afinado al reactor dueño vía `PartitionRouter` , conduce la FSM (`tick / take_messages / on_*`), **persiste el estado antes de enviar** y dispara la compactación por política.
- El `Server` se monta sobre `ReactorPool` y el `RequestRouter` enruta async al reactor dueño.

Los dos planos (cliente vs Raft, en puertos separados) se ven en el [diagrama 15](#).

10.5 Failover sin pérdida de datos committed

Al expirar el *election timeout* sin *heartbeat* del líder, un *follower* pasa a *candidate*, incrementa el `term` y solicita votos (`RequestVote` , con *pre-vote* opcional); si obtiene mayoría, se promueve a líder. **Solo puede ganar quien tenga el log al menos tan actualizado como la mayoría**, así que **no se pierden datos committed**. Los clientes redescubren el líder vía *metadata* (un líder obsoleto responde `NOT_LEADER_FOR_PARTITION` , detectado por el `leaderEpoch`). El proceso está en el [diagrama 12](#).

10.6 Compactación por snapshot

Para que el log replicado no crezca sin límite, [ADR-0024](#) añade una **base de snapshot** (`last_included_index, last_included_term, last_included_offset`) persistida en un sidecar dedicado (`raft-snapshot` , registro fijo con CRC32C). `compact_to(index)` descarta el prefijo ya aplicado (exacto en el índice) y recorta el prefijo **físico** del `PartitionLog` por **segmentos sellados enteros** (*best-effort*). Un seguidor muy rezagado se pone al día con `InstallSnapshot` , adoptando la base del líder ([diagrama 14](#)).

10.7 Postura de consistencia

NexusMQ es **CP / PACELC PC-EC** ([ADR-0007](#)): ante una partición de red, una partición de datos sin quórum **deja de aceptar escrituras** (no diverge); en operación normal, las escrituras esperan al quórum (`acks=quorum`), priorizando consistencia sobre latencia. La lectura por defecto es desde el **líder** hasta el *high-watermark*; las lecturas *stale* desde *followers* son *opt-in* y documentadas.

11. Ingress

La capa de entrada: cómo llegan los clientes al broker. NexusMQ ofrece **dos modos** con jerarquía explícita —nativo directo (primario) y proxy/REST (secundario)— en lugar de "un ingress que lo hace todo". Decisiones: [ADR-0006](#), [0027](#), [0019](#), [0018](#).

11.1 Dos modos, un *trade-off* explícito

En un plano de datos de alto rendimiento, interponer un proxy añade un salto y rompe el *zero-copy*; pero exigir siempre un *smart-client* excluye a clientes simples y a HTTP. La respuesta son **dos modos** (ver [diagrama 19](#)):

- **Nativo directo (primario).** Un *smart-client* consulta *metadata* y va **directo al líder** de la partición, sin proxy. Es el camino óptimo en latencia; el cliente gestiona el descubrimiento de líder y reacciona a `NOT_LEADER_FOR_PARTITION`.
- **Proxy/REST (secundario, *opt-in*).** El *ingress* enruta clientes "tontos" (*consistent hashing*) y expone un **gateway REST**, asumiendo el salto extra a sabiendas.

11.2 Modo proxy: *upstream pool* por reactor

El cableado del modo proxy ([ADR-0027](#)) mantiene un **pool de conexiones aguas arriba por reactor** (`UpstreamPool`), coherente con *shared-nothing*: cada reactor reutiliza sus propias conexiones al broker destino (resuelto vía `PeerDirectory`), sin compartir *sockets* entre núcleos. Esto evita reabrir conexión por petición (el *handshake* TCP+TLS cuesta RTTs) y mantiene la localidad del modelo *thread-per-core*.

11.3 TLS opcional sobre el proactor

La terminación TLS 1.3 se hace con **OpenSSL** mediante un **punto de BIOS de memoria** sobre el **Proactor** ([ADR-0019](#)): OpenSSL procesa el *handshake* y el cifrado **en memoria** (memory BIOs), mientras el socket asíncrono mueve los bytes cifrados; así una librería de TLS síncrona convive con la I/O por *completions* sin acoplarse a ella (ver [diagrama 20](#)). La dependencia es **opcional**: se compila si `find_package(OpenSSL)` la encuentra (`NEXUS_HAVE_OPENSSL`); si falta, el nodo **degrada a texto en claro** en lugar de no compilar. Se contempla **mTLS** intra-cluster (ver [capítulo 27](#)).

11.4 Gateway REST de administración

El plano de control HTTP (`/api/v1`, salud, métricas) se sirve por un **adaptador** (`AdminService` en `nexus-ingress`, `AdminApi` en `nexus-server`) en su propio puerto ([ADR-0018](#)), sin acoplar el núcleo al framework web. El detalle del contrato está en el [capítulo 15](#).

11.5 Resiliencia

El ingress aplica los patrones de estabilidad clásicos (Nygard):

- **Circuit Breaker** (estados *closed/open/half-open*, ventana deslizante de errores) para no insistir contra un destino caído.
- **Bulkhead**: *pools* de conexiones aislados por destino, para que un fallo no agote todo.
- **Retry con *backoff* exponencial + *jitter*** (con tope) sobre operaciones idempotentes.
- **Rate limiting** con *token bucket* por cliente/*topic*.
- **Health checks** activos (ping periódico) y pasivos (errores en tráfico real).
- **Backpressure por créditos** propagado hasta el cliente, en lugar de descartar en silencio (ver [capítulo 7](#)).

12. Observabilidad

Cómo se ve por dentro un broker en marcha: métricas, logs y salud. La observabilidad es de grado producción y se concentra en el target `nexus-telemetry` ([ADR-0017](#)). El flujo se ilustra en el [diagrama 23](#).

12.1 Un target dedicado bajo el broker

La telemetría vive en su propia librería, `nexus-telemetry`, por debajo del broker en el grafo de dependencias ([ADR-0017](#)): así el núcleo emite métricas y logs sin depender del detalle de exposición, y se evita un ciclo de dependencias. Agrupa tres piezas: `metrics`, `logging` y `tracing`.

12.2 Métricas (Prometheus)

El nodo expone `GET /metrics` en formato de exposición de Prometheus, sin autenticación (para no entorpecer al *scraper*). Las métricas cubren las **cuatro señales de oro** —latencia, tráfico, errores, saturación— y los recursos por el método **USE** (utilización, saturación, errores). Magnitudes típicas de un broker: throughput de *produce/fetch*, latencias por percentil, *lag* de consumidores, tamaño y *commit* del log, estado de Raft (líder/seguidor, `commitIndex`), créditos de *backpressure*. La latencia se reporta por **percentiles** (p50/p99/p999), no por medias (ver [capítulo 23](#)).

12.3 Logs estructurados

Los logs son **estructurados (JSON)**, con **correlation IDs** que permiten seguir una petición a través de los planos y los reactores. Se evita el *logging* síncrono caro en el camino caliente. El idioma de los mensajes de cara al operador es español; los identificadores, en inglés.

12.4 Tracing

`tracing` propaga *correlation IDs* a través del *pipeline* (recepción → decodificación → *append* → replicación → *ack*) y entre reactores, de modo que una operación distribuida pueda reconstruirse. Como toda observabilidad, tiene coste: se mide su *overhead* (*observer effect*) y no se deja activado a ciegas bajo carga.

12.5 Salud: liveness y readiness

El plano de salud distingue dos preguntas distintas:

- `GET /healthz` (**liveness**): ¿el proceso está vivo? Si falla, el orquestador reinicia.
- `GET /readyz` (**readiness**): ¿puede recibir tráfico? Comprueba disco, estado de Raft y *lag*; si falla, el orquestador lo saca de balanceo **sin** reiniciarlo.

Esta distinción evita reiniciar procesos "vivos pero no listos" (p. ej. poniéndose al día tras un *failover*). El `HEALTHCHECK` de la imagen y las *probes* de Kubernetes se cablean a `/readyz` (ver [capítulo 25](#)). El despliegue de observabilidad (Prometheus + Grafana *self-hosted*) se describe en el [capítulo 26](#).

13. Protocolo binario nativo

Este capítulo **explica y enlaza** el protocolo binario propio del plano de datos de NexusMQ —el rationale, las capas que lo implementan y los flujos de petición/respuesta—. El contrato *as-built* (byte-layout exacto, tipos campo a campo) es [../protocol.md](#) ; este capítulo no lo duplica.

13.1. Por qué un protocolo propio

El plano de datos necesita un protocolo, y había dos caminos en tensión: un **binario propio** o la **compatibilidad con Apache Kafka** desde el inicio. La decisión está registrada en [ADR-0004](#): se implementa un **protocolo binario propio** para las fases nucleares, con un **gateway REST** para interoperabilidad temprana, y se **difiere** un subconjunto Kafka-compatible a la Fase 4 (ver [capítulo 14](#)).

Las razones, en una frase: un protocolo propio **maximiza el aprendizaje y el control** —*framing*, varint, *multiplexing* por *correlation id*, versionado negociado, control de flujo por créditos— y compone **sin impedance mismatch** con la arquitectura *shared-nothing thread-per-core*. La compatibilidad Kafka habría dado ecosistema, pero obliga a implementar una especificación ajena y enorme (más de 70 *API keys* versionadas, *consumer group protocol*...), atando el diseño antes de existir. El titular «Kafka-compatible» queda **recuperable más tarde** sin pagar su coste por adelantado.

13.2. La capa `nexus-wire` (framing sobre conexión)

El protocolo está partido en dos capas con responsabilidades distintas, una decisión de capas limpias registrada en [ADR-0013](#):

- `nexus-protocol` es **protocolo puro**: *codec* (`Encoder / Decoder`), `FrameHeader` , mensajes y códigos de error. Depende **solo** de `nexus-common` —sin E/S, sin *async*—, de modo que la (de)serialización se prueba y se *fuzzea* sin tocar `sockets` ni `io_uring`.
- `nexus-wire` aloja el **framing sobre conexión**: `Frame` , `FrameReader` y `FrameWriter` . Lee y escribe tramas longitud-prefijo sobre un `Socket` mediante el `Proactor` , así que depende de `nexus-io` (`sockets`, `proactor`) y de las corutinas (`task<expected<T>>`). Los consumidores del transporte de tramas —`broker`, `client`, `ingress`, `server`— dependen de `nexus-wire` .

`FrameReader::read_frame` expone el payload como **vista dentro del búfer del lector** (válida hasta la siguiente lectura): es *zero-copy* a cambio de una invariante de vida documentada. Esto encaja con la lectura del framing descrita abajo.

13.3. Estructura de la cabecera de trama

Toda trama es **longitud-prefijo**: una cabecera de **14 bytes** (`FrameHeader::kEncodedSize`) seguida del payload, en **little-endian**. La cabecera lleva, en orden, `length:u32` , `apiKey:u16` , `apiVersion:u16` , `correlationId:u32` y `flags:u16` ; el payload ocupa `length - 10` bytes. El byte-layout exacto está en el [diagrama 16](#) y en [../protocol.md](#) ; aquí basta con el modelo de lectura y la semántica de cada campo.

- **Lectura en dos fases**: se leen 4 bytes (`length`), y luego **exactamente** `length` bytes más. Como `length` cuenta los bytes *tras* su propio campo (resto de cabecera + payload), un lector nunca necesita conocer de antemano el tamaño total.

- **Cotas de seguridad:** el lector valida una **cota inferior** (debe caber el resto de la cabecera) y una **cota superior anti-DoS** (`max_frame` , por defecto **16 MiB**) antes de reservar el búfer.
- **flags** : un mapa de bits de control. Hoy se usa `0x0001` (*credit update*): la trama acarrea una actualización de créditos del control de flujo/*backpressure*. `FrameHeader::has_credit_update()` lo consulta.

ApiKeys

El campo `apiKey` selecciona la operación. El enum `ApiKey` (`src/protocol/frame.hpp`) define las operaciones del plano de datos: `ApiVersions` , `Metadata` , `Produce` , `Fetch` , la familia de offsets de grupo (`OffsetCommit` / `OffsetFetch`), el protocolo de grupos de consumidores (`JoinGroup` / `SyncGroup` / `Heartbeat` / `LeaveGroup`) y administración de topics (`CreateTopic` / `DeleteTopic`). La tabla de valores numéricos y su descripción está en [../protocol.md](#) .

13.4. Versionado negociado

Cada operación tiene un **esquema versionado** por su `apiVersion` , y la versión efectiva se **negocia** al abrir la conexión:

1. El cliente envía `ApiVersions` anunciando el **máximo** que soporta por `ApiKey` .
2. El servidor publica un rango `[min, max]` por `ApiKey` .
3. La versión efectiva es la **mayor que ambos soportan**: `min(client_max, server.max)` si alcanza `server.min` ; `0` si la `ApiKey` no es negociable o no hay solape.

A partir de ahí, cada trama lleva su `apiVersion` y el *codec* (de)serializa según el esquema de esa versión. La lógica de negociación vive en `src/protocol/versioning.hpp` . Este versionado permite **evolucionar el esquema sin romper** clientes antiguos (campos opcionales en versiones nuevas), coherente con la evolución de contratos del proyecto.

13.5. Correlation id y multiplexing

El transporte es **TCP**, y una **única conexión multiplexa muchas peticiones** simultáneas. El mecanismo es el `correlationId` :

- Lo **elige el cliente** en cada petición (típicamente un contador monótono por conexión).
- La respuesta del broker **espeja** (`echo`) ese mismo `correlationId` .
- El cliente casa respuesta con petición por ese id, sin necesidad de que las respuestas lleguen en orden de envío.

Esto evita el coste de abrir una conexión TCP (+TLS) por petición y minimiza RTTs: las peticiones se pueden *pipeline*-ar sobre una conexión persistente.

13.6. Flujos produce/fetch

Sobre este framing se construyen los dos flujos calientes del plano de datos. El **detalle de los mensajes** (campos de `ProduceRequest` / `FetchResponse` , etc.) está en `src/protocol/messages.hpp` y resumido en [../protocol.md](#) ; aquí, el flujo de extremo a extremo:

- **Produce** publica un `RecordBatch` en una partición. El batch viaja **intacto** por el log y por la replicación: es la **unidad de escritura y de entrada de Raft** (ADR-0014). La respuesta acarrea el offset asignado y un `errorCode:i16`. La política de `acks` (0/1/quorum) se resuelve sobre el `commitIndex` de Raft de la partición (ver [capítulo 10](#)).
- **Fetch** lee registros desde un offset hasta el *high-watermark* de la partición. El blob de records puede ir **comprimido** (LZ4/Zstd, indicado en los 2 bits bajos de `attrs`): el broker lo trata como **opaco** —lo guarda y replica comprimido— y solo el cliente lo descomprime al consumir. El bloque comprimido lleva su tamaño original como prefijo (defensa anti *decompression bomb*).

Las respuestas siempre incluyen el `errorCode:i16` del contrato externo de errores; su taxonomía y la traducción en el borde se tratan en el [capítulo 16](#).

13.7. Seguridad del transporte

El plano de datos puede terminar **TLS 1.3** (y **mTLS** intra-clúster) por delante del framing (ADR-0019): el protocolo binario es idéntico, cifrado o en claro. El puente TLS sobre el proactor asíncrono se trata en el [capítulo 11 \(Ingress\)](#) y el [capítulo 27 \(Seguridad\)](#).

13.8. Referencias

- Contrato *as-built*: [../protocol.md](#).
- Diagrama del frame: [diagrama 16](#).
- ADRs: [ADR-0004](#) (protocolo propio), [ADR-0013](#) (`nexus-wire`), [ADR-0009](#) (errores por capa).
- Capítulos relacionados: [14 \(Kafka\)](#), [16 \(modelo de errores\)](#).

14. Subconjunto Kafka

Este capítulo **explica y enlaza** el *listener* compatible con el protocolo de Apache Kafka: por qué existe, qué subconjunto cubre y cómo encaja en la arquitectura *shared-nothing*. El contrato *as-built* (APIs, versiones, framing, RecordBatch) es [../kafka.md](#); este capítulo no lo duplica.

14.1. Por qué un subconjunto, y por qué diferido

La compatibilidad Kafka no es el plano de datos principal de NexusMQ —ese es el [protocolo binario nativo](#)—. Según [ADR-0004](#), implementar la especificación completa de Kafka desde el inicio habría atado el diseño a un contrato ajeno y enorme antes de que el broker existiera. Por eso se **difirió un subconjunto** a la **Fase 4** como *stretch*: lo justo para la demo «`kcat` habla con mi broker», con **interop verificada en vivo** (`kcat /librdkafka`), sin apostar el proyecto a la especificación entera.

El cableado real al servidor —el adaptador que conecta el protocolo Kafka al broker vivo— está registrado en [ADR-0029](#), cuya pieza central se explica abajo.

- **Activación:** `nexusd --kafka-port <N>` (apagado si no se indica). Convive en el mismo proceso con el plano de datos nativo (`--port`) y el de administración (`--admin-port`).

14.2. Adaptador asíncrono cross-core (ADR-0029)

El plano de datos es *shared-nothing thread-per-core* con **sharding partición → núcleo** ([ADR-0026](#)): una partición la **posee un único núcleo**. Pero una conexión Kafka aceptada en un reactor cualquiera puede pedir particiones que pertenecen a **otro** núcleo. La solución de [ADR-0029](#):

- El **puerto** `KafkaBroker` es **asíncrono**: sus métodos (`metadata / produce / fetch / list_offsets`) devuelven `task<...>`, y el `KafkaGateway` hace `co_await broker_.X(req)`. Así el adaptador puede **saltar al reactor dueño** de la partición y volver, en vez de serializar todo el tráfico Kafka en el núcleo 0.
- El **adaptador** concreto `KafkaServerBroker` (en `nexus-server`) implementa ese puerto **enrutando cada partición a su núcleo dueño** vía `PartitionRouter::route`. El puerto `KafkaBroker` no conoce `nexus-broker` (DIP): se testea con un `FakeBroker`.
- El `Server` abre el *listener* en `--kafka-port`; cada conexión la sirve `serve_kafka_connection`, que alimenta `KafkaGateway::handle_request`. El código vive en `src/kafka/` (codec y mensajes) y `src/server/kafka_*` (listener y adaptador).

14.3. Transporte y framing

El framing de Kafka difiere del nativo en dos puntos clave:

- Cada mensaje va **longitud-prefijo** con un `Size:INT32` **big-endian**: 4 bytes con el tamaño del resto del mensaje, seguido de ese número de bytes.
- **Endianness big-endian** en todo el protocolo (contrato de Kafka), a diferencia del protocolo nativo, que es **little-endian**.

Como en el nativo, el transporte es **TCP** y la petición/respuesta se multiplexa por `correlationId`. El detalle está en [../kafka.md](#) y en el [diagrama 17](#).

14.4. APIs y versiones soportadas

El subconjunto cubre **cinco APIs**: `ApiVersions`, `Metadata`, `Produce`, `Fetch` y `ListOffsets` —lo imprescindible para *metadata*, *produce* y *consume*—. El cliente abre con `ApiVersions`, el broker publica los rangos `[min, max]` soportados, y a partir de ahí cada petición lleva su versión; **lo no listado no se soporta** (el cliente recibe el rango y se ajusta). La tabla exacta de `ApiKeys` y rangos está en [../kafka.md](#).

`ApiVersions` anuncia deliberadamente rangos que **cubren lo que negocia librdkafka 2.x** (notablemente `Produce v7` y `Fetch v11`), no las versiones máximas teóricas.

14.5. Clásico vs flexible (tagged fields)

A partir de ciertas versiones, Kafka cambió a un formato **flexible**: cabeceras con *tagged fields* y arrays/strings *compact* con longitud en `UNSIGNED_VARINT`. El codec elige el formato **por API y versión**, según un umbral `flexible_since`: una versión `>=` su umbral es **flexible**; por debajo, **clásica**. Los umbrales por API están en [../kafka.md](#).

La consecuencia práctica, y un detalle que [ADR-0029](#) corrige explícitamente: las versiones que negocia librdkafka (**`Produce v7` y `Fetch v11`**) son **clásicas** (longitudes `INT16` / `INT32`, sin *tagged fields*), mientras que **`Metadata v9`, `ListOffsets v7` y `ApiVersions v3`** son **flexibles**. Por eso el codec **debe** ramificar por versión, con *gates* de campo por versión (p. ej. `last_fetched_epoch` solo v12+ — el bug que rompía `Fetch v11`). Asumir versiones flexibles e ignorar las clásicas rompe con `kcat`: era el fallo de las iteraciones previas, descartado en [ADR-0029](#).

Una excepción heredada del propio Kafka: `ApiVersions` responde **siempre** con cabecera de respuesta v0 (clásica) aunque su cuerpo sea flexible.

14.6. RecordBatch tratado como opaco

`Produce` / `Fetch` transportan **RecordBatch v2 de Kafka** (cabecera de 61 bytes con su `CRC32C`). El broker lo guarda y replica como **blob opaco** dentro de su envoltorio interno: **no reinterpreta el contenido**, solo su `record_count` gobierna la asignación de offsets. Esta decisión evita el coste y la fragilidad de traducir el `RecordBatch` a/desde el formato interno (reencode + recálculo de CRC en el *hot path*).

En `Fetch`, el blob se reconstruye y se le **reescribe el `baseOffset`** autoritativo. Es barato: el CRC de Kafka v2 cubre desde `attributes` en adelante, así que parchear `baseOffset` **no** invalida el CRC.

Un detalle de interop importante: un `Fetch` sin datos nuevos (consumidor al día con el *high-watermark*) devuelve un `MessageSet` **presente pero de longitud 0**, nunca `null` (-1) — librdkafka rechaza un `MessageSetSize` de -1 como trama corrupta.

14.7. Mapeo al broker e interop

El adaptador traduce cada petición Kafka a operaciones del broker vivo: `Metadata` → topics/particiones del nodo; `Produce` → `produce` sobre la partición destino (`RecordBatch` opaco); `Fetch` → lectura desde el offset pedido hasta el *high-watermark*; `ListOffsets` → *earliest* (`log_start_offset`) / *latest* (*high-watermark*); `ApiVersions` → los rangos publicados. El mapeo completo está en [../kafka.md](#).

Con esto, `kcat` interopera en vivo: `-L` (metadata), `-P` (produce) y `-c` (consume), incluido **multi-partición cross-core** (mensajes servidos desde el núcleo dueño de cada partición).

14.8. Límites conocidos

El subconjunto es deliberadamente parcial (las limitaciones se detallan también en el [capítulo 30](#)):

- **Un-nodo:** la `Metadata` anuncia este nodo como único broker y líder de sus particiones; no hay descubrimiento de clúster por el plano Kafka.
- **Sin grupos de consumidores** por el plano Kafka (`OffsetCommit` / `JoinGroup` / ... no están en el subconjunto): el seguimiento de offsets del lado servidor vive en el plano nativo.
- **Sin transacciones ni *idempotent producer*** de Kafka: la idempotencia del broker es la del plano nativo.

14.9. Referencias

- Contrato *as-built*: `../kafka.md`.
- Diagrama: [diagrama 17](#).
- ADRs: [ADR-0029](#) (adaptador async cross-core), [ADR-0004](#) (subconjunto diferido a Fase 4).
- Capítulos relacionados: [13](#) (protocolo nativo), [7](#) (conurrencia / sharding), [30](#) (limitaciones).

15. API REST de administración

Plano de control HTTP para operar el broker: gestión de *topics* y grupos, salud y métricas. Este capítulo **explica y enlaza** el contrato; la fuente de verdad —rutas, esquemas y respuestas exactas— es `docs/openapi.yaml`.

15.1 Rationale y ubicación

La administración no viaja por el plano de datos binario, sino por una **API REST** separada, expuesta en su propio puerto (`--admin-port`). La separación es deliberada ([ADR-0018](#)): el contrato HTTP/JSON se sirve por un **adaptador** (`AdminService` en `nexus-ingress` , `AdminApi` en `nexus-server`) que traduce peticiones REST a operaciones del broker, sin acoplar el núcleo al framework web. Es el plano de control —no el camino caliente—, así que aquí **sí** rigen las excepciones y la traducción central de errores (ver [capítulo 16](#)).

15.2 Recursos y operaciones

Bajo `/api/v1` , recursos en plural y verbos HTTP (sin acciones en la URL):

Método y ruta	Propósito
GET <code>/api/v1/topics</code>	Lista los <i>topics</i> (paginado).
POST <code>/api/v1/topics</code>	Crea un <i>topic</i> ; responde <code>201</code> con cabecera <code>Location</code> .
GET <code>/api/v1/topics/{name}</code>	Describe un <i>topic</i> (resumen + particiones).
DELETE <code>/api/v1/topics/{name}</code>	Borra un <i>topic</i> .
GET <code>/api/v1/groups</code>	Lista los grupos de consumidores (paginado).

Fuera de `/api/v1` y **sin autenticación**, el plano de salud/observabilidad:

Ruta	Propósito
GET <code>/healthz</code>	<i>Liveness</i> : ¿el proceso está vivo?
GET <code>/readyz</code>	<i>Readiness</i> : ¿puede recibir tráfico? (disco/Raft/lag).
GET <code>/metrics</code>	Métricas en formato de exposición de Prometheus.

La distinción *liveness/readiness* permite que un orquestador reinicie procesos "vivos pero no listos" sin sacarlos de servicio prematuramente (ver [capítulo 12, Observabilidad](#)).

15.3 Autenticación

Las rutas `/api/v1/*` se autentican con **Bearer JWT** (HS256), firmado con el secreto del nodo. La autenticación **solo se exige si el nodo arrancó con** `--jwt-secret` ; en ese caso, una petición sin un token

válido recibe `401`. El plano de salud/métricas queda siempre sin autenticar para no entorpecer a las sondas y al *scraper*. El detalle de TLS y la postura de seguridad están en el [capítulo 27](#).

15.4 Paginación y errores

- **Colecciones paginadas:** `GET /api/v1/topics` y `GET /api/v1/groups` paginan en lugar de devolver listas sin límite.
- **Errores RFC 7807:** toda respuesta de error usa el esquema `ProblemDetail` (`application/problem+json`), con una **política central** de traducción. Esto da al cliente un formato de error uniforme y predecible (códigos `400` / `401` / `404` / `409` / ...).

15.5 Documentación como contrato

`openapi.yaml` no es solo documentación: es un **artefacto de tooling** (Swagger UI, generación de clientes, *contract testing*). Por eso se mantiene como fichero independiente y este capítulo no reproduce sus esquemas: consúltalos en [docs/openapi.yaml](#). El flujo de autenticación y petición se ilustra en el [diagrama 18](#).

16. Modelo de errores y códigos de wire

Cómo se representa y propaga un fallo en NexusMQ. La regla es **una política por capa**: el mecanismo cambia según la superficie (protocolo, núcleo, plano de control), pero las invariantes transversales no. Decisión de fondo: [ADR-0009](#).

16.1 Tres superficies de error

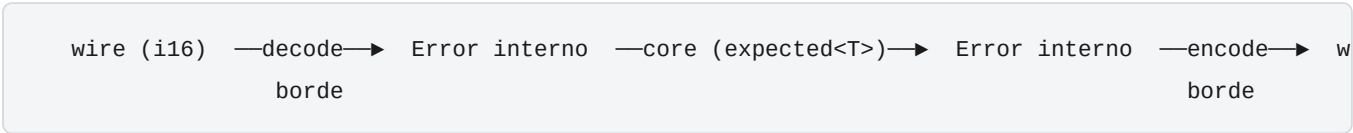
El broker tiene tres superficies con exigencias distintas, y por eso tres mecanismos:

Superficie	Mecanismo	Por qué
Protocolo / wire	<code>errorCode:i16</code> (código numérico)	Es contrato de red, parte del <i>wire format</i> ; legítimo y estable entre versiones.
Núcleo / <i>hot-path</i> (storage, reactor, consensus)	<code>std::expected<T, Error></code>	El error es un valor de coste predecible; sin <i>unwinding</i> de excepciones en el camino caliente.
Plano de control (admin, configuración)	Excepciones + <code>std::optional</code>	Lógica de aplicación: las excepciones separan el flujo normal del fallo; ausencia → <code>optional</code> .

Esta política reconcilia las dos autoridades de la biblioteca: "excepciones, no códigos de error" rige la **lógica de aplicación**, mientras que en **sistemas/baja latencia** se prefiere un error explícito como valor (`std::expected`). Ambas conviven sin contradicción porque se aplican a capas distintas.

16.2 Traducción en el borde

Los códigos de wire **no se propagan crudos** por el núcleo: se **traducen al modelo interno en el borde** (al decodificar una petición) y de vuelta a un código al codificar la respuesta. El núcleo razona con `Error / expected`, no con enteros del protocolo. Del mismo modo, los errores de librerías de terceros (p. ej. OpenSSL) se **envuelven** en tipos propios para no acoplar el núcleo a su modelo de errores.



16.3 El tipo `Error` y `expected<T>`

El núcleo usa el alias `expected<T> = std::expected<T, Error>` (C++23, que es la razón de subir el estándar y de exigir libc++ en Clang — [ADR-0011](#)). Las corutinas del *hot-path* devuelven `task<expected<T>>`. La propagación se encadena con los monádicos `and_then / or_else`, evitando el ruido de comprobar cada retorno a mano. Todo `Error` aporta **contexto** (qué operación falló y de qué tipo de fallo se trata): nunca se "traga" un error en silencio.

16.4 Taxonomía de códigos del protocolo

El protocolo nativo define un conjunto acotado de códigos `i16` (definidos en `src/protocol/error_code.hpp`) que cubren las familias habituales de un broker, por ejemplo:

- **Entrada inválida:** mensaje/batch demasiado grande (`MESSAGE_TOO_LARGE`), petición malformada, versión de API no soportada.
- **Enrutado/topología:** *topic*/partición inexistente, **no soy el líder** de esa partición (el cliente debe redirigir al líder vía *metadata*).
- **Disponibilidad/consistencia:** sin quórum para escribir (postura CP), *timeout* de replicación.
- **Offset/retención:** offset fuera de rango (purgado por retención).

La lista exacta y sus valores son **contrato**: viven en el código y en el [contrato del protocolo](#). Para el subset Kafka, los códigos se mapean a los **error codes de Kafka** en el borde del adaptador (ver [capítulo 14](#)).

16.5 En la API REST

En el plano de control, el borde REST traduce a `ProblemDetail` (**RFC 7807**) con una **política central** (ver [capítulo 15](#)). Es la misma filosofía —una sola política de traducción de errores hacia el exterior— materializada en HTTP en lugar de en el *wire* binario.

17. Mapa de módulos

La traducción del diseño a *targets* de build: qué librerías existen, cómo dependen entre sí y qué se entregó en cada fase. Es la vista de implementación de la [vista de conjunto](#).

17.1 Targets

Un único árbol CMake produce **15 librerías** `nexus-*`, los ejecutables y las herramientas:

- **Librerías:** `nexus-common`, `nexus-io`, `nexus-wire`, `nexus-protocol`, `nexus-storage`, `nexus-reactor`, `nexus-consensus`, `nexus-cluster`, `nexus-broker`, `nexus-kafka`, `nexus-ingress`, `nexus-telemetry`, `nexus-server`, `nexus-client`, `nexus-ffi`.
- **Ejecutables:** `nexusrd` (el servidor), y las *tools* `nexus-cli`, `nexus-bench`, `nexus-loadgen`, más `wincheck` (arnés Windows-only).

`nexus-wire` es una librería **INTERFACE** (solo cabeceras: el framing sobre conexión); `nexus-ffi` se compila como **SHARED** (su superficie es una ABI C estable, ver [capítulo 20](#)).

17.2 Grafo de dependencias y regla de capas

Las dependencias **apuntan hacia el núcleo**; ningún *target* depende de otro de su misma capa o superior. La regla la fuerza CMake (`target_link_libraries` con `PUBLIC / PRIVATE`). De abajo arriba: `nexus-common` (sin dependencias internas) → `io / wire` → `protocol / storage` → `reactor` → `consensus / cluster` → `broker / kafka` → `ingress / telemetry` → `server / client`; `nexusrd` depende de `nexus-server`. El grafo completo —incluidas las aristas exactas— está en el [diagrama 3](#).

Esta disciplina mantiene el núcleo (storage, broker, consenso) **independiente del detalle de I/O y de protocolo**: cuando una capa inferior necesitaría a una superior, se invierte la dependencia con un **puerto** en el consumidor y un **adaptador** arriba (DIP), como en `nexus-telemetry` ([ADR-0017](#)) y en la REST admin ([ADR-0018](#)).

17.3 Mapa fase → targets

Cada fase entregó un subconjunto demoable (ver [diagrama 4](#) y la [historia de desarrollo](#)):

Fase	Targets principales	Entregable
1	nexus-common , nexus-storage , nexus-bench	Motor de log monopartición + benchmarks (I/O bloqueante).
1b	nexus-io , nexus-wire , nexus-protocol , nexus-reactor , nexus-broker , nexus-client	Broker de un nodo <i>thread-per-core</i> ; <i>produce/fetch</i> con cliente nativo.
2	nexus-consensus , nexus-cluster (+ broker)	Raft por partición, <i>failover</i> , grupos de consumidores.
3	nexus-ingress , nexus-telemetry , nexus-server , nexus-cli	Ingress (TLS, REST admin), CLI, métricas.
4	nexus-kafka , nexus-ffi , nexus-loadgen , wincheck (+ IOCP en nexus-io)	Subset Kafka, binding Python, loadgen, port Windows.

El detalle de cada librería está en el [catálogo por subsistema](#).

18. Catálogo por subsistema

Una ficha por librería `nexus-*`: su responsabilidad, sus tipos clave, su **afinidad** de concurrencia y sus invariantes principales. El nivel es "tipos clave + invariantes", no clase-a-clase: el detalle exhaustivo (cada campo y firma) vive en el código fuente de cada subsistema (`src/`). La afinidad sigue la anotación del proyecto: **REACTOR-LOCAL** (de un reactor, no *thread-safe*) · **INMUTABLE** (solo lectura tras construir) · **CROSS-CORE** (solo se comunica por paso de mensajes) · **THREAD-SAFE**.

nexus-common

Responsabilidad: tipos y utilidades transversales sin dependencias internas. **Tipos clave:** `Bytes` / `span` (vistas no propietarias), `Error` y `expected<T>`, `Record` y `RecordCodec`, `crc32c`, `varint` / `zigzag`, `base64`, `sha256`, `fnv1a`, `compression` (LZ4/Zstd opcional), `task<T>` (corutinas), `move_only_function`. **Afinidad:** mayormente **INMUTABLE**/valor puro; `task<T>` es **REACTOR-LOCAL** en uso. **Invariantes:** `crc32c` coincide con el hardware (SSE4.2) y el *fallback* software; el codec de `Record` es *round-trip* (`decode(encode(x)) == x`).

nexus-io

Responsabilidad: I/O asíncrona por *completions* (el proactor) y recursos del SO. **Tipos clave:** `Proactor` (puerto), `IoUringBackend`, `IoctlBackend`, `File`, `Socket`, `Listener`, `AlignedBuffer`, `BlockReader`, `awaitable`; tipos portables `NativeHandle` / `IoResult`. **Afinidad:** **REACTOR-LOCAL** (un anillo `io_uring` por núcleo; sin compartición). **Invariantes:** todo recurso del SO es RAII (se libera en el destructor); toda *completion* comprueba su resultado (puede fallar como una `syscall` síncrona).

nexus-wire

Responsabilidad: *framing* sobre conexión (separar el framing de la codificación pura). **Tipos clave:** `FrameReader` / `FrameWriter` (cabecera `INTERFACE`, [ADR-0013](#)). **Afinidad:** **REACTOR-LOCAL** (por conexión). **Invariantes:** `nexus-protocol` queda **puro** (sin I/O); el framing no interpreta el cuerpo.

nexus-protocol

Responsabilidad: protocolo binario nativo: codificación, frames, versionado y códigos de error. **Tipos clave:** `FrameHeader` (`length:u32`, `apiKey:u16`, `apiVersion:u16`, `correlationId:u32`, `flags:u16`), `Codec`, `messages`, `versioning`, `error_code` (`i16`). **Afinidad:** **INMUTABLE**/sin estado (codec puro). **Invariantes:** *round-trip* de serialización; los códigos de wire son **contrato** y se traducen en el borde (ver [capítulo 16](#)).

nexus-storage

Responsabilidad: motor de log append-only (Fase 1). **Tipos clave:** `PartitionLog`, `Segment`, `Index` (`IndexEntry`), `LogConfig`, `Retention`, `LogCompactor`, `FetchResult`. **Afinidad:** **REACTOR-LOCAL**

(pertenece al reactor dueño de la partición). **Invariantes:** `logEndOffset` es monótono; la retención nunca borra el segmento activo; cada `RecordBatch` está protegido por CRC32C verificado al leer/recuperar.

nexus-reactor

Responsabilidad: modelo de ejecución *thread-per-core* y coordinación entre núcleos. **Tipos clave:**

`Reactor`, `ReactorPool`, `SpscQueue`, `MpmcQueue`, `CrossCore / CrossCoreCall`, `PartitionRouter`,

`Scheduler`, `Allocator` (por núcleo), `CacheLine`, `PeriodicTimers`. **Afinidad:** `Reactor` **REACTOR-**

LOCAL; las colas y `CrossCore / PartitionRouter` son el puente **CROSS-CORE**. **Invariantes:** un reactor por núcleo, *pinned*; nada bloqueante en el reactor; *head/tail* de cada cola en líneas de caché distintas (`alignas`); SPSC con un único productor/consumidor.

nexus-consensus

Responsabilidad: Raft como máquina de estados sin E/S. **Tipos clave:** `RaftNode` (FSM), `RaftLog`, `RaftState / RaftStateStore`, `RaftRpc`, `RaftWire`, `RaftCarrier`, `*MessageSink` (deferred/null).

Afinidad: **REACTOR-LOCAL** (la FSM la conduce el portador del reactor dueño). **Invariantes:** la FSM no hace I/O (entradas → cola de salidas, `now` inyectado, [ADR-0015](#)); el estado persistente se guarda *antes* de enviar; el índice de entrada Raft es espacio distinto del offset por record ([ADR-0014](#)).

nexus-cluster

Responsabilidad: transporte inter-nodo de los RPC de Raft (plano separado). **Tipos clave:**

`RaftTransport`, `RaftLink`, `RaftReceiver`, `PeerDirectory`. **Afinidad:** **CROSS-CORE**/borde de red

(rutea por (`topic`, `partition`) al reactor dueño). **Invariantes:** sobre de wire

`topic|partition|from|to|type|payload` con longitud-prefijo; plano separado del de cliente ([ADR-0025](#)).

nexus-broker

Responsabilidad: dominio del broker: topics, particiones, grupos, offsets, enrutado. **Tipos clave:** `Topic`, `Partition`, `PartitionBase`, `ReplicatedPartition`, `TopicCatalog`, `TopicManager`, `TopicCluster`, `ConsumerGroup`, `GroupCatalog`, `GroupCoordinator`, `OffsetManager`, `ProducerSession` (idempotencia), `CreditWindow` (backpressure), `RequestRouter`. **Afinidad:** **REACTOR-LOCAL** por partición/grupo (cada uno tiene un único dueño; ver [sharding](#)). **Invariantes:** una partición es la unidad de serialización (estilo *actor*); `leaderEpoch` descarta líderes obsoletos; secuencia idempotente por `producerId` (duplicado/hueco).

nexus-kafka

Responsabilidad: subconjunto Kafka-compatible (interop `kcat`). **Tipos clave:** `Codec`, `messages`, `produce`, `fetch`, `metadata`, `list_offsets`, `record_batch` (opaco), `gateway`, `error_code` (Kafka).

Afinidad: adaptador **asíncrono cross-core** sobre el broker vivo ([ADR-0029](#)). **Invariantes:** codec por versión (clásico vs flexible); el `RecordBatch` se almacena **opaco**; los códigos se mapean a los de Kafka en el borde (ver [capítulo 14](#)).

nexus-ingress

Responsabilidad: capa de entrada (dos modos) y plano de control HTTP. **Tipos clave:** `Proxy`, `UpstreamPool`, `Tls`, `RestGateway`, `AdminService`, `CircuitBreaker`, `LoadBalancer`, `HealthMonitor` / `HealthCheck`, `RateLimiter`, `ProblemDetail`, `Jwt`, `Pagination`, `Http / Json`, `ConnectionState`. **Afinidad:** `UpstreamPool` **REACTOR-LOCAL** (pool por reactor); el resto, por conexión. **Invariantes:** el modo nativo es el primario; TLS opcional con degradación a claro ([ADR-0019](#)); validación en el borde (*fail-fast*).

nexus-telemetry

Responsabilidad: observabilidad: métricas, logs y *tracing*. **Tipos clave:** `metrics` (exposición Prometheus), `logging` (JSON estructurado), `tracing`. **Afinidad:** **THREAD-SAFE** en la agregación/exposición de métricas. **Invariantes:** sin *logging* síncrono caro en el camino caliente; las métricas cubren las cuatro señales de oro (ver [capítulo 12](#)).

nexus-server

Responsabilidad: *composition root* de `nexusrd`; ensambla todo y enruta peticiones. **Tipos clave:** `Server`, `Connection`, `AdminApi` / `AdminHttp` / `AdminRouter`, `KafkaAdapter`, `KafkaConnection`, `main`. **Afinidad:** monta el `ReactorPool`; el `RequestRouter` enruta async al reactor dueño. **Invariantes:** el cableado de dependencias vive aquí, no en el dominio (ver [capítulo 19](#)).

nexus-client

Responsabilidad: librería cliente nativa en C++. **Tipos clave:** `Client`, `Producer`, `Consumer`, `DeadLetter`, `Endpoint`. **Afinidad:** usada por aplicaciones externas; el *smart-client* va directo al líder vía *metadata*. **Invariantes:** reacciona a `NOT_LEADER_FOR_PARTITION` redescubriendo el líder; respeta los créditos de *backpressure*.

nexus-ffi

Responsabilidad: frontera ABI C estable para *bindings* (Python vía `ctypes`). **Tipos clave:** `nexus_ffi.h` (API C), `nexus_ffi.cpp` (implementación). **Afinidad:** **frontera**; traduce entre la ABI C y el cliente C++. **Invariantes:** ABI C estable (no C++), por lo que el binding no necesita `python3-dev` ni recompilación ([ADR-0020](#)).

19. Arranque y composition root

Cómo cobra vida un nodo: `nexusd` como *composition root* donde se cablea todo, su configuración 12-factor y sus puertos. El cableado de dependencias ocurre **aquí**, no en el dominio (clean architecture: separar construcción de uso).

19.1 `nexusd` y el composition root

El ejecutable `nexusd` (`src/server/main.cpp`) es el **composition root**: lee la configuración, construye el `Proactor` adecuado a la plataforma, levanta el `ReactorPool` (un reactor por núcleo), instancia el broker, el stack Raft, el ingress y la telemetría, y los **inyecta por constructor**. Ningún componente de dominio instancia su infraestructura: la recibe ya construida (DIP, ver [inyección de dependencias](#)). En C++ esto se materializa con inyección por **constructor** (interfaz + `unique_ptr`) en los bordes y por **parámetro de plantilla** (polimorfismo estático, coste cero) en los caminos calientes.

19.2 Configuración 12-factor

La configuración entra por **flags de línea de comandos** y variables de entorno (config en el entorno, no en la imagen — [ADR-0008](#)), de modo que **el mismo binario** corre en local o en cloud sin cambios. Los **secretos** (p. ej. el de JWT) van por entorno, nunca horneados (ver [capítulo 27](#)). Flags principales de `nexusd`:

Flag	Propósito
<code>--host</code>	Interfaz de escucha (p. ej. <code>0.0.0.0</code>).
<code>--port</code>	Puerto del plano de datos (cliente nativo; <code>9092</code> por convención).
<code>--admin-port</code>	Puerto del plano de operación (REST admin + salud + métricas; <code>9644</code>).
<code>--kafka-port</code>	Puerto del listener Kafka (subset; opcional).
<code>--data-dir</code>	Directorio de datos (logs de partición).
<code>--node-id</code>	Identificador del nodo en el cluster.
<code>--topic</code>	<i>Topic</i> inicial, formato <code>nombre:particiones</code> (p. ej. <code>demo:3</code>).
<code>--jwt-secret</code>	Activa la autenticación JWT en <code>/api/v1/*</code> (si se omite, REST sin auth).

Los nombres y valores por defecto son contrato operativo; el catálogo completo de configuración (retención, `segment.bytes`, política de `fsync`, etc.) vive en el [capítulo 26](#).

19.3 Puertos y planos

`nexusd` separa los planos también en puertos (ver [diagrama 15](#)):

- **Plano de datos / cliente** (`--port`): *produce/fetch* por el protocolo binario.

- **Plano Kafka** (`--kafka-port`): subset Kafka, interop `kcat` .
- **Plano de operación** (`--admin-port`): REST `/api/v1` , `/healthz` , `/readyz` , `/metrics` .
- **Plano inter-nodo Raft**: RPC de consenso entre réplicas, por su transporte separado ([ADR-0025](#)).

19.4 Arranque, afinidad y apagado limpio

En el arranque, cada reactor se **fija a su núcleo** (afinidad). El port a Windows hace esto y las señales **por plataforma** ([ADR-0028](#)): `make_default_proactor` selecciona `io_uring` o `IOCP`; la afinidad usa la API de cada SO (`SetThreadAffinityMask` en Windows). El **apagado limpio** atiende `SIGTERM` / `SIGINT` (`SetConsoleCtrlHandler` en Windows) para **drenar** el trabajo en curso y cerrar recursos antes de salir; en el camino crítico solo se usan funciones *async-signal-safe* (despertar el bucle vía *self-pipe* `eventfd`).

20. Herramientas y bindings

Lo que rodea al servidor: la CLI de operación, los generadores de carga y *benchmark*, el arnés de verificación de Windows y el *binding* de Python. Son piezas de soporte, no del camino caliente, pero esenciales para operar, medir y demostrar el sistema.

20.1 `nexus-cli`

CLI de administración y diagnóstico. Habla con el **plano de operación** (REST `/api/v1`) y/o el plano de datos: `crea/lista/describe/borra topics`, lista grupos de consumidores y produce/ consume de forma puntual. Acepta un *token* Bearer (`--token`) cuando el nodo arranca con `--jwt-secret` . Es el camino humano equivalente a las llamadas REST del [capítulo 15](#).

20.2 `nexus-bench`

Microbenchmarks reproducibles (Google Benchmark) sobre el núcleo —principalmente el *storage engine*— para medir el coste de operaciones concretas (`append`, lectura, CRC32C, codificación). Se usa para establecer **líneas base** y comparar antes/después de una optimización, con la metodología del [capítulo 23](#).

20.3 `nexus-loadgen`

Generador de carga *end-to-end* contra un nodo/cluster vivo, sobre el cliente nativo (`nexus-client`). Parámetros típicos: `--connections` , `--rate` , `--count` , `--payload` , `--warmup` , `--topic` , `--partition` . Es **open-loop** (tasa fija, independiente de las respuestas) para **evitar la *coordinated omission*** y medir la latencia de cola real bajo una carga representativa (ver [capítulo 23](#)).

20.4 `wincheck`

Arnés de verificación **Windows-only** que confirma el backend IOCP en runtime ([ADR-0023](#)): comprueba `File` (bloqueante y directa), un eco IOCP por *loopback* end-to-end con cierre limpio, y `submit_timer` . Los cuatro casos pasan con MSVC (VS 2026, `/w4 /wx`); fue la herramienta que cerró la deuda de runtime del port a Windows.

20.5 Binding de Python (`nexus-ffi` + `ctypes`)

El *binding* de Python no usa `pybind11` sino una **ABI C estable** ([ADR-0020](#)): `nexus-ffi` expone una API en C (`nexus_ffi.h`) compilada como librería **SHARED**, y el lado Python la carga con `ctypes` (en `bindings/python/`). Ventajas: el binding **no necesita `python3-dev`** ni recompilarse contra cada versión de Python (que era el motivo de descartar `pybind11`), y la frontera C es estable y portable. La ABI C envuelve al cliente nativo (`nexus-client`), de modo que Python produce y consume hablando el mismo protocolo que un cliente C++.

21. Estrategia de pruebas

Cómo se gana confianza en NexusMQ: TDD como base, y un arsenal específico para código de sistemas, concurrente y distribuido. Las pruebas son **código de primera clase**; el no-determinismo se **inyecta**, no se tolera.

21.1 TDD y organización

El desarrollo sigue **TDD** (rojo → verde → refactor) sobre la lógica de dominio, con GoogleTest y nombres `Metodo_Escenario_ResultadoEsperado`. La suite vive en `tests/`, organizada por tipo:

Carpeta	Qué cubre
<code>tests/unit/</code>	Pruebas unitarias por subsistema (una carpeta por librería: <code>common</code> , <code>storage</code> , <code>reactor</code> , <code>consensus</code> , <code>protocol</code> , <code>broker</code> , <code>kafka</code> , <code>ingress</code> , <code>io</code> , <code>wire</code> , <code>telemetry</code> , <code>server</code> , <code>client</code> , <code>ffi</code> , <code>cli</code> , <code>bench</code> , <code>loadgen</code> , <code>cluster</code>).
<code>tests/sim/</code>	Simulación determinista de Raft (<code>raft_sim</code>).
<code>tests/crash/</code>	<i>Crash testing</i> (<code>partition_crash_test</code>).
<code>tests/e2e/</code>	Extremo a extremo: broker, cliente, cluster, admin HTTP, CLI, loadgen.
<code>tests/support/</code>	<i>Helpers</i> : <code>fake_proactor</code> (proactor virtual) y <code>test_certs</code> .

21.2 Property-based y fuzzing

- **Property-based**: en serialización se afirman **invariantes** sobre entradas generadas —el *round-trip* `decode(encode(x)) == x` del `Record` / `RecordBatch` y del codec del protocolo—, con *shrinking* al contraejemplo mínimo.
- **Fuzzing**: los parsers de **entrada no confiable** (frames del protocolo, `RecordBatch`, subset Kafka) se alimentan con datos aleatorios/guiados por cobertura para descubrir *panics*, UB y desbordes. Es obligatorio en todo lo que parsea formato externo.

21.3 Simulación determinista (la pieza clave)

El núcleo de Raft es una **máquina de estados sin E/S** ([ADR-0015](#)), y eso es precisamente lo que permite probarlo de forma **determinista**: en `tests/sim/` se inyecta un **reloj virtual** y una **red virtual** (con `fake_proactor`), y se reproducen *timing*, elecciones de líder y particiones de red de forma **repetible**. Así el consenso cumple **FIRST** (determinismo/independencia) sin *flakiness*: una prueba *flaky* es un bug —de la prueba o del diseño—, no algo tolerable. El no-determinismo se controla **inyectándolo**.

21.4 Crash y chaos

- **Crash testing** (`tests/crash/`): se mata el proceso a mitad de escritura (`kill -9`) y se verifica la **recuperación** —validación de CRC y truncado de la cola *torn*— y que no se pierden datos *committed*.

- **Chaos** (local, sin coste): particionar la red (`tc netem`), limitar recursos (`cgroups`) y matar nodos para verificar *failover* e invariantes tras el fallo ([ADR-0008](#)).

21.5 Sanitizers y detección de carreras

Toda la suite se ejecuta también bajo **sanitizers** (ASan/UBSan), que convierten bugs latentes (UB, accesos fuera de rango, fugas) en fallos visibles. Para el código concurrente —las colas lock-free SPSC/MPMC— se usa **ThreadSanitizer + estrés aleatorio**, porque las carreras no aparecen en pruebas deterministas de un solo hilo. Estos pasos forman parte de la **puerta de calidad** (ver [capítulo 22](#)).

22. Puerta de calidad y CI/CD

Lo que tiene que estar verde antes de cada *commit*. La puerta es **obligatoria** y se ejecuta en local; el CI la replica. Nunca se *pushea* en rojo.

22.1 Los dos compiladores

Todo compila y pasa la suite en **GCC/libstdc++** y **Clang/libc++**. No es redundante: los dos *toolchains* **divergen** en puntos sensibles —en especial `std::expected` y las versiones de `clang-tidy`—, así que un cambio que pasa en uno puede romper en el otro. Clang **requiere libc++** (`-stdlib=libc++`): el `<expected>` de libstdc++ no compila con Clang por `__cpp_concepts` ([ADR-0011](#)).

```
cmake --preset linux-gcc && cmake --build --preset linux-gcc && ctest --preset linux-gcc
cmake --preset linux-clang && cmake --build --preset linux-clang && ctest --preset linux-clang
```

22.2 Sanitizers, formato y análisis estático

- **Sanitizers**: la build de pruebas corre bajo **ASan/UBSan** (`linux-gcc-asan`) y **ThreadSanitizer** (`linux-gcc-tsan`) para el código concurrente.
- **Formato**: `clang-format --dry-run --Werror` sobre todo `*.hpp / *.cpp` (sin salida = ok).
- **clang-tidy** (autoridad de convenciones) sobre `src/`, con el preset Clang/libc++. El `.clang-tidy` versionado manda; sus checks desactivados son **decisiones de proyecto** (`pragma-once`; `pro-bounds-* / pro-type-vararg` para búferes de bytes y POSIX variádico; `magic-numbers...`). Se aplica solo a `src/` porque tests y *bench* usan macros de terceros.
- **cppcheck** como análisis estático complementario.
- `-Wall -Wextra -Wpedantic -Werror`: un aviso es un bug en potencia.

22.3 El pipeline de CI

El CI (GitHub Actions, `.github/workflows/ci.yml`) replica la puerta en varios *jobs*:

- **build & test** en matriz [`linux-gcc`, `linux-clang`] (configurar → compilar con *warnings-as-errors* → `ctest`).
- **sanitizers** (ASan/UBSan, `linux-gcc-asan`).
- **ThreadSanitizer** (`linux-gcc-tsan`) para las colas lock-free.

El gate de CI **no es solo build+test**: incluye lint, formato y sanitizers; todo debe pasar.

Estado actual: las GitHub Actions están **desactivadas temporalmente** (cuota); el workflow está versionado como `ci.yml.disabled` y se reactivará al publicar. Mientras tanto, la puerta de calidad se ejecuta **en local** antes de cada *push*.

22.4 Build-once, promote

El artefacto se **construye una sola vez** y se promueve el **mismo binario** entre entornos (dev → staging → prod), sin recompilar por entorno: la diferencia es la **configuración** (12-factor), no el binario (ver [capítulo 19](#) y [ADR-0008](#)). Es coherente con la imagen Docker reproducible del [capítulo 25](#).

23. Rendimiento y benchmarks

Cómo se mide NexusMQ sin engañarse. Este capítulo **explica la metodología**; las cifras concretas y su entorno son contrato y viven en [docs/benchmarks.md](https://docs.nexusmq.com/benchmarks.md) — no se reproducen aquí para no duplicar ni desincronizar.

23.1 Principios de medición

- **Mide, no adivines.** Primero correcto y claro; se optimiza **con datos** y solo el cuello de botella dominante (ley de Amdahl). No se micro-optimiza el código frío.
- **Objetivos en percentiles, no en medias.** Se reporta **p50/p99/p999/max**: la media oculta la *long tail* que sufre el usuario peor servido. Se fija un **SLO** de latencia explícito.
- **Baseline + un cambio cada vez.** Sin línea base no hay mejora demostrable; cambiar varias cosas a la vez impide atribuir el efecto.
- **Perfila antes de tocar** (CPU profiler, *flame graphs*) para localizar el *hot path* real.

23.2 Evitar la *coordinated omission*

El error de medición más traicionero. Un cliente **closed-loop** (que espera cada respuesta antes de enviar la siguiente) **subestima la cola** cuando el sistema se satura: deja de enviar justo las peticiones que más tardarían. Por eso NexusMQ mide con un generador **open-loop** (`nexus-loadgen` , ver [capítulo 20](#)): **tasa fija**, independiente de las respuestas, a una carga representativa (no al throughput máximo, que no dice nada de la latencia de cola). Los histogramas son de alta resolución (estilo **HdrHistogram**). El detalle está en [docs/benchmarks.md](https://docs.nexusmq.com/benchmarks.md) .

23.3 Qué se mide

- **Throughput** de *produce/fetch* (msg/s y MiB/s) a *batch* y *acks* dados.
- **Latencia** de *produce* (`acks=1` y `acks=quorum`) y de *fetch*, por percentiles.
- **Impacto del `fsync`** (agrupado vs por escritura) y de la replicación de quórum.
- **Lectura** desde *page cache/mmap* vs disco.
- **Microbenchmarks** del núcleo (append, CRC32C, codec) con `nexus-bench` .

Se sigue el **método USE** (utilización, saturación, errores) por recurso (CPU, memoria, disco, red) y se tiene en cuenta el *observer effect*: la propia observabilidad tiene coste.

23.4 Throughput ≠ latencia, y escalabilidad

Throughput y latencia se optimizan distinto y a veces se oponen (el *batching* sube throughput pero puede subir latencia). En escalabilidad, la fracción serie pone el techo (Amdahl) y la contención puede **empeorar** al añadir núcleos (USL): por eso el diseño *shared-nothing* reduce la sección crítica a (idealmente) cero —cada partición la sirve su único reactor— antes de añadir núcleos (ver [capítulo 7](#)).

23.5 Entorno y reproducibilidad

Las cifras se toman **en local**, con *hardware* y parámetros documentados (núcleos aislados con `isolcpus / taskset`), nunca en *free tiers* de cloud —que no sirven para medidas serias ([ADR-0008](#))—. El procedimiento exacto de reproducción y los resultados están en [docs/benchmarks.md](#) .

24. Portabilidad

Linux primero, Windows después —pero de verdad—. El objetivo primario es **Linux x86-64**; el port a Windows recorrió tres niveles de verificación hasta funcionar en runtime con MSVC.

24.1 Una abstracción pensada para portar

La portabilidad se diseñó desde el principio, no se parcheó al final: la I/O se modela como **proactor** (*completions*), la forma común a **io_uring** (Linux) e **IOCP** (Windows) ([ADR-0002](#)). El núcleo es independiente de plataforma tras ese puerto; cambiar de SO es cambiar de **adaptador** (`io_uring_backend` ↔ `iocp_backend`) y de *preset* (`linux-gcc` / `linux-clang` ↔ `windows-msvc` / `windows-clang-cl`), sin reestructurar. Los tipos portables `NativeHandle` / `IoResult` aíslan el resto.

24.2 Tres niveles de "funciona"

El port a Windows hace explícita una distinción que suele pasarse por alto —**compila ≠ funciona**— recorriéndola por estados (cadena de ADR):

Nivel	Qué garantiza	Hito
Diseño	El backend está diseñado (<code>iocp_backend.hpp</code> , <code>preset windows-msvc</code>); implementación diferida.	ADR-0021
Compile-verified	Compila con headers Win32 reales (MinGW-w64); <code>File</code> / <code>Socket</code> / <code>Listener</code> Win32+Winsock, <code>IoCpBackend</code> sobre <code>GetQueuedCompletionStatusEx</code> / <code>AcceptEx</code> .	ADR-0022
Runtime-verified	Funciona en Windows real con MSVC (VS 2026, /W4 /WX): arnés <code>wincheck</code> con <code>File</code> (bloqueante+directa), eco IOCP por <code>loopback</code> y <code>submit_timer</code> — 4/4 PASAN.	ADR-0023

El valor real llegó al **verificar en runtime**: compilar no demostraba que el ciclo *submit* → *completion* funcionara de extremo a extremo.

24.3 Port completo de `nexusrd`

[ADR-0028](#) amplía el alcance de "solo `nexus-io` " a **`nexusrd` completo** en Windows: backend, **afinidad y señales** por plataforma. `make_default_proactor` elige `io_uring` o IOCP; la afinidad usa la API de cada SO (`SetThreadAffinityMask`); el apagado limpio atiende las señales equivalentes (`SetConsoleCtrlHandler`). Así el servidor entero —no solo la capa de I/O— corre en ambos SO.

24.4 Diferencias por compilador

Portar no es solo el SO, también el **compilador**. El único arreglo de fondo que necesitó el runtime de Windows fue `crc32c.cpp` : GCC/Clang usan *builtins* para detectar e invocar las instrucciones de CRC de SSE4.2; MSVC necesita `__cpuid` + los **intrínsecos** SSE4.2 explícitos. La detección de *CPU features* y su uso se seleccionan con `#if` por compilador, manteniendo un *fallback* software portable. Es un recordatorio de que las mismas instrucciones hardware se expresan distinto en cada *toolchain* —el mismo motivo por el que el proyecto compila siempre en los **dos compiladores** (ver [capítulo 22](#)).

25. Despliegue

Cómo se empaqueta y se levanta NexusMQ: imagen Docker reproducible, un cluster local con docker-compose y un despliegue con estado en Kubernetes. Los artefactos reales viven en [deploy/](#).

25.1 Imagen Docker

La imagen sigue las buenas prácticas (`deploy/Dockerfile`):

- **Build multi-stage:** una etapa compila con el *toolchain* C++ y otra ejecuta sobre una **base mínima/distroless**, copiando solo el binario.
- **Usuario no-root** (UID/GID `65532`, *distroless nonroot*): limita el impacto de una intrusión.
- **HEALTHCHECK** cableado a `/readyz` (el plano de operación): el runtime reinicia instancias que no responden de verdad.
- **Tags de versión explícitos** (no `latest` en producción), `.dockerignore` para no copiar artefactos de build, y **secretos por variables de entorno**, nunca horneados.
- Se construye **una sola vez** y se promueve el mismo binario entre entornos (ver [capítulo 22](#)).

25.2 Cluster local con docker-compose

`deploy/docker-compose.yml` levanta el entorno de referencia en una sola máquina (ver [diagrama 21](#)):

- **3 nodos** `nexus1 / nexus2 / nexus3` (imagen `nexusmq:dev`), cada uno con `--port 9092` (plano de datos) y `--admin-port 9644` (operación), `--data-dir /var/lib/nexusmq`, `--node-id` y un *topic* de demo (`--topic demo:3`). Los puertos se exponen sin solapar (`9092/9093/9094` datos; `9644/9645/9646` operación) y cada nodo tiene su **volumen** persistente.
- **Prometheus** (`:9090`) scrapea `/metrics` de los tres nodos.
- **Grafana** (`:3000`) con *datasource* Prometheus aprovisionado.

El cluster de 3 nodos es suficiente para Raft, quórum y *failover* ([ADR-0008](#)). El runtime de cluster multi-nodo (descubrimiento de líder, replicación entre nodos) llegó en fases posteriores; el compose sirve además para validar el empaquetado, la observabilidad y los *probes* sobre varias instancias.

25.3 Kubernetes

`deploy/k8s/` despliega NexusMQ como **StatefulSet** —el broker tiene estado persistente (los logs de partición)— con `volumeClaimTemplates` (un PVC por réplica) y un **Service headless** para DNS estable (`nexusmq-0.nexusmq`, etc.), que es como las réplicas se descubren entre sí (ver [diagrama 22](#)). Detalles:

- **3 réplicas**, `securityContext no-root` (`runAsUser/Group/fsGroup 65532`).
- **Probes** *liveness/readiness* contra el puerto de operación (`/healthz`, `/readyz`).
- Se asumen **límites de CPU/memoria** por pod y un nodo con soporte de **io_uring**.

`docker-compose` es para local/dev; Kubernetes, cuando se necesita escala, *self-healing* y orquestación real.

25.4 Cloud-ready, sin coste obligatorio

El diseño es *cloud-ready* (imagen Docker, configuración 12-factor) de modo que el **mismo binario** corre en local o en cloud sin cambios. Una demo desplegada es cubrible con *free tiers* (AWS/Azure/Oracle Always Free) bajo demanda, pero **no es necesaria** para desarrollar, probar ni medir, que se hacen en local sin coste ([ADR-0008](#)).

26. Configuración y operación

Operar un nodo en marcha: cómo se configura, qué puertos usa, qué se observa y qué hacer cuando algo va mal. La configuración es 12-factor (entorno, no imagen).

26.1 Puertos

Plano	Flag	Puerto (convención)
Datos / cliente nativo	<code>--port</code>	9092
Operación (REST <code>/api/v1</code> , <code>/healthz</code> , <code>/readyz</code> , <code>/metrics</code>)	<code>--admin-port</code>	9644
Subset Kafka (interop <code>kcat</code>)	<code>--kafka-port</code>	opcional
Inter-nodo Raft (consenso)	—	transporte interno

26.2 Configuración

Por **flags** y variables de entorno (mismo binario en todos los entornos). Además de los flags de arranque (`-host`, `--node-id`, `--data-dir`, `--topic nombre:particiones`, `--jwt-secret`; ver [capítulo 19](#)), el comportamiento del log y la durabilidad se gobierna por **política** (mecanismo vs política): retención (`retention.ms` / `retention.bytes`), tamaño de segmento (`segment.bytes` / `segment.ms`), compactación, compresión (none/LZ4/Zstd) y política de `fsync` (cada N mensajes / N ms / por *commit*). Los **secretos** (p. ej. el de JWT) van por entorno, nunca en la imagen ni en el repositorio (ver [capítulo 27](#)).

26.3 Observabilidad en operación

- **Métricas:** `GET /metrics` (Prometheus) — throughput, latencias por percentil, *lag* de consumidores, estado de Raft, créditos de *backpressure*.
- **Dashboards:** Grafana *self-hosted* con *datasource* Prometheus aprovisionado (`deploy/grafana/`), siguiendo las cuatro señales de oro (latencia, tráfico, errores, saturación). Ver [capítulo 12](#) y [diagrama 23](#).
- **Salud:** `/healthz` (liveness) y `/readyz` (readiness: disco/Raft/*lag*), usados por los *probes* y el `HEALTHCHECK`.

26.4 Runbook (esbozo)

- **Arranque/parada:** `docker compose up --build` (local). El apagado atiende `SIGTERM` / `SIGINT` y *drena* el trabajo en curso antes de salir.
- **Backup:** un *snapshot* de `--data-dir` por nodo. **Restore:** arrancar desde ese `data.dir`.
- **Recuperación tras *crash*:** automática al arrancar (validación de CRC + truncado de la cola *torn*); el nodo queda listo en pocos segundos (ver [capítulo 9](#)).
- **Pérdida de quórum** (p. ej. 2 de 3 nodos caídos): por la postura **CP**, las particiones afectadas quedan **no disponibles para escritura** y se recuperan al restaurar nodos. Forzar progreso sin quórum solo mediante un procedimiento **manual y documentado** de *unsafe-recovery* (asume riesgo de pérdida).

- **Rotación de certificados TLS:** recarga en caliente si está disponible, o *rolling restart*.
- **Escalado:** *thread-per-core* escala con el número de núcleos del nodo; el cluster, añadiendo nodos/particiones (la asignación partición → núcleo es `partition % N`, ver [capítulo 7](#)).

27. Seguridad

La postura de seguridad de NexusMQ: cifrado en tránsito, autenticación del plano de control, validación en el borde y mínimo privilegio. Se apoya en la normativa de seguridad de la `BibliotecaDocumentacion` (OWASP, defensa en profundidad).

27.1 Cifrado en tránsito (TLS)

- **TLS 1.3 en el borde:** terminación en la capa de *ingress* mediante OpenSSL con un puente de **BIOS de memoria** sobre el proactor ([ADR-0019](#), ver [capítulo 11](#) y [diagrama 20](#)). La dependencia es **opcional**: si OpenSSL no está, el nodo degrada a texto en claro (útil en dev), pero en redes no confiables **TLS es obligatorio** y los certificados se **validan**.
- **mTLS intra-cluster (contemplado):** ambos extremos presentan y validan certificado, dando identidad criptográfica a cada nodo del plano inter-nodo. Conviene **rotar** los certificados (vida corta, emisión automatizada).

27.2 Autenticación y autorización

- **AuthN ≠ AuthZ.** El plano de control (REST `/api/v1`) se autentica con **Bearer JWT** (HS256), exigido solo si el nodo arranca con `--jwt-secret`; sin token válido → `401` (ver [capítulo 15](#)). Los tokens se validan (firma, expiración).
- **Autorización por recurso** en el servidor, en cada petición (no por ocultar acciones en el cliente). El plano de salud/métricas queda sin auth a propósito, para las sondas y el *scraper*.

27.3 Validación en el borde

Nunca se confía en la entrada del cliente. Todo dato externo —frames del protocolo, RecordBatch, peticiones Kafka, JSON de la REST— se **valida en el borde** (*fail-fast*) contra límites explícitos antes de entrar al núcleo: tamaño máximo de mensaje/*batch* (`MESSAGE_TOO_LARGE`), versiones de API soportadas, y **límites de descompresión** para evitar *decompression bombs* (un *batch* que se expande desmesuradamente). El núcleo asume datos ya válidos (ver [capítulo 16](#)).

27.4 Mínimo privilegio y secretos

- **Usuario no-root** en la imagen y en Kubernetes (UID/GID `65532`), para limitar el impacto de una intrusión (ver [capítulo 25](#)).
- **Secretos por variables de entorno** (o gestor de secretos), **nunca** horneados en la imagen ni commiteados al repositorio; si un secreto se filtra, se **rota** (no basta con borrarlo).
- **Defensa en profundidad:** varias capas (TLS, JWT, validación, aislamiento de proceso), cada componente con los permisos justos, asumiendo que una capa fallará.
- **Mensajes de error sin detalles internos** hacia el exterior (códigos de wire / RFC 7807), para no filtrar información sensible.

27.5 Fuera de alcance

- **Cifrado en reposo** queda fuera de alcance (solo en tránsito).
- **Multi-tenancy y ACLs avanzadas** no se abordan en las fases actuales (ver [capítulo 30](#)).

28. Registro de decisiones (ADR)

Este capítulo es la puerta de entrada al **registro de decisiones de arquitectura** (ADR) de NexusMQ: explica qué es un ADR, la regla de inmutabilidad que rige su ciclo de vida, y ofrece el **índice completo** de los 29 ADR del proyecto, agrupados por tema. Las decisiones íntegras viven en el directorio `../adr/`, una por fichero.

28.1 Qué es un ADR

Un *Architecture Decision Record* (ADR) documenta **una** decisión de arquitectura: la fuerza o problema que la motiva (*Contexto*), lo que se decide (*Decisión*), los efectos positivos y negativos que se aceptan (*Consecuencias*) y las opciones descartadas con su porqué (*Alternativas consideradas*). El valor de un ADR no es la decisión en sí, sino el **rastro del razonamiento**: por qué se eligió esto y no aquello, qué se sacrificó y bajo qué supuestos. Es la pieza de portfolio que evidencia el **criterio de ingeniería**, no solo el resultado.

Cada ADR de NexusMQ sigue la plantilla común (*Contexto · Decisión · Consecuencias · Alternativas consideradas*) y conserva su **estado** y su **fecha**. Cada decisión vive en un fichero individual `adr-NNNN-titulo-corto.md` bajo `../adr/`.

28.2 La regla de inmutabilidad

Una decisión aceptado no se edita. Si la realidad obliga a cambiarla, no se modifica el ADR existente: se **escribe uno nuevo** que la reemplaza, y el antiguo pasa al estado *reemplazado por adr-NNNN*. La historia de decisiones es inmutable, igual que la historia de git: poder leer la decisión vieja —y por qué se superó— es tan valioso como leer la nueva.

En NexusMQ esto produce una **cadena de reemplazo** visible en el backend de I/O de Windows (IOCP), que se diseñó y verificó en tres pasos sucesivos:

ADR-0021 (diseño fijado, implementación diferida) → **ADR-0022** (implementado, compile-verificado con MinGW) → **ADR-0023** (verificado en runtime con MSVC).

Cada eslabón cae cuando una premisa se demuestra falsa: ADR-0021 asumía que el backend «no era verificable en este entorno»; resultó que el contenedor **sí** podía instalar el cross-compiler MinGW-w64 (ADR-0022), y finalmente se **ejecutó** en una máquina Windows real (ADR-0023). La cadena documenta esa progresión sin reescribir el pasado.

28.3 Índice de los 29 ADR

ADR	Título	Estado	Fecha
0001	Plataforma de desarrollo y <i>target</i> (Linux primero, Windows después)	aceptado	2026-06-07
0002	Modelo de I/O asíncrona (proactor; <code>io_uring</code> primero, IOCP después)	aceptado	2026-06-07
0003	Modelo de replicación (Raft por partición)	aceptado	2026-06-07
0004	Protocolo del plano de datos (binario propio + gateway REST)	aceptado	2026-06-07
0005	Arquitectura de concurrencia (<i>shared-nothing thread-per-core</i>)	aceptado	2026-06-07
0006	Rol del <i>ingress</i> (dos modos: nativo directo + proxy/REST)	aceptado	2026-06-07
0007	Postura de consistencia (CP / PACELC PC-EC)	aceptado	2026-06-07
0008	Viabilidad de coste cero (desarrollo y test gratuitos)	aceptado	2026-06-07
0009	Política de manejo de errores por capa	aceptado	2026-06-10
0010	Entorno de desarrollo (VS Code sobre WSL)	aceptado	2026-06-11
0011	Estándar C++23 + libcpp en Clang (para <code>std::expected</code>)	aceptado	2026-06-11
0012	Backend <code>io_uring</code> directo sobre el uapi del kernel (sin liburing)	aceptado	2026-06-12
0013	Capa <code>nexus-wire</code> para el framing sobre conexión	aceptado	2026-06-13
0014	Modelo del log de Raft (índice = ordinal de entrada; término en sidecar)	aceptado	2026-06-14
0015	RaftNode como máquina de estados síncrona sin E/S	aceptado	2026-06-14
0016	<code>ReplicatedPartition</code> como tipo paralelo a <code>Partition</code>	aceptado	2026-06-15
0017	Target <code>nexus-telemetry</code> para observabilidad	aceptado	2026-06-17
0018	REST admin por puerto/adaptador (<code>AdminService</code> / <code>AdminApi</code>)	aceptado	2026-06-18
0019	TLS opcional vía OpenSSL con puente de BIOs de memoria	aceptado	2026-06-19
0020	<i>Binding</i> de Python vía ABI C estable (<code>nexus-ffi</code> + <code>ctypes</code>)	aceptado	2026-06-20

ADR	Título	Estado	Fecha
0021	Backend IOCP (Windows) — diseño fijado, implementación diferida	reemplazado por adr-0022	2026-06-20
0022	Backend IOCP (Windows) — implementado, compile-verificado con MinGW	reemplazado por adr-0023	2026-06-20
0023	Backend IOCP (Windows) — verificado en runtime con MSVC	aceptado	2026-06-20
0024	Compactación del log de Raft por snapshot	aceptado	2026-06-20
0025	Activación en producción del stack Raft + multi-reactor (transporte inter-nodo)	aceptado	2026-06-21
0026	Sharding por núcleo del plano de datos (partición → núcleo, grupos por hash)	aceptado	2026-06-21
0027	Cableado del modo proxy — <i>pool</i> de conexiones aguas arriba por reactor	aceptado	2026-06-25
0028	Port completo de <code>nexusd</code> a Windows (backend, afinidad y señales)	aceptado	2026-06-27
0029	Adaptador Kafka asíncrono cross-core sobre el broker vivo	aceptado	2026-06-28

28.4 ADR agrupados por tema

Los 29 ADR cubren ocho áreas. Cada grupo se resume en una frase; los números enlazan al fichero correspondiente en [../adr/](#).

Plataforma e I/O

[0001](#), [0002](#), [0008](#), [0010](#), [0011](#), [0012](#), [0013](#).

Fijan el terreno: Linux primero y Windows después, modelo **proactor** sobre *completions*, C++23 con libcpp en Clang (que `std::expected` exige), `io_uring` directo sobre el uapi del kernel sin `liburing`, y la capa `nexus-wire` que aloja el framing sobre conexión, todo desarrollable y comprobable a coste cero.

Replicación y consenso

[0003](#), [0007](#), [0014](#), [0015](#), [0016](#), [0024](#), [0025](#).

Definen el corazón distribuido: **Raft por partición** con postura **CP** (PACELC PC/EC), el log de Raft como vista (`term`, `índice`) sobre el `PartitionLog`, una FSM síncrona **sin E/S** que el portador conduce, `ReplicatedPartition` paralelo a `Partition`, la compactación por *snapshot* y la activación del stack con transporte inter-nodo real.

Protocolo

[0004](#), [0029](#).

Establecen el plano de datos: un **protocolo binario propio** (con *gateway* REST para administración) y, como *stretch*, un **subconjunto Kafka** servido por un adaptador asíncrono que interopera en vivo con `kcat`.

Concurrencia

[0005](#), [0026](#).

La tesis arquitectónica: **shared-nothing thread-per-core** (un reactor *pinned* por núcleo) con **sharding** del plano de datos (partición → núcleo por `p % N`, grupos por `hash(id) % N`) para evitar la contención.

Ingress y seguridad

[0006](#), [0018](#), [0019](#), [0027](#).

El borde del sistema: *ingress* en dos modos (nativo directo y proxy *opt-in*), REST admin desacoplado por **puerto/adaptador** (DIP), TLS/mTLS opcional con puente de BIOs de memoria sobre el proactor, y el *pool* de conexiones aguas arriba por reactor que cablea el modo proxy.

Observabilidad

[0017](#).

Un target dedicado `nexus-telemetry` que concentra métricas Prometheus y logging estructurado, evitando esparcir la instrumentación por todas las capas.

Windows

[0021](#) → [0022](#) → [0023](#), [0028](#).

La portabilidad a Windows: la **cadena de reemplazo** del backend IOCP (diseño → MinGW → runtime con MSVC) y, sobre ella, el port completo de `nexusd` (backend, afinidad de hilos y señales).

Lenguajes y errores

[0009](#), [0020](#).

Dos decisiones transversales: la **política de errores por capa** (`expected` en el hot-path, excepciones en el plano de control, códigos de wire traducidos en el borde) y el *binding* de Python vía una **ABI C estable** consumida con `ctypes`.

28.5 Dónde está cada decisión

El registro vivo y completo es el directorio `../adr/`, con su propio [README](#) e índice. Este capítulo lo resume y lo contextualiza; para leer una decisión entera —contexto, consecuencias y alternativas— se va al fichero correspondiente.

29. Historia de desarrollo

Retrospectiva del proyecto por fases. NexusMQ se construyó **por capas**, cada una autocontenida y demoable, registrando las decisiones de arquitectura como ADR a medida que surgían. Aquí se resume el recorrido y los aprendizajes; el catálogo de decisiones vive en [docs/adr/](#) y el capítulo [28](#).

29.1 Filosofía de avance

El principio rector fue **incrementos pequeños y siempre compilables**, con TDD (rojo → verde → refactor) y una puerta de calidad en local antes de cada *commit* (dos compiladores, sanitizers, formato, `clang-tidy`). La ambición se persiguió como **profundidad dentro de una sola tesis arquitectónica** (*thread-per-core* + Raft por partición), no como amplitud. Cada decisión de calado quedó como ADR; un ADR aceptado no se edita, se reemplaza (ver [capítulo 28](#)).

29.2 Fase 1 — Storage engine

Motor de log monopartición con **I/O bloqueante y sin reactor** (decisión deliberada: validar correctitud y durabilidad antes que rendimiento). Hitos: `Record` + **CRC32C** por hardware con *fallback* software; `Segment` (`.log` + `.index`); `PartitionLog` (secuencia de segmentos, *rolling*, recuperación con validación de CRC y truncado de la cola *torn*); política de `fsync`; retención por tiempo/tamaño; primeros *benchmarks*.

Aprendizajes: la recuperación ante *crash* y la corrupción silenciosa (*torn writes*) exigen *checksums* por registro verificados al leer; la durabilidad real depende de `fsync` / `fdatasync`, no de `write`.

29.3 Fase 1b — Reactor + broker monolítico

Aparece el **reactor *thread-per-core* propio** sobre el proactor (`io_uring` + corutinas C++23), el **protocolo binario** (capa `nexus-wire` para el framing — [ADR-0013](#)), el cliente nativo y los flujos *produce/fetch* con **backpressure por créditos**. Aquí se consolidan dos decisiones mayores: **C++23 + libc++ en Clang** por `std::expected` ([ADR-0011](#)) y **`io_uring` directo sobre el `uapi`**, sin `liburing` ([ADR-0012](#)).

Aprendizajes: bajo *thread-per-core*, **nada bloqueante puede vivir en el reactor** (un `fsync` o un `malloc` que entre al kernel congela el núcleo) → todo I/O pasa por el proactor y se usan *allocators* por núcleo.

29.4 Fase 2 — Distribución (Raft por partición)

Replicación y consenso con **Raft por partición**. Decisiones clave: el **log de Raft es el WAL**, con el **índice de entrada** como espacio distinto del **offset por record** ([ADR-0014](#)); `RaftNode` como **máquina de estados síncrona sin E/S** —entradas `tick / on_*` con `now` inyectado → cola de mensajes— que habilita la **simulación determinista** ([ADR-0015](#)); `ReplicatedPartition` como tipo paralelo a `Partition` ([ADR-0016](#)); grupos de consumidores y rebalanceo.

Aprendizajes: separar el **núcleo de consenso de la E/S** es lo que hace Raft *testable* de forma determinista (reloj/red virtuales), cumpliendo FIRST sin *flakiness*.

29.5 Fase 3 — Ingress y operación

Plataforma operable y observable: *ingress* en **dos modos** (nativo/proxy — [ADR-0006](#)), **TLS opcional** vía OpenSSL con puente de BIOs de memoria sobre el proactor ([ADR-0019](#)), **API REST de administración** por puerto/adaptador ([ADR-0018](#)), CLI, y observabilidad con el target `nexus-telemetry` ([ADR-0017](#)): métricas Prometheus, logs JSON, *health/readiness*.

Aprendizajes: un puente **TLS síncrono sobre I/O asíncrona** se resuelve con *memory BIOs* (OpenSSL procesa en memoria; el socket asíncrono mueve los bytes cifrados), evitando acoplar la librería de TLS al modelo de *completions*.

29.6 Fase 4 — *Stretch* y producción

Activación del stack distribuido y funcionalidades avanzadas: **transporte inter-nodo real** con plano separado y portador por partición ([ADR-0025](#)); **compactación por snapshot** del log de Raft ([ADR-0024](#)); **sharding por núcleo** del plano de datos ([ADR-0026](#)); **modo proxy** con *upstream pool* por reactor ([ADR-0027](#)); **subconjunto Kafka-compatible** asíncrono cross-core con interop `kcat` ([ADR-0029](#)); *binding* de Python por ABI C ([ADR-0020](#)); y el **port a Windows** (IOCP), que recorrió tres estados —diseño, compile-verificado con MinGW y, finalmente, **verificado en runtime con MSVC**— hasta el **port completo de nexusrd** ([ADR-0021](#) → [0022](#) → [0023](#); [ADR-0028](#)).

Aprendizajes: "compila" ≠ "funciona"; el valor real del port a Windows llegó al **verificarlo en runtime**. El único arreglo de fondo fue portar `crc32c.cpp` a MSVC (`__cpuid` + intrínsecos SSE4.2), lo que evidencia que las *CPU features* se detectan y usan distinto según el compilador.

29.7 Estado actual

Fases 1 → 4 implementadas; el árbol compila y pasa la suite con GCC/libstdc++ y Clang/libc++, y el backend Windows está verificado en runtime con MSVC. Las **GitHub Actions** están desactivadas temporalmente (cuota) y se reactivan al publicar; la puerta de calidad se mantiene en local. La documentación se está consolidando en su forma final (esta misma).

30. Limitaciones y trabajo futuro

Qué **no** hace NexusMQ (a propósito) y por dónde podría crecer. Acotar el alcance es una decisión de ingeniería: mantiene el proyecto profundo en una sola tesis arquitectónica en lugar de disperso. Base: el alcance declarado del proyecto y las consecuencias registradas en los ADR.

30.1 Limitaciones conscientes (fuera de alcance)

- **Compatibilidad total con Kafka.** Solo se implementa un **subconjunto** (`ApiVersions` / `Metadata` / `Produce` / `Fetch` y lo necesario para `kcat`), no las 70+ *API keys* versionadas ni el *consumer group protocol* completo ([ADR-0004](#), [ADR-0029](#)).
- **Exactly-once transaccional entre particiones.** Sí se contempla **productor idempotente** (*effectively-once* por partición); no hay transacciones multi-partición.
- **Tiered storage** a almacenamiento de objetos (idea futura, inspirada en Pulsar).
- **Multi-tenancy y ACLs avanzadas; cifrado en reposo** (solo en tránsito, TLS).
- **Dashboard gráfico propio:** la observabilidad se expone vía Prometheus/CLI; un panel Grafana es opcional y *self-hosted*.
- **HLC y ordenación causal entre particiones.** El orden es por **offset** dentro de una partición; no hay relojes lógicos híbridos entre particiones.

30.2 Deudas acotadas

- **CI desactivado temporalmente.** Las GitHub Actions están en pausa por cuota; la puerta de calidad se ejecuta en local y se reactivará el CI al publicar (ver [capítulo 22](#)).
- **Madurez del modo proxy.** El modo nativo (*smart-client* al líder) es el primario y el más optimizado; el modo proxy con *upstream pool* ([ADR-0027](#)) es secundario y opt-in.
- **Escala de muchos grupos Raft.** Raft por partición implica un grupo (y su *heartbeat*) por partición; no es problema a escala de portfolio, pero a escala de miles de particiones requeriría agrupación de *heartbeats* y rebalanceo más sofisticado ([ADR-0003](#)).
- **Profundidad opcional de I/O.** *Direct I/O* (`O_DIRECT`) con caché/*readahead* propios quedó como profundidad medida, no como camino por defecto.

30.3 Líneas de evolución posibles

- **Ampliar el subset Kafka** hacia más *API keys* y versiones según demanda real de herramientas del ecosistema.
- **Tiered storage:** descargar segmentos sellados a almacenamiento de objetos para retención larga a bajo coste.
- **Lecturas desde followers** (*opt-in*, con garantías documentadas) para repartir la carga de lectura sin romper la postura CP.
- **Pre-vote / leadership transfer / learners** de Raft (de la tesis de Ongaro) para *failover* más suave y reconfiguración de membresía.
- **Optimización medida** del camino caliente (registered buffers/fixed files de *io_uring*, *allocators* NUMA-aware, *hugepages*) allí donde el *profiling* lo justifique.

- **Demo de despliegue en cloud** (el diseño ya es *cloud-ready*, 12-factor) con infraestructura como código, bajo demanda y sin coste recurrente ([ADR-0008](#)).

Apéndice A. Bibliografía

Fuentes canónicas que sustentan las decisiones de diseño de NexusMQ. Coinciden con la autoridad de la [BibliotecaDocumentacion](#) (fundamentos de sistemas, concurrencia, rendimiento, datos distribuidos).

Datos distribuidos y consenso

- Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017. (DDIA)
- Ongaro, D.; Ousterhout, J. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC, 2014.
- Ongaro, D. *Consensus: Bridging Theory and Practice* (tesis; *pre-vote, leadership transfer, learners, snapshots*).
- Petrov, A. *Database Internals*. O'Reilly. (Storage engines, replicación, consenso.)
- Apache Kafka — *Documentation*: diseño del log particionado, *high-watermark* y formato *record batch* v2.

Arquitectura de brokers de referencia

- Redpanda — arquitectura *thread-per-core* / *shared-nothing* (sobre Seastar) y Raft por partición.
- Seastar — *framework* asíncrono (referencia conceptual del reactor *per-core*: `submit_to`, `futures`, *shared-nothing*).

Concurrencia y rendimiento

- Herlihy, M.; Shavit, N. *The Art of Multiprocessor Programming*. (Concurrencia, lock-free, consenso.)
- Williams, A. *C++ Concurrency in Action*. (Modelo de memoria, atómicos, lock-free.)
- Gregg, B. *Systems Performance*. (Método USE, *profiling*, latencia.)
- Tene, G. *How NOT to Measure Latency* / HdrHistogram. (*Coordinated omission*, percentiles.)
- Drepper, U. *What Every Programmer Should Know About Memory*. (Caché, NUMA, localidad.)

Sistemas, SO e I/O

- Bryant, R.; O'Hallaron, D. *Computer Systems: A Programmer's Perspective* (CS:APP).
- Kerrisk, M. *The Linux Programming Interface* (TLPI). (`syscalls`, `fsync`, `mmap`.)
- Axboe, J. *Efficient IO with io_uring*. (Documento de diseño de `io_uring`.)

C++ y diseño

- Meyers, S. *Effective Modern C++*.
- Stroustrup, B.; Sutter, H. *C++ Core Guidelines*.
- Stroustrup, B. *A Tour of C++*.
- Nygard, M. *Release It!*. (Circuit Breaker, Bulkhead, patrones de estabilidad.)
- Martin, R. C. *Clean Code / Clean Architecture*.

Las convenciones de proyecto (naming, errores, testing, concurrencia, memoria, rendimiento, redes, datos distribuidos, seguridad, CI/CD) están normalizadas en la `BibliotecaDocumentacion` , que es la autoridad de estilo importada por el `CLAUDE.md` .

Diagrama 1: Contexto del sistema (C4 nivel 1)

NexusMQ visto como una caja negra y los actores y sistemas externos que interactúan con él: productores y consumidores (cliente nativo y `kcat` por el subset Kafka), administradores (REST/CLI), Prometheus (scrape de `/metrics`) y los demás nodos del propio cluster.

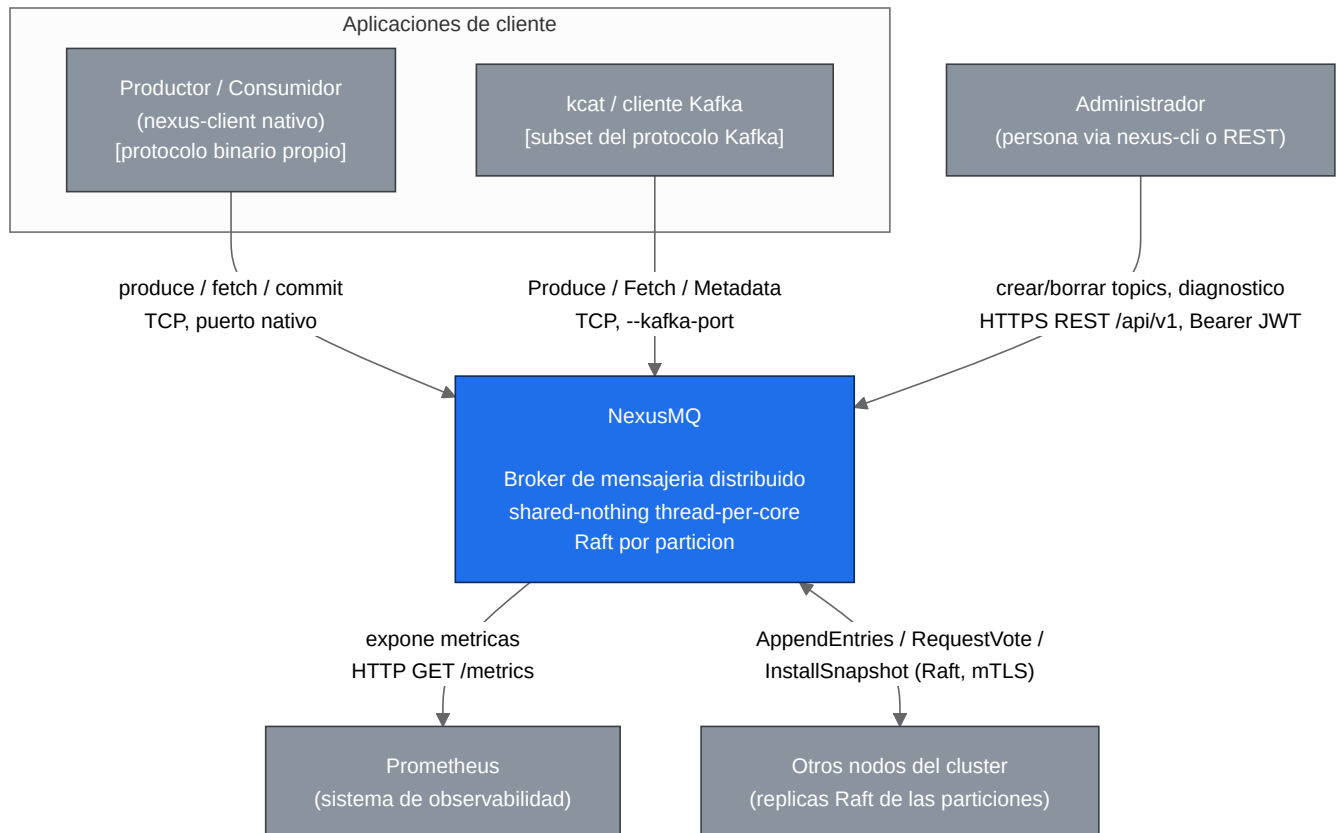
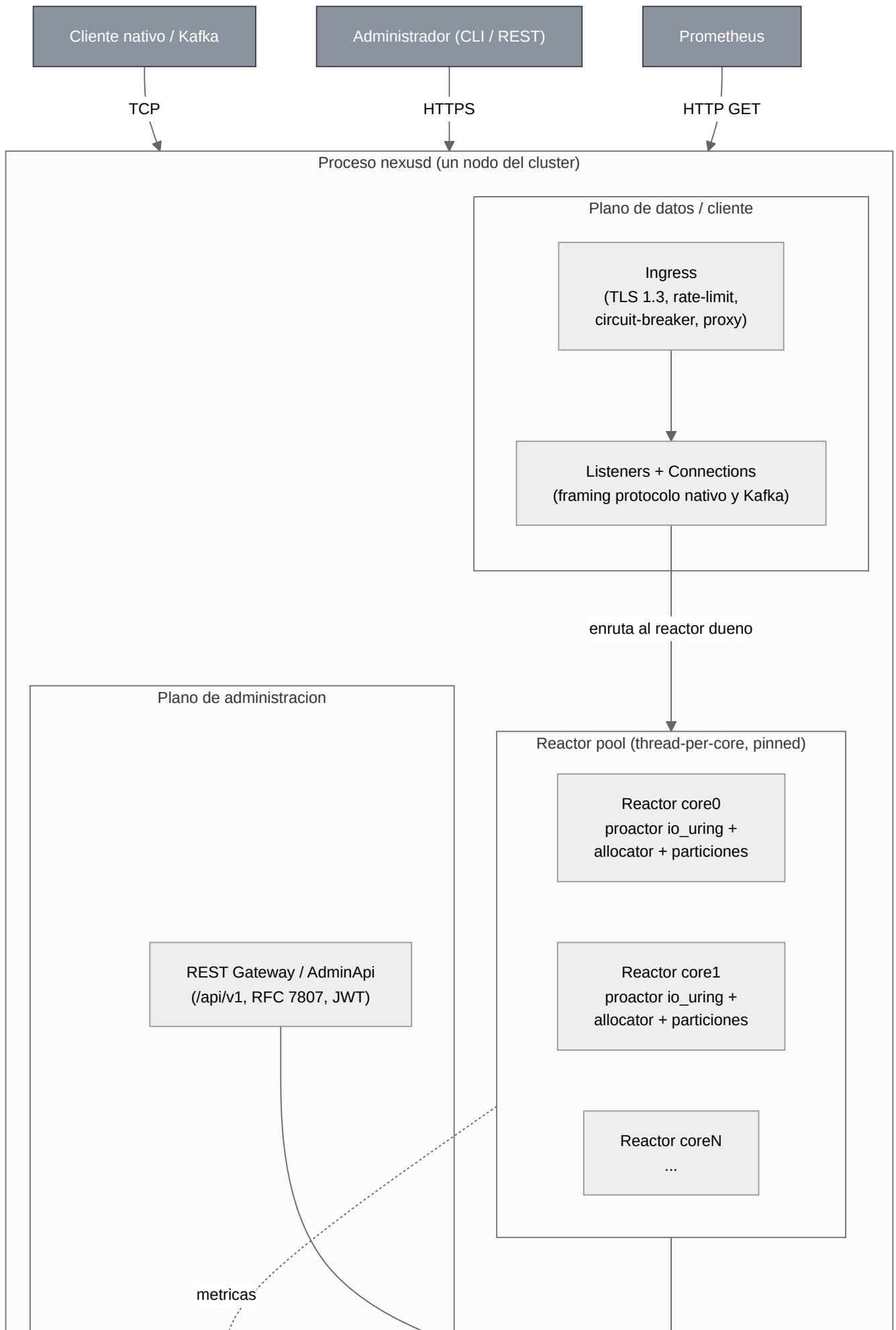


Diagrama 2: Contenedores de un nodo (C4 nivel 2)

Vista interna de un proceso `nexusd` : los tres planos (datos/cliente, Raft inter-nodo y administración REST), el *reactor pool* thread-per-core que ejecuta la lógica del broker, y el almacenamiento append-only en disco. Cada reactor es dueño exclusivo de un subconjunto de réplicas de partición; la interacción entre núcleos es paso de mensajes (SPSC), nunca estado compartido.



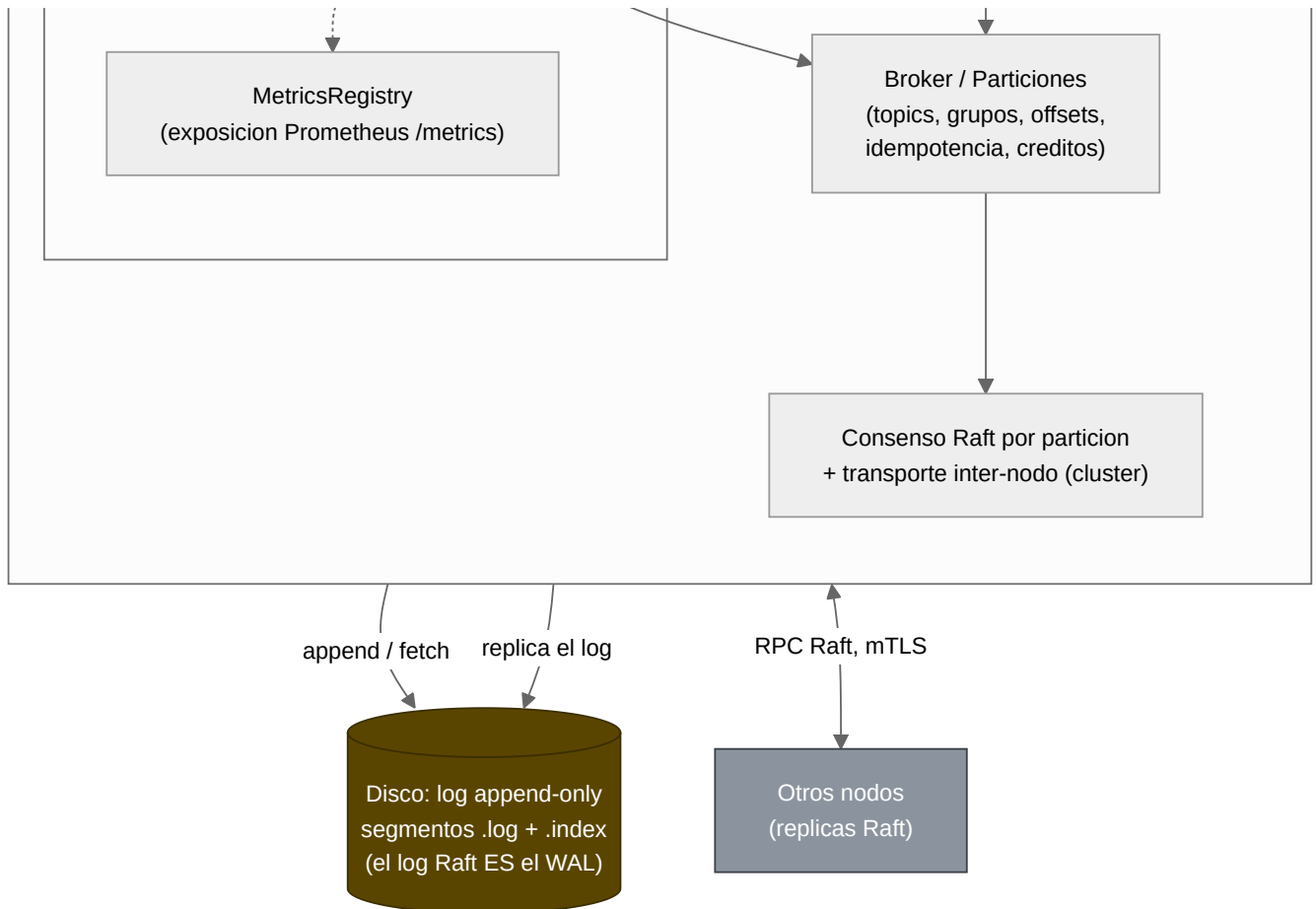


Diagrama 3: Grafo de dependencias de targets

Dependencias internas entre las 15 librerías `nexus-*`, los ejecutables (`nexusrd`, `nexus-cli`, `nexus-bench`, `nexus-loadgen`) y la librería compartida `nexus-ffi`. Refleja el grafo real de dependencias del proyecto: las capas suben de abajo (`nexus-common`, sin dependencias internas) hacia arriba (`nexus-server` / `nexus-client`). Regla forzada en CMake: un target nunca depende de otro de su misma capa o superior. Una flecha `A --> B` significa "A depende de B".

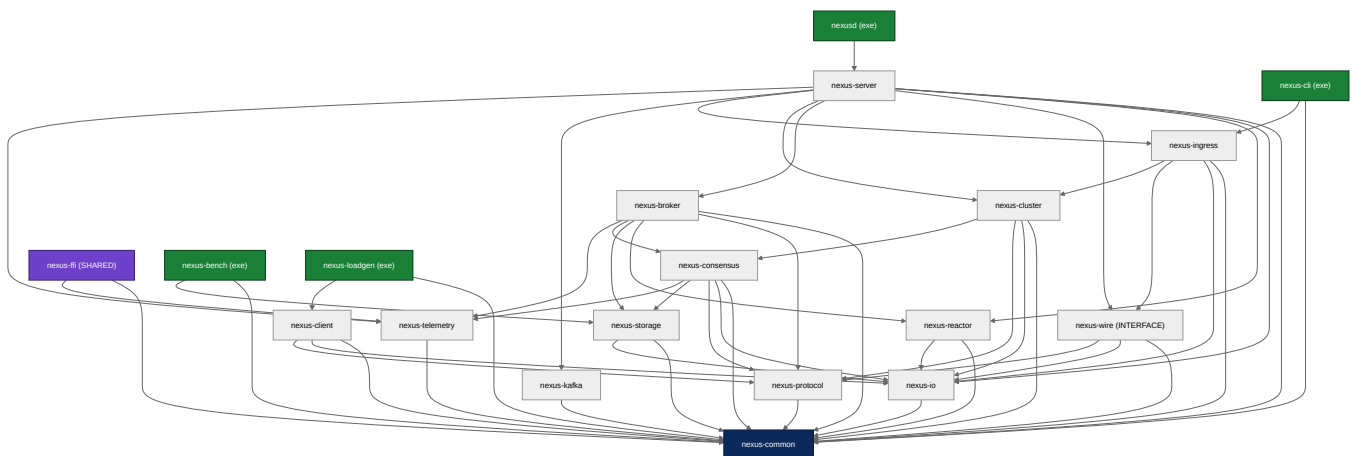


Diagrama 4: Mapa fase → targets

Qué librerías y ejecutables se entregan en cada fase del plan de desarrollo. Las fases son incrementales: cada una se apoya en lo construido antes. Estado as-built: las fases 1 → 4 están implementadas, con el cierre 4b (Windows, deuda diferida y benchmarks).

Tabla resumen

Fase	Foco	Targets entregados
1	Monohilo + I/O bloqueante	nexus-common , nexus-storage (I/O bloqueante), nexus-bench , nexus-tests (unit/property/crash)
1b	Reactor + mono-nodo	nexus-io (proactor), nexus-reactor , nexus-protocol , nexus-wire , nexus-broker (mono-nodo), nexus-client , nexus-server (mono-nodo); tests de integración
2	Distribuido (Raft)	nexus-consensus , nexus-cluster , broker distribuido (grupos, rebalanceo); tests sim/chaos
3	Ingress + observabilidad	nexus-ingress , nexus-telemetry , nexus-cli , deploy/ (Docker, Prometheus, Grafana, k8s)
4	Stretch	direct I/O, nexus-kafka (subset), nexus-ffi (binding Python), backend IOCP (Windows)
4b	Cierre	bloque W (nexusrd Windows completo), bloque D (deuda diferida), bloque L (nexus-loadgen + benchmarks)

Diagrama

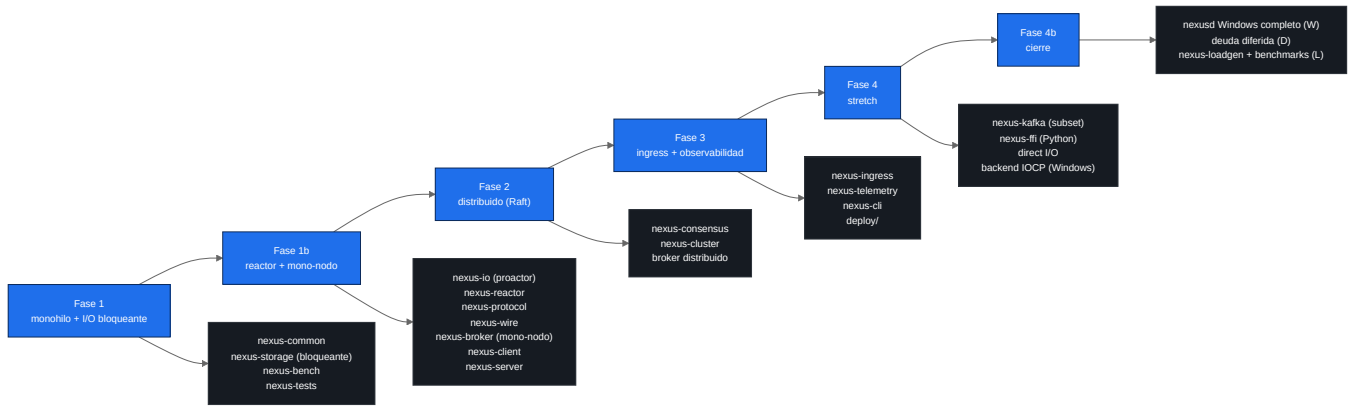
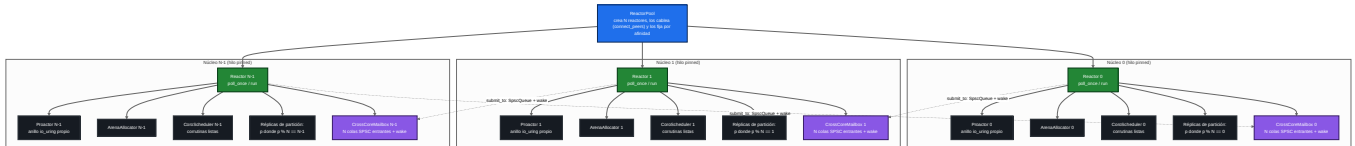


Diagrama 5: Topología thread-per-core (shared-nothing)

Cómo NexusMQ materializa el modelo *shared-nothing thread-per-core* (ADR-0005): el `ReactorPool` crea N `Reactor`, uno por núcleo físico, y fija cada hilo a su núcleo (`pthread_setaffinity_np`). Cada `Reactor` es **dueño** de todo su estado reactor-local —su anillo `io_uring` (`Proactor`), su `ArenaAllocator`, su `CoroScheduler`, sus temporizadores y el subconjunto de réplicas de partición que le tocan ($p \% N$)— y **no comparte nada mutable** con los demás. El único canal entre reactores es el paso de mensajes: cada reactor expone un `CrossCoreMailbox` con una cola `spscQueue` lock-free por núcleo origen; `Reactor::submit_to` encola en la cola del destino y lo despierta (`Proactor::wake`). Así no hay locks ni *cache ping-pong* en el plano de datos.



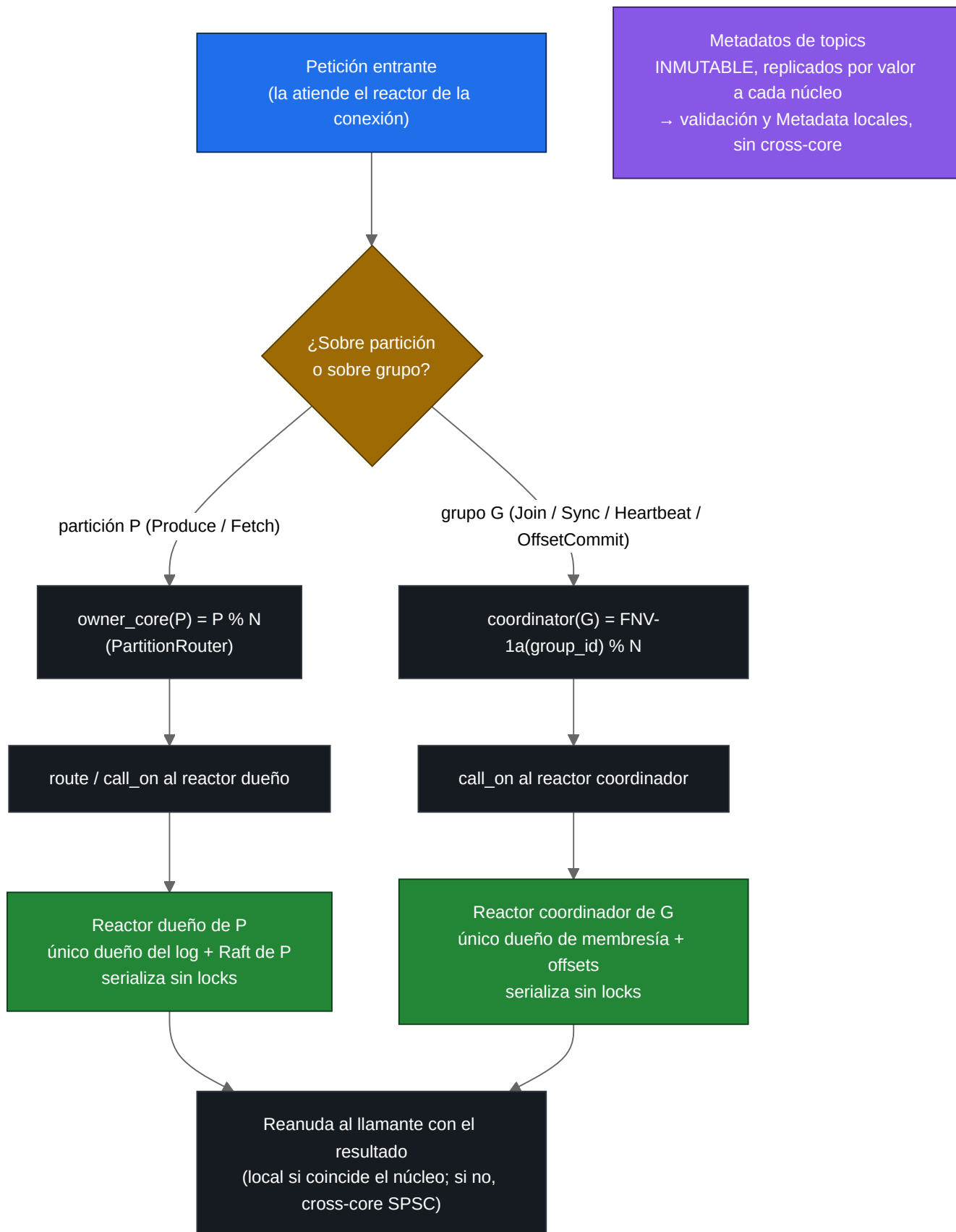
Las flechas continuas son propiedad (cada reactor posee su estado local). Las flechas discontinuas son el **único** acoplamiento entre núcleos: trabajo movido por `submit_to` a la `spscQueue` del buzón destino, seguido de un `wake`. Cada cola SPSC tiene un único productor (el núcleo origen) y un único consumidor (el reactor dueño), con `head_ / tail_` en líneas de caché distintas (`alignas`) para evitar *false sharing*.

Diagrama 6: Sharding del plano de datos por núcleo

Cómo se reparte el estado entre los N reactores sin compartir nada mutable (ADR-0026, sobre ADR-0005). Cada pieza de estado tiene un **dueño único**, lo que la hace linealizable sin locks: un único núcleo serializa todas las operaciones sobre ella. Dos reglas de ubicación gobiernan el plano de datos:

- **Partición** → **núcleo dueño** = `partition % N` (lo fija `PartitionRouter::owner_core`, la misma regla que `ReactorPool::reactor_for`). El log y la pila Raft de cada partición viven **solo** en su reactor dueño; el resto de núcleos ni la instancian.
- **Grupo de consumidores** → **núcleo coordinador** = `hash(group_id) % N` con `hash = FNV-1a` sobre los bytes del `group_id` (contrato interno de ubicación, no viaja por *wire*). La membresía (`GroupCoordinator`) y los offsets confirmados (`OffsetManager`) viven en un solo núcleo, el coordinador del grupo.

Si la conexión la atiende un núcleo distinto del dueño/coordinador, la operación se enruta con `call_on` (petición/respuesta cross-core por el buzón SPSC); si coincide, es un *fast-path* local sin salto. Los metadatos de topics son la excepción: inmutables entre cambios y replicados por valor a cada núcleo, se validan localmente sin cross-core.

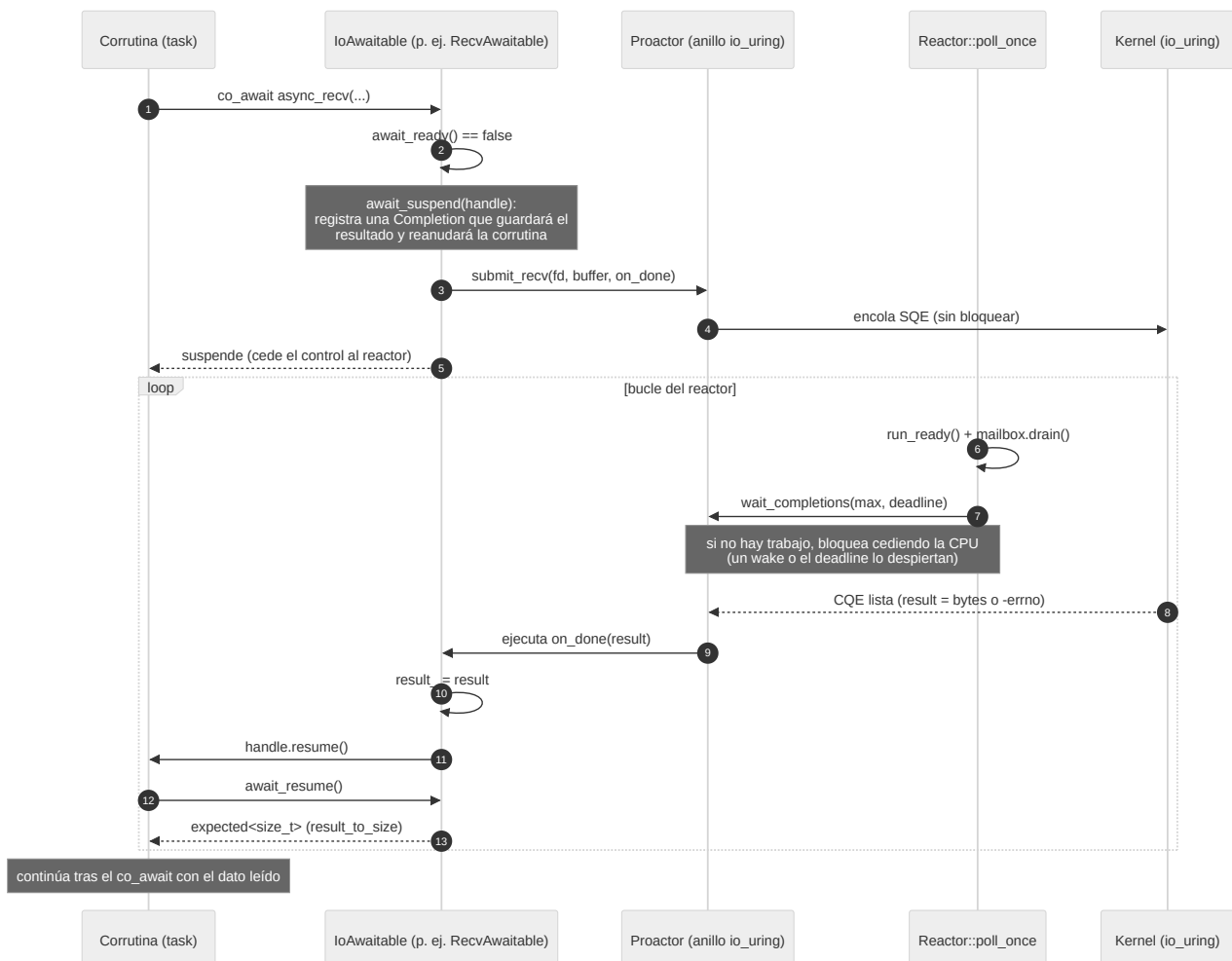


El reparto por `partition % N` y por `FNV-1a(group_id) % N` da a cada shard un dueño único: no hay un `TopicManager` ni un coordinador global con lock, sino confinamiento por reactor. El coste es que

tocar una partición o un grupo de otro núcleo paga un salto cross-core (ver [topología](#) y [secuencia del proactor](#)).

Diagrama 7: Secuencia de una operación de E/S sobre el proactor

El ciclo de una operación de E/S asíncrona modelada como *proactor* por *completions* (ADR-0002): una corrutina hace `co_await` sobre un `awaitable` (`ReadAwaitable`, `RecvAwaitable`, `WriteAwaitable` ...), que **encola** la operación en el anillo `io_uring` del reactor (`Proactor::submit_*`) y suspende la corrutina sin bloquear el hilo. El bucle del reactor (`poll_once`) sigue atendiendo otras tareas; cuando no hay nada que hacer, **bloquea** en `wait_completions` cediendo la CPU. Al terminar la operación, el reactor drena la *completion* del anillo y ejecuta su callback, que guarda el resultado (`result_`) y reanuda la corrutina justo donde quedó. En `await_resume`, el resultado estilo `io_uring` (`>= 0` éxito, `< 0` = `-errno`) se traduce a `expected<T>` (modelo de errores del núcleo, ADR-0009).



Las *completions* se sirven **fuera** del scheduler de corrutinas: `submit_*` nunca ejecuta el callback de forma síncrona (reentraría en una corrutina aún suspendiéndose), siempre se difiere a `wait_completions / run_completions`. El `buffer` y, en `connect`, la dirección deben sobrevivir hasta la *completion*: viven en el *frame* de la corrutina que hace `co_await`. Para enrutar a otro núcleo antes o después de la E/S, ver [sharding](#) y [topología](#).

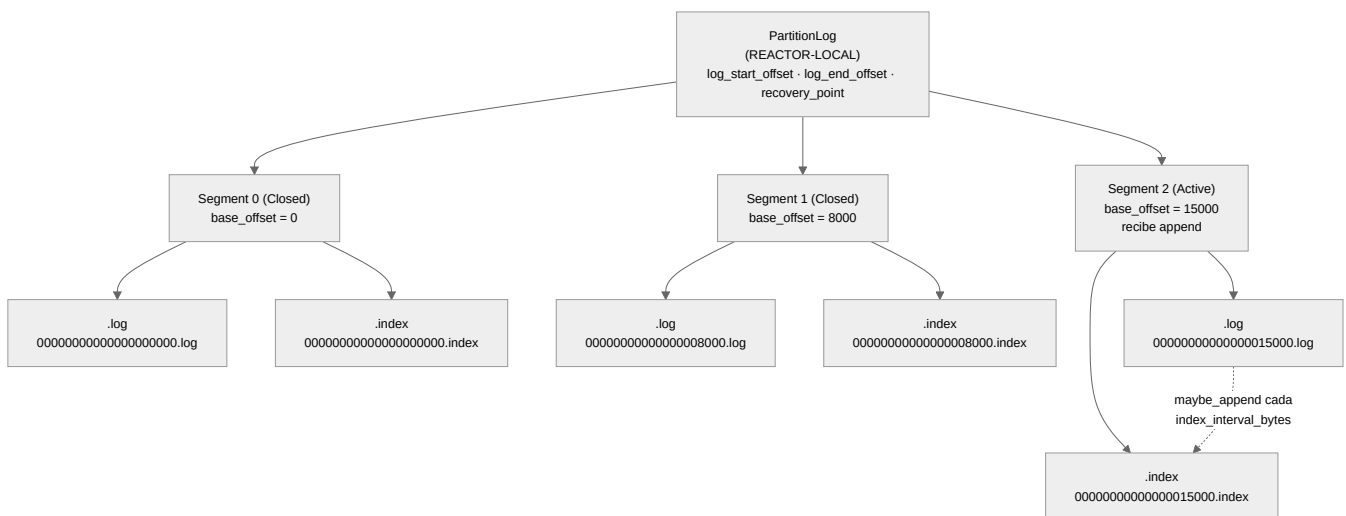
Diagrama 8: Layout del log de una partición

El log de una partición es una secuencia *append-only* de **segmentos** sellados más un **segmento activo**; cada segmento es un par de ficheros `.log` (los `RecordBatch` en orden de *offset*) y `.index` (índice disperso *offset* → posición). Este diagrama detalla esa estructura física, el byte-layout de la cabecera de batch y la mecánica de *seek*.

Fuentes: `src/storage/partition_log.hpp`, `src/storage/segment.hpp`, `src/storage/index.hpp`, `src/common/record.{hpp,cpp}`. Contrato de bytes: `../protocol.md` (§ «Registros y batches»).
Diseño: capítulo 9 (Almacenamiento).

Composición: `PartitionLog` → `Segment` → (`.log` / `.index`)

Un `PartitionLog` ordena sus `Segment` por *offset* base; el último es el **activo** (recibe `append`). Al superar el activo `segment_bytes` (por defecto 64 MiB en `LogConfig`, configurable como `segment.bytes`), el siguiente `append` **rota**: sella el activo (`seal`: persiste el `.index` y `fsync`) y abre uno nuevo en `log_end_offset()`. Afinidad de todos estos tipos: **REACTOR-LOCAL** (no *thread-safe*).



- **Nombrado de ficheros:** cada segmento se nombra con su *offset* base a **20 dígitos** con relleno de ceros, de modo que el orden lexicográfico coincide con el orden de *offset* (`Segment` : `00000000000000000000000000000000.log` / `.index`).
- **Estados del segmento** (`Segment::State`): `Active` (admite `append`; es el último del log) → `Closed` (sellado, índice persistido, solo lectura). La retención (Diagrama 9) borra después segmentos `Closed` enteros.
- **Offsets del log** (`PartitionLog`): `log_start_offset()` (base del primer segmento), `log_end_offset()` (siguiente *offset* a asignar) y `recovery_point()` (hasta dónde está sincronizado a disco estable, garantía de durabilidad).

Byte-layout del `.log`: secuencia de `RecordBatch`

El `.log` es una concatenación de `RecordBatch` (estilo Kafka v2) escritos en orden por su *offset* base. El batch viaja **intacto** por el log y la replicación: es la unidad de escritura y de entrada de Raft (ADR-0014). El

payload de records se trata como **bytes opacos** a este nivel.

```
.log = [ RecordBatch ][ RecordBatch ][ RecordBatch ]... (append-only)
```

RecordBatch (little-endian; cabecera fija = kHeaderSize = 36 bytes):

offset	campo	tipo	notas
0	base_offset	i64	offset del primer record (NO cubierto por el CRC)
8	length	i32	bytes tras este campo (NO cubierto por el CRC)
12	crc	u32	CRC32C; cubre desde attrs (16) hasta el final
16	attrs	u16	códec de compresión (2 bits bajos) + flags ↴
18	producer_id	i64	productor idempotente (-1 = ninguno) región
26	producer_epoch	i16	época del productor cubierta
28	base_sequence	i32	secuencia base (idempotencia) por el CRC
32	record_count	i32	nº de records del payload
36	records[]	bytes	blob opaco (puede ir comprimido, F5) ↵

Notas fieles al código (`src/common/record.cpp`):

- **length** cuenta los bytes **tras** el propio campo `length`: `encoded_size = 12 (offset+length) + length`, o equivalentemente `kHeaderSize + records.size()`.
- **El CRC32C cubre [attrs .. fin]**, es decir, la cola de cabecera (desde el byte 16) más el blob de records. `base_offset`, `length` y `crc` quedan **fuera** del CRC: por eso el log puede **reasignar** `base_offset` al anexar (el líder asigna el *offset* autoritativo) sin invalidar la integridad.
- **Compresión (F5)**: los **2 bits bajos** de `attrs` indican el códec (0 =None, 1 =LZ4, 2 =Zstd). El broker guarda y replica el blob comprimido como opaco; solo el cliente lo descomprime. Ver [../protocol.md](#).
- **Decodificación defensiva** (`decode / peek`): se acota `length` (no negativo, `total ≤ bytes disponibles`) antes de derivar el tamaño; un batch *torn* (escrito a medias por un *crash*) se detecta así y por el CRC. Invariante de serialización: `decode(encode(x)) == x`.

Byte-layout de cada `record` dentro del blob

Cada record del blob va con prefijo de longitud (estilo Kafka v2, `src/common/record_codec.hpp`):

```
record = length:varint(zigzag) | attributes:i8 | timestampDelta:varint |  
        offsetDelta:varint | keyLen:varint(-1=nulo) | key |  
        valueLen:varint(-1=nulo) | value | headerCount:varint | headers[]
```

- El **offset absoluto** de un record se reconstruye al decodificar como `base_offset + offsetDelta`.
- Un record con `value` nulo es un **tombstone** (marca de borrado por clave para la compactación; ver Diagrama 9).
- Topes anti-DoS al decodificar entrada no confiable: `kMaxRecordsPerBatch` (1 000 000), `kMaxHeadersPerRecord` (10 000), `kMaxRecordBytes` (64 MiB descomprimido).

Byte-layout del `.index`: índice disperso (`sparseIndex`)

El `.index` **no** indexa cada batch, sino uno aproximadamente cada `index_interval_bytes` (por defecto 4096 en `LogConfig`): un compromiso espacio/tiempo que mantiene el índice pequeño y permite *seek* con búsqueda binaria más un barrido corto del `.log`.

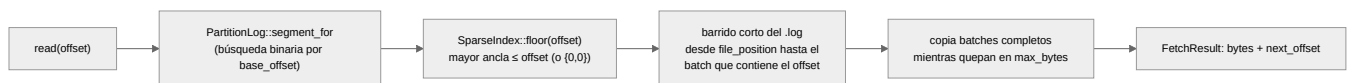
```
.index = [ IndexEntry ][ IndexEntry ]... (ordenadas estrictamente por offset)
```

`IndexEntry` (little-endian; `kEntrySize` = 8 bytes):

offset	campo	tipo	notas
0	<code>relative_offset</code>	u32	offset absoluto - <code>base_offset</code> del segmento
4	<code>file_position</code>	u32	byte donde empieza ese batch en el <code>.log</code>

- `relative_offset` = offset absoluto - base del segmento; cabe en u32 porque el segmento está acotado por `segment_bytes`. Ambos campos crecen de forma **estrictamente monótona** a lo largo del índice (invariante).
- `SparseIndex` posee el fichero `.index` (RAII) y mantiene las entradas en memoria como espejo del disco; `maybe_append(offset, file_position, batch_len)` siembra una entrada solo cuando el intervalo lo exige; `flush()` persiste lo pendiente y hace `fsync` (al sellar); `reset()` lo vacía para reconstruirlo durante `recover`.

Mecánica de seek (fetch desde un offset)



- `PartitionLog::read` localiza el segmento por `offset`, delega en `Segment::read` y, si no se llena `max_bytes` y quedan datos, **continúa en el segmento siguiente** (lectura cruzando segmentos) hasta `log_end_offset()`. `OutOfRange` si el `offset` es anterior a `log_start_offset()`.
- `Segment::read` no revalida el CRC (eso es de `recover`); ante un batch *torn* al final, se detiene.

Recuperación al arrancar (`Segment::recover`)

Al abrir el log, `PartitionLog::open` descubre los `.log` del directorio, abre todos los segmentos y **recupera el activo**: recorre los batches desde el inicio validando `length` y CRC32C, se detiene en el primer batch incompleto o corrupto (la cola *torn* de un *crash*), **trunca el .log** justo tras el último batch válido y **re-siembra el .index**. Así se fija `log_end_offset()` y `recovery_point`.

Durabilidad (`FsyncPolicy`)

`PartitionLog::maybe_sync` aplica la política de `fsync` configurada (`LogConfig::fsync_policy`) tras cada `append`:

Política	Cuándo hace fsync	Compromiso
None	Nunca explícitamente (durabilidad solo por el SO)	Máximo rendimiento
Interval	Al acumular <code>fsync_interval_bytes</code> (def. 1 MiB)	Equilibrado (por defecto)
Commit	En cada <code>append</code>	Durabilidad por escritura, el más lento

Tras un fallo, solo lo sincronizado hasta `recovery_point` está garantizado. Cada `seal` (rotación) sincroniza el segmento y avanza `recovery_point` a `log_end_offset`.

Diagrama 9: Retención y compactación del log

El log de una partición se acota por dos mecanismos independientes: la **retención** borra segmentos sellados enteros cuando superan `retention.ms / retention.bytes`, y la **compactación por clave** (`LogCompactor`) conserva solo el último record por clave. Este diagrama detalla ambos flujos tal como los implementa `nexus-storage`.

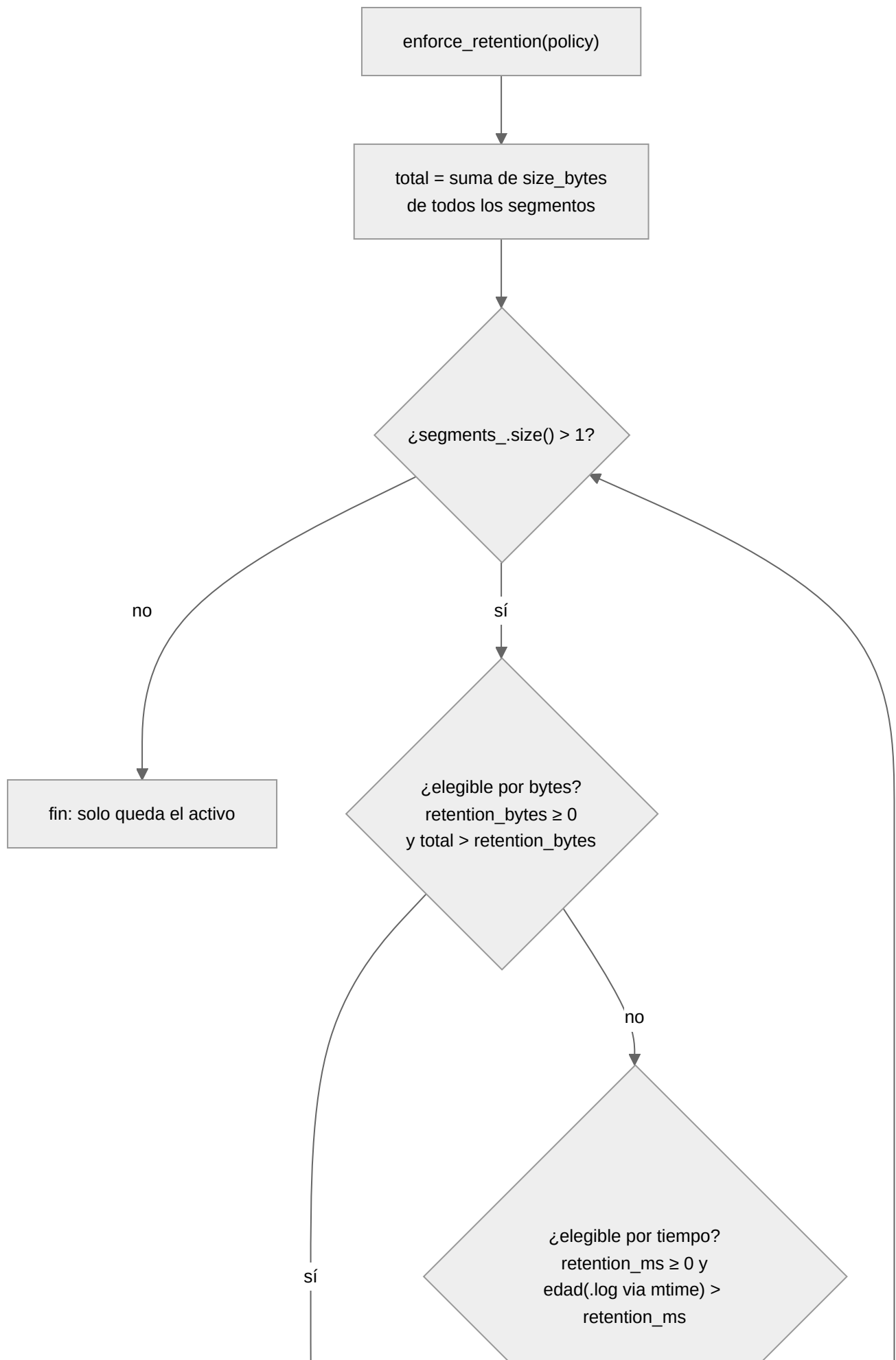
Fuentes: `src/storage/partition_log.{hpp,cpp}` (`enforce_retention`),
`src/storage/retention.hpp` (`RetentionPolicy`), `src/storage/log_compactor.{hpp,cpp}`
(`LogCompactor`). Diseño: [capítulo 9 \(Almacenamiento\)](#). Mecanismo vs política: el storage ofrece el mecanismo; la política la fija el llamante.

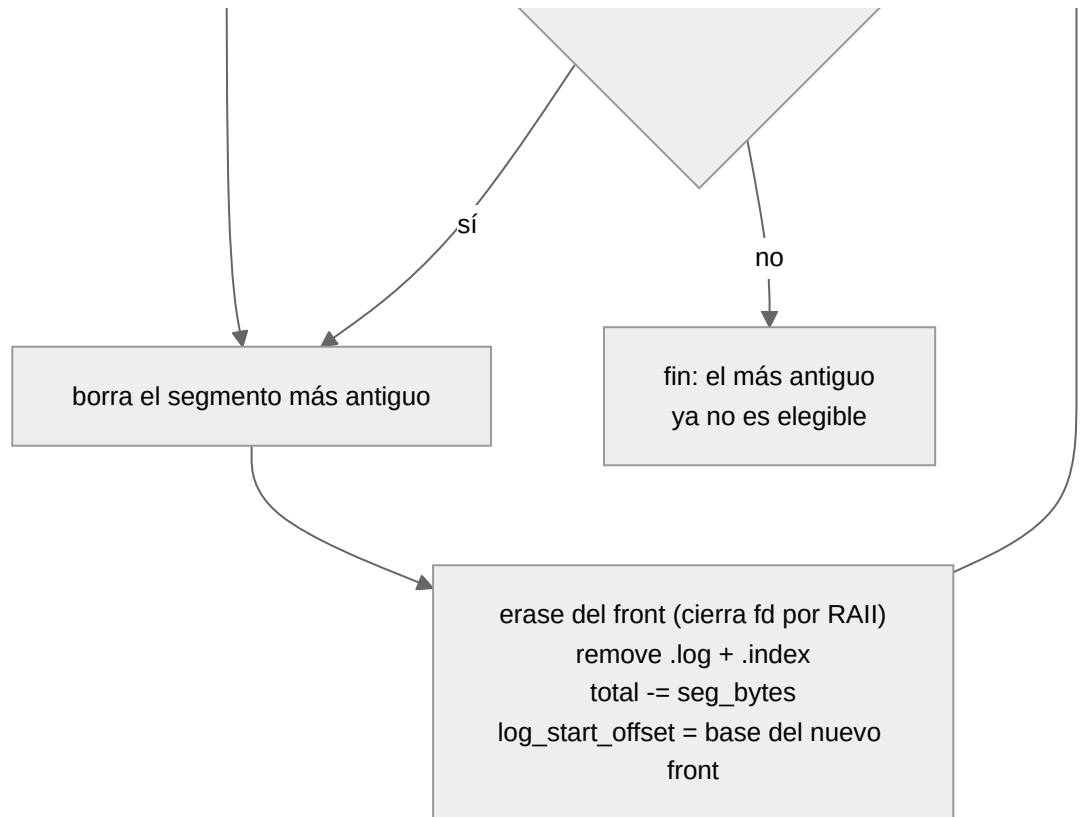
1. Retención por segmentos sellados enteros (`PartitionLog::enforce_retention`)

La retención **nunca borra a media trama ni el segmento activo**: la reclamación física es siempre por **segmentos `closed` completos**. Recorre del más antiguo al más nuevo mientras quede más de un segmento (`segments_.size() > 1`) y el más antiguo sea **elegible**; en cuanto el más antiguo deja de serlo, el resto tampoco lo es y se detiene.

Criterios de elegibilidad (`RetentionPolicy` , ambos opcionales con `< 0` = sin límite):

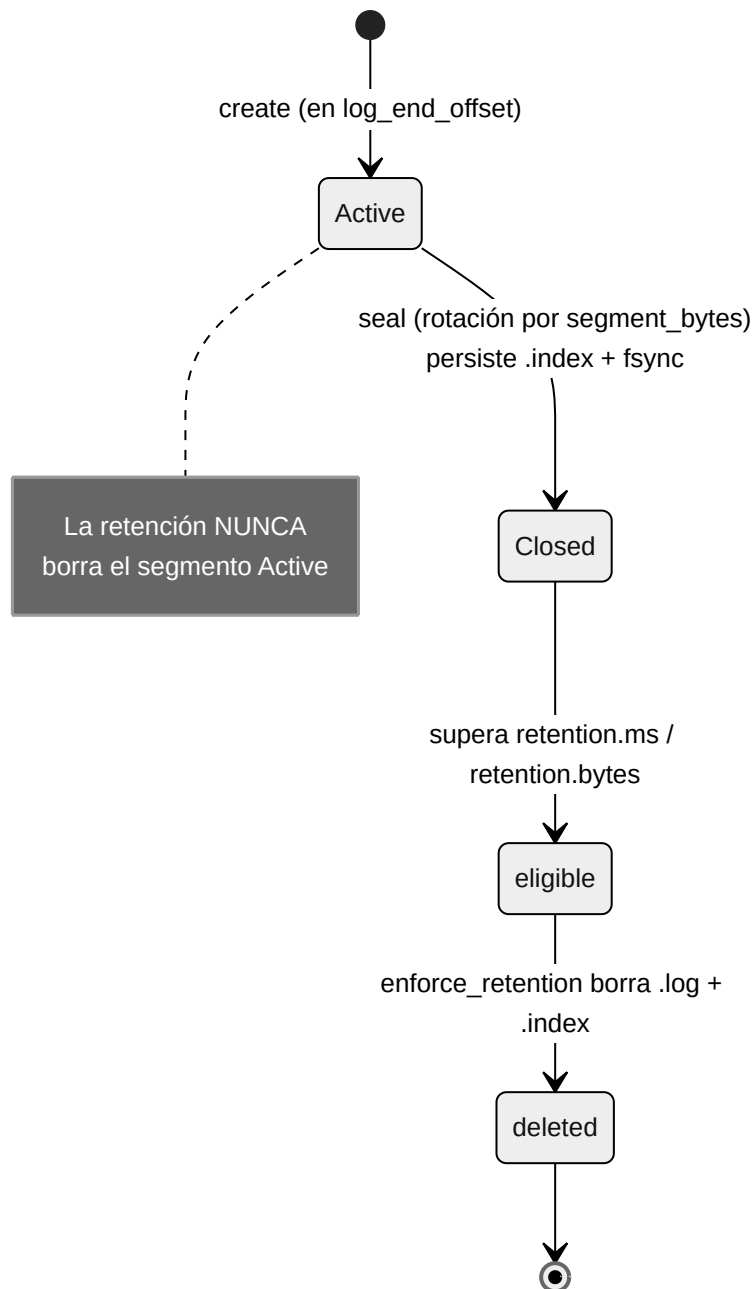
- **Por tamaño** (`retention_bytes`): el **total** de bytes del log supera `retention_bytes`.
- **Por tiempo** (`retention_ms`): la edad del segmento, tomada del *mtime* de su `.log`, supera `retention_ms`. (Los records aún no llevan *timestamp*; cuando lo lleven, podrá usarse el *timestamp* máximo del segmento.)





Efectos de borrar un segmento (fieles a `enforce_retention`): se quita del frente del vector (lo que **cierra sus descriptores por RAI**), se eliminan los ficheros `.log` e `.index`, se descuenta su tamaño del total y se **avanza** `log_start_offset()` hasta la base del nuevo primer segmento.

Ciclo de vida del segmento



`eligible` y `deleted` son fases lógicas del ciclo de vida; en el código el estado persistente del `Segment` es solo `Active` / `Closed`, y la elegibilidad se evalúa dentro de `enforce_retention`.

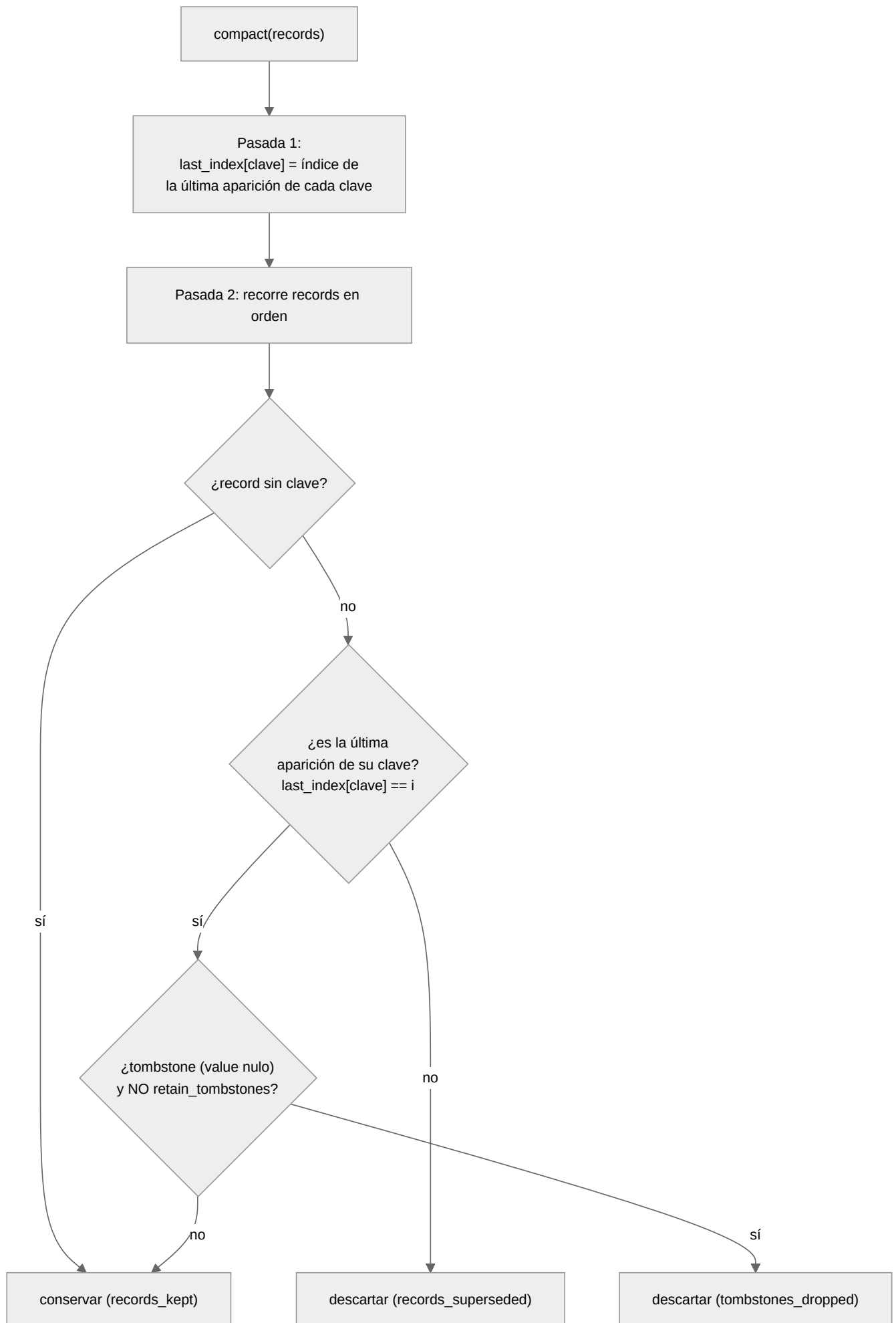
2. Compactación por clave (`LogCompactor`)

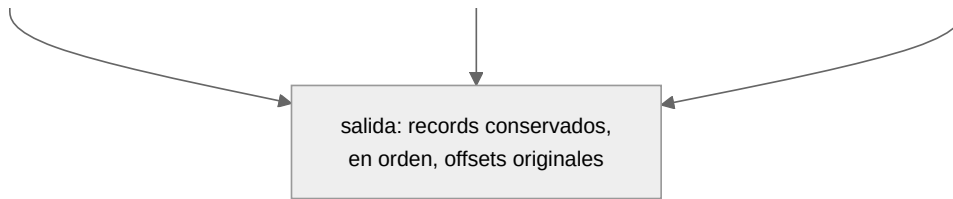
Política **alternativa/complementaria** a la retención por tiempo/tamaño (§6.5): conserva, **por clave, solo el record más reciente** (la última aparición en orden de *offset*); las apariciones anteriores se descartan. Es REACTOR-LOCAL y **sin estado mutable**. Los *offsets* originales se **preservan** (pueden quedar huecos), igual que en Kafka.

Reglas (de `compact`):

- **Records sin clave** (`key == null`): se conservan **siempre** (la compactación solo colapsa por clave).

- **Record con clave reemplazado** por otro posterior con la misma clave: se descarta (`records_superseded`).
- **Tombstone** (record con `value == nullopt`): supera a los anteriores de su clave y, salvo que se construya el `LogCompactor` con `retain_tombstones = true` , se **descarta también él** (`tombstones_dropped`), haciendo desaparecer la clave del log compactado.





`CompactStats` reporta la pasada: `records_in` , `records_kept` , `records_superseded` , `tombstones_dropped` .

Lectura del log completo (`compact_log`)

`compact_log(log)` lee **todos** los records del `PartitionLog` (de `log_start_offset()` a `log_end_offset()`) por trozos de hasta 4 MiB, decodificando cada `RecordBatch` (`peek` para el tamaño, `decode` con validación de CRC, `decode_records` para el blob, descomprimiéndolo si procede) y los compacta preservando los *offsets* absolutos. Devuelve error de E/S o `Corrupt` si el log está dañado; incluye defensas anti bucle infinito (sin progreso de `next_offset` o lectura vacía).

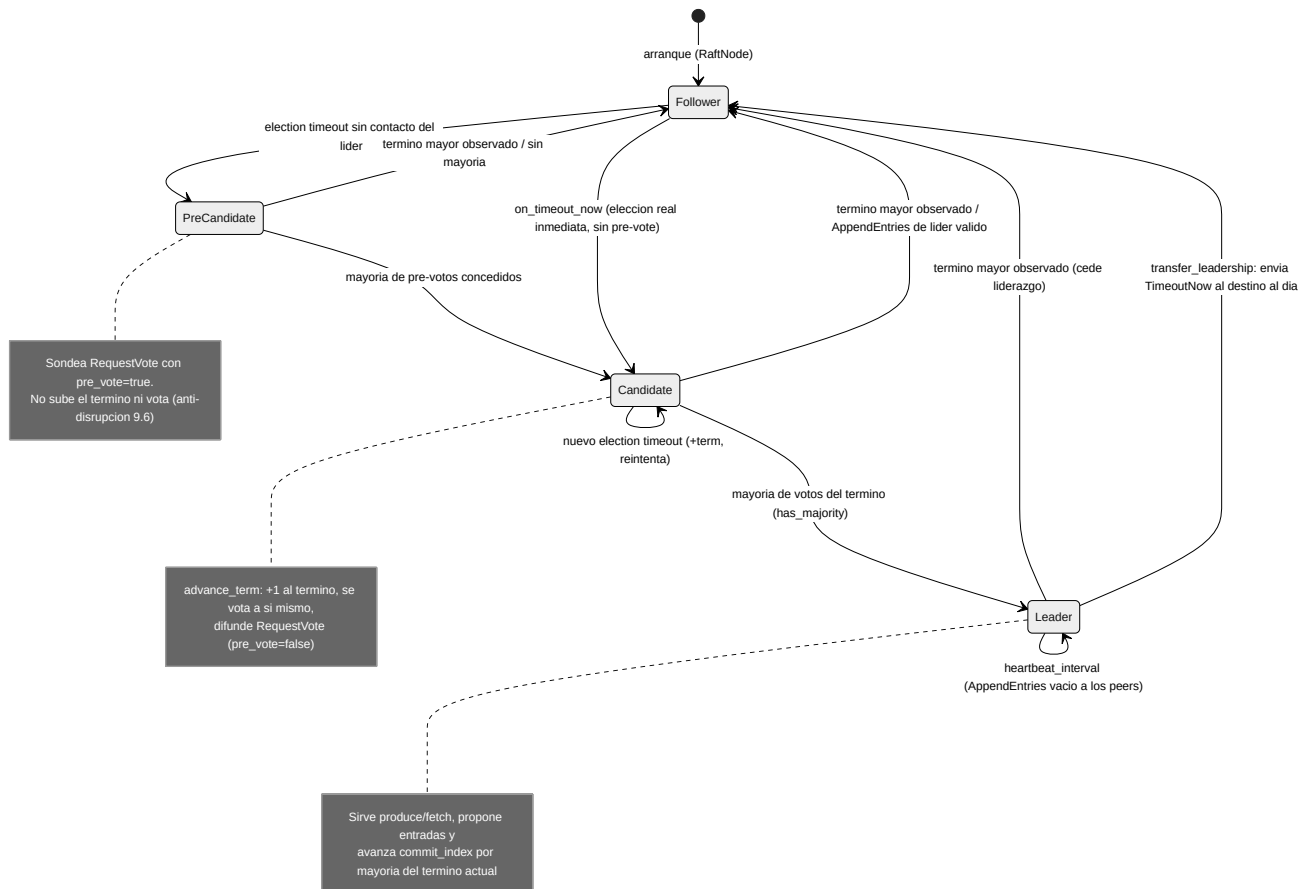
Relación entre ambos mecanismos

- La **retención** es física y gruesa (segmentos enteros); reclama espacio por antigüedad/tamaño sin mirar el contenido.
- La **compactación** es lógica y por clave; produce un conjunto de records compactado (la materialización a nuevos segmentos es responsabilidad del llamante, no de `LogCompactor`).
- Existe además un recorte de **prefijo por snapshot de Raft** (`PartitionLog::truncate_prefix_to` , ADR-0024): pieza simétrica de la retención que borra segmentos sellados enteros por debajo de un *offset* preciso (reclamación física por segmentos; la exactitud en el índice la lleva el `RaftLog`). Distinto de la política de tamaño/tiempo de este diagrama.

Ver también el [Diagrama 8](#) (layout del log y segmentos) y [../protocol.md](#) (formato del `RecordBatch` y *tombstones*).

Diagrama 10: Estados y transiciones de una réplica Raft

Ciclo de vida del rol de un `RaftNode` por partición (`Follower` $\rightleftharpoons$ `PreCandidate` $\rightleftharpoons$ `Candidate` $\rightleftharpoons$ `Leader`), con la fase de *pre-vote* (§9.6) y la transferencia ordenada de liderazgo (§3.10) tal como las implementa `src/consensus/raft_node.hpp` (ADR-0003/0015).

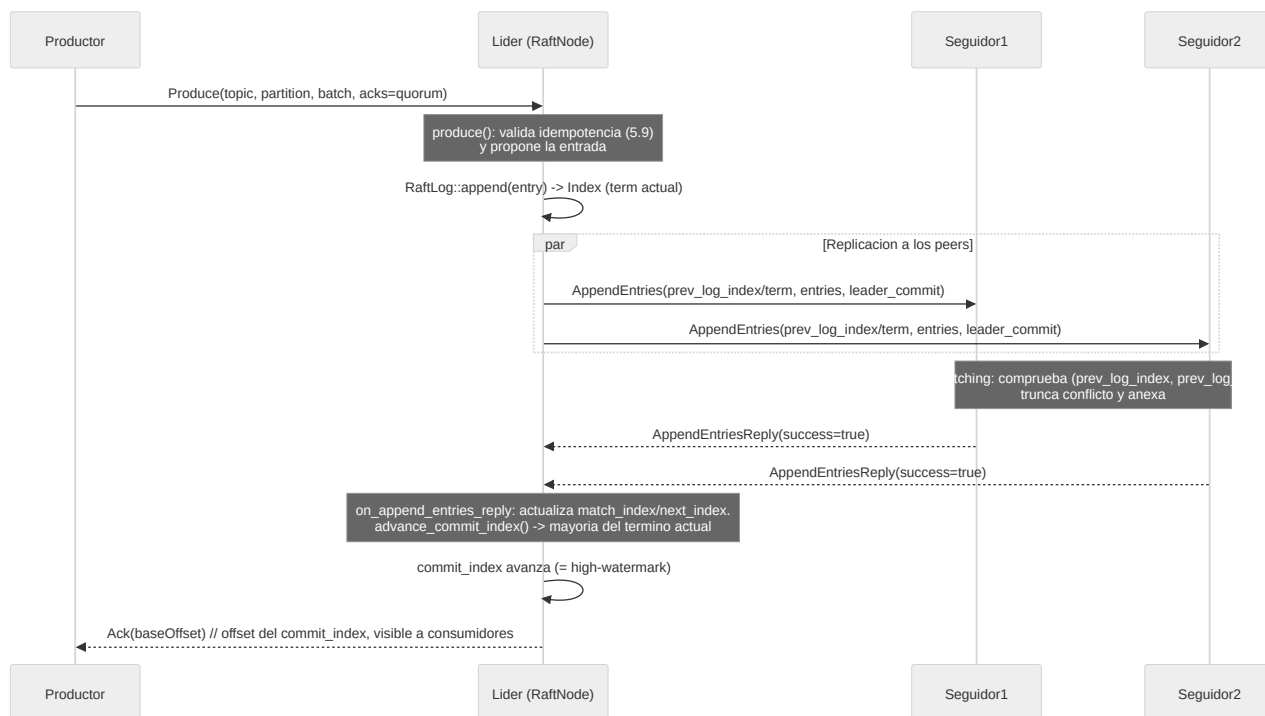


Regla transversal (cualquier rol): observar un término mayor en cualquier RPC degrada el nodo a `Follower` y adopta ese término (`become_follower`). Un `learner` (§4.2.1) nunca se postula: replica el log sin votar ni contar para el quórum.

Diagrama 11: Replicación y confirmación a quórum

(acks=quorum)

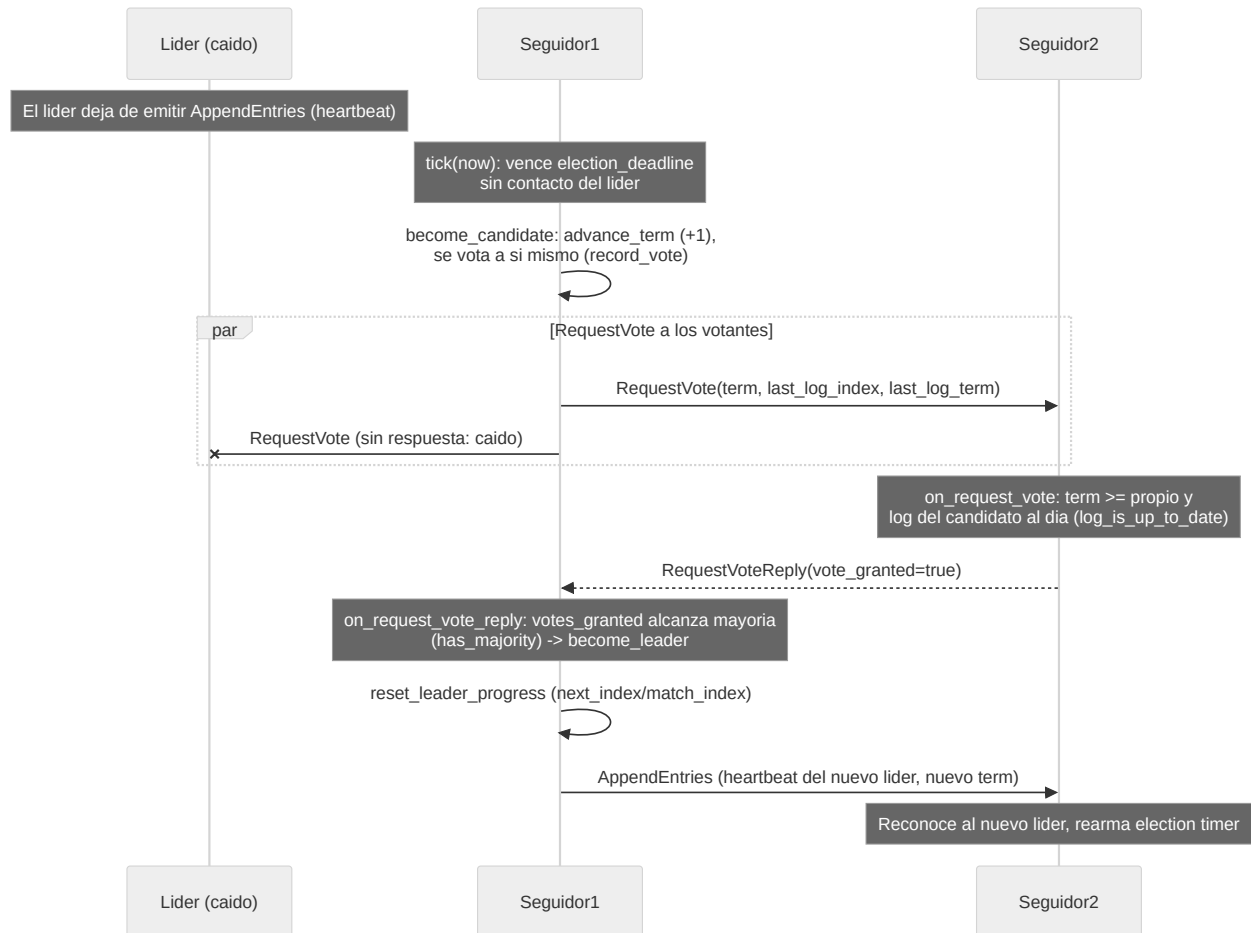
Camino de escritura con Raft por partición: el productor envía al líder, este añade la entrada al log, la replica con `AppendEntries`, y cuando una mayoría confirma avanza `commit_index` (= *high-watermark*) y responde (§7.8, ADR-0003/0014). Una entrada de Raft es un `RecordBatch`; `ReplicatedPartition::produce` propone y la escritura es durable cuando `high_watermark()` la supera.



El `commit_index` es la *high-watermark*: el offset de partición del `last_offset` de la entrada en `commit_index` (ADR-0014). `acks=0` responde sin esperar; `acks=1`, tras el *append* local del líder; `acks=quorum` (por defecto), tras el *commit* de Raft. Solo se cuenta en el quórum a los miembros votantes (los `learner` replican pero no cuentan).

Diagrama 12: Failover y elección de líder

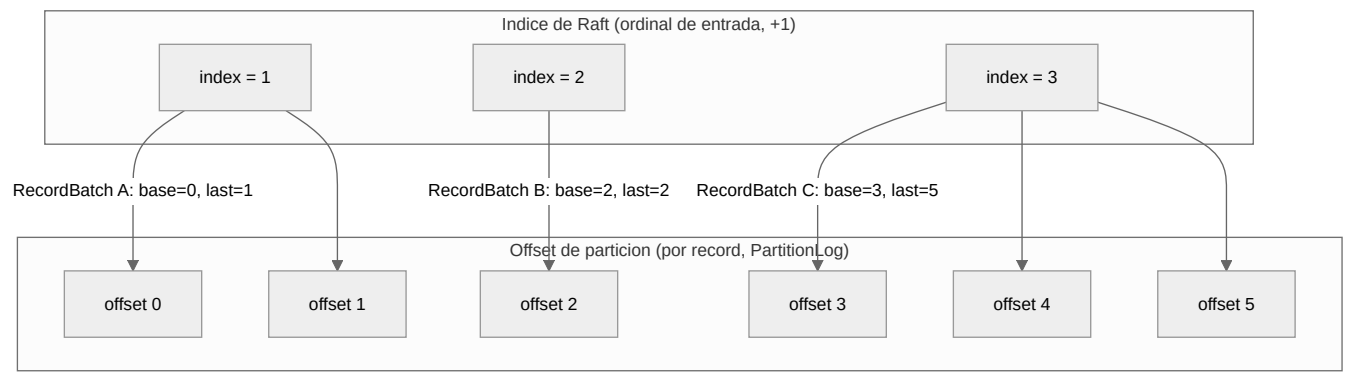
Cuando el líder cae, un seguidor agota su *election timeout* sin recibir *heartbeats*, se postula incrementando el término, difunde `RequestVote` y, al reunir mayoría, se convierte en líder y reanuda la replicación (§5.2, ADR-0003/0015). El diagrama omite la fase de *pre-vote* previa (§9.6) para centrarse en el *failover*.



El estado persistente (término y voto) se guarda con `fsync` **antes** de responder al RPC (regla §5 de Raft): lo gobierna `persistent_state_dirty()` y lo ejecuta el `RaftCarrier` antes de transportar. Ningún dato confirmado con `acks=quorum` se pierde mientras sobreviva la mayoría del grupo: el nuevo líder tiene, por la restricción de elección, todas las entradas *committed*.

Diagrama 13: Espacios de coordenadas (índice de entrada Raft vs offset por record)

NexusMQ mantiene **dos espacios de coordenadas distintos** (ADR-0014): el **índice de Raft** es el ordinal de cada entrada (1-based, +1 por entrada), mientras que el **offset de partición** es por *record* (un `RecordBatch` de N records ocupa N offsets). Una entrada de Raft equivale a un `RecordBatch`, así que cada índice mapea a un **rango** de offsets `[base_offset, last_offset]`. El `RaftLog` posee ese mapeo y lo persiste en el sidecar `raft-meta`.



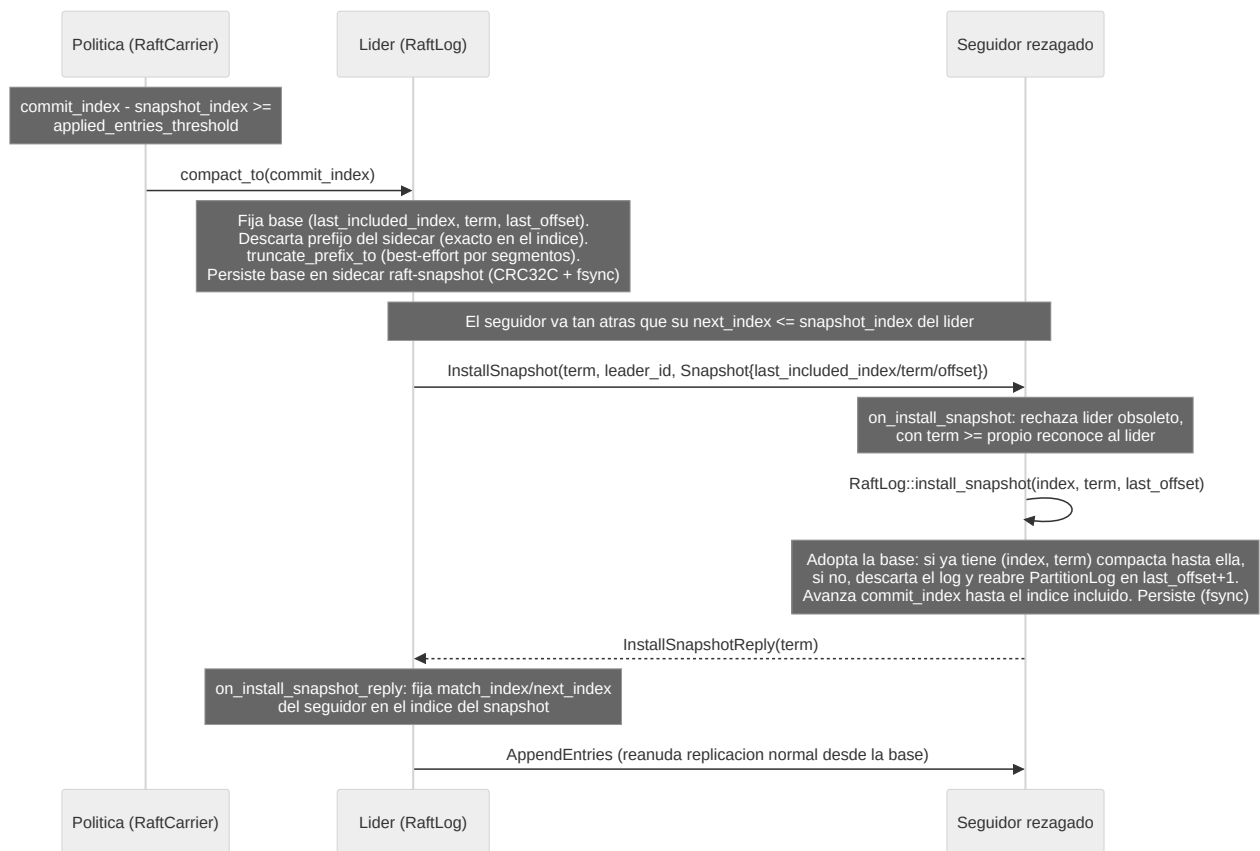
Correspondencia que mantiene el sidecar `raft-meta` (`term:i64 | base_offset:i64 | last_offset:i64 | type:u8, 25 B/entrada`):

Índice Raft	term	base_offset	last_offset	Records del batch
1	t1	0	1	2
2	t1	2	2	1
3	t2	3	5	3

El algoritmo de Raft opera sobre índices contiguos (`prev_log_index`, `commit_index`, `truncate_from`); la capa de partición traduce al espacio de offsets. La **high-watermark** visible a consumidores = `last_offset` de la entrada en `commit_index` (`offsets_at(commit_index)`). La resolución de conflictos `truncate_from(index)` recorta el `PartitionLog` por la **frontera de batch** `base_offset(index)`, dejando los `RecordBatch` intactos en disco.

Diagrama 14: Compactación por snapshot e `InstallSnapshot`

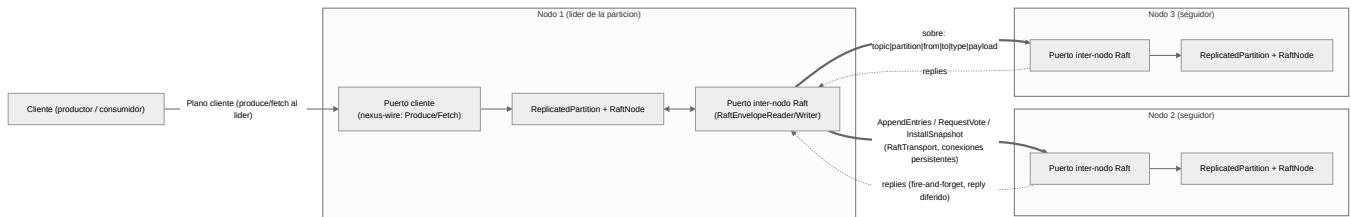
El log replicado es el WAL y crece sin techo; la compactación por *snapshot* (ADR-0024) fija una **base** (`last_included_index`, `last_included_term`, `last_included_offset`), descarta el prefijo aplicado y permite poner al día a un seguidor demasiado rezagado con `InstallSnapshot` en vez de re-replicar desde el índice 1 (§7 del paper). En NexusMQ el "estado" del snapshot **es la propia base** (índice/término/offset): los datos viven en el `PartitionLog`.



Coherente con ADR-0015 (FSM sin E/S): instalar un snapshot es una transición síncrona que reposiciona la base del log; la E/S durable la hace el `RaftLog` /portador, no el `RaftNode`. La compactación es *best-effort*: un fallo de E/S deja el log intacto y se reintenta en el próximo `tick`.

Diagrama 15: Planos de red (cliente vs inter-nodo Raft)

NexusMQ separa el **plano de cliente** (produce/fetch contra el líder, sobre el framing de `nexus-wire`) del **plano inter-nodo de Raft** (`AppendEntries` / `RequestVote` / `InstallSnapshot`), que viaja por su **propio puerto y conexiones persistentes** (ADR-0025). El plano inter-nodo usa el sobre `RaftEnvelope` (`topic | partition | from | to | type:u8 | payload`) con prefijo de longitud sobre TCP, sin `FrameHeader` / `ApiKey` , para mantener ambos protocolos ortogonales.

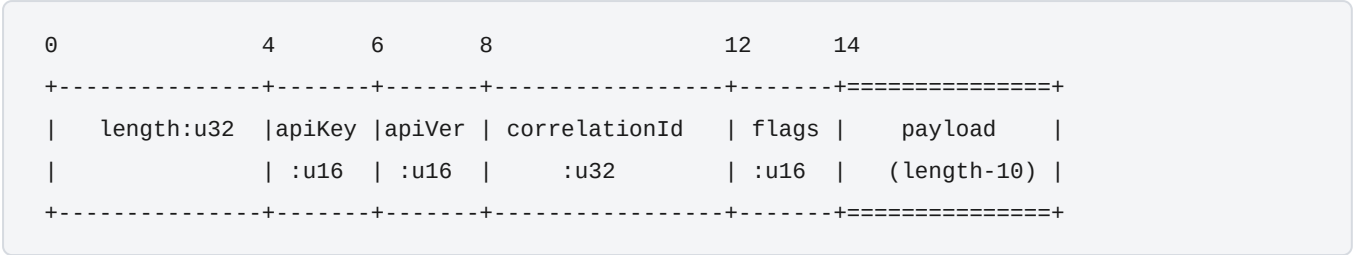


El `RaftTransport` (sumidero `RaftMessageSink`) mantiene una conexión TCP persistente por peer, resolviendo `NodeId` -> `PeerAddress` (host + **puerto inter-nodo**) vía `PeerDirectory`. Es *best-effort*: Raft reenvía en el próximo `tick`. A diferencia del `call_on` cross-core (local y síncrono), un RPC a un nodo remoto **no vuelve** al completarse: su respuesta llega después como **otro** sobre (notificación asíncrona, ADR-0025). El `RaftCarrier` vive en el reactor dueño de la partición (`core = partition % cores`), respetando *shared-nothing*.

Diagrama 16: Layout de la cabecera del frame nativo

El protocolo binario propio de NexusMQ (plano de datos, puerto 9092) es **longitud-prefijo**: toda trama empieza con una cabecera de **14 bytes** (`FrameHeader::kEncodedSize`) seguida del payload. Los enteros son **little-endian** y de ancho fijo. El lector lee 4 bytes (`length`) y luego exactamente `length` bytes más (resto de cabecera + payload). Contrato: [../protocol.md](#) ; código: [../../src/protocol/frame.hpp](#) .

Byte-layout (little-endian)



Campo	Offset	Tamaño	Tipo	Descripción
length	0	4	u32	Bytes tras este campo (resto de cabecera + payload) = 10 + payload .
apiKey	4	2	u16	Operación (ver tabla de ApiKeys).
apiVersion	6	2	u16	Versión del esquema de esa operación (negociada vía ApiVersions).
correlationId	8	4	u32	Eco petición ↔ respuesta; lo elige el cliente.
flags	12	2	u16	Bits de control (ver abajo).
payload	14	var.	bytes	Cuerpo específico de la operación (length - 10 bytes).

La cabecera codificada ocupa 14 bytes; `length` (= `length_for(payload)`) cuenta solo los 10 bytes de cabecera posteriores al propio campo más el payload. El lector valida una cota inferior (debe caber el resto de cabecera) y una **cota superior** anti-DoS (`max_frame` , por defecto **16 MiB**).

Campo flags

Bit	Nombre	Significado
0x0001	credit update	La trama acarrea una actualización de créditos (control de flujo / <i>backpressure</i>).

ApiKeys

Valor	ApiKey	Descripción
0	ApiVersions	Negociación de versiones soportadas.
1	Metadata	Brokers del clúster y topics/particiones.
2	Produce	Publica RecordBatch en una partición.
3	Fetch	Lee registros desde un offset.
4	OffsetCommit	Confirma el offset consumido de un grupo.
5	OffsetFetch	Recupera el offset confirmado de un grupo.
6	JoinGroup	Une un miembro a un grupo de consumidores.
7	SyncGroup	Distribuye la asignación de particiones del grupo.
8	Heartbeat	Mantiene viva la pertenencia al grupo.
9	LeaveGroup	Abandona el grupo.
10	CreateTopic	Crea un topic.
11	DeleteTopic	Borra un topic.

Diagrama 17: Petición/respuesta del subconjunto Kafka

El *listener* compatible con Apache Kafka (`nexusd --kafka-port <N>`) habla el *wire format* de Kafka: **big-endian**, *framing* `Size:INT32` y cabeceras **clásicas o flexibles** según API y versión. El `RecordBatch v2` de Kafka viaja como **blob opaco** (el broker no lo reinterpreta). Contrato: `../kafka.md` ; código: `../src/server/kafka_connection.hpp` , `../src/kafka/codec.hpp` .

Framing (big-endian)

Cada mensaje —petición y respuesta— va longitud-prefijo con un `Size:INT32` big-endian, seguido de la cabecera correspondiente y el cuerpo de la API.

Campo	Tamaño	Tipo	Descripción
Size	4	INT32	Bytes del resto del mensaje (cabecera + cuerpo), big-endian.
cabecera	var.	—	Cabecera de request/response (clásica o flexible, ver abajo).
cuerpo	var.	—	Cuerpo de la API negociada (<code>Produce</code> / <code>Fetch</code> / <code>Metadata</code> / <code>ListOffsets</code> / <code>ApiVersions</code>).

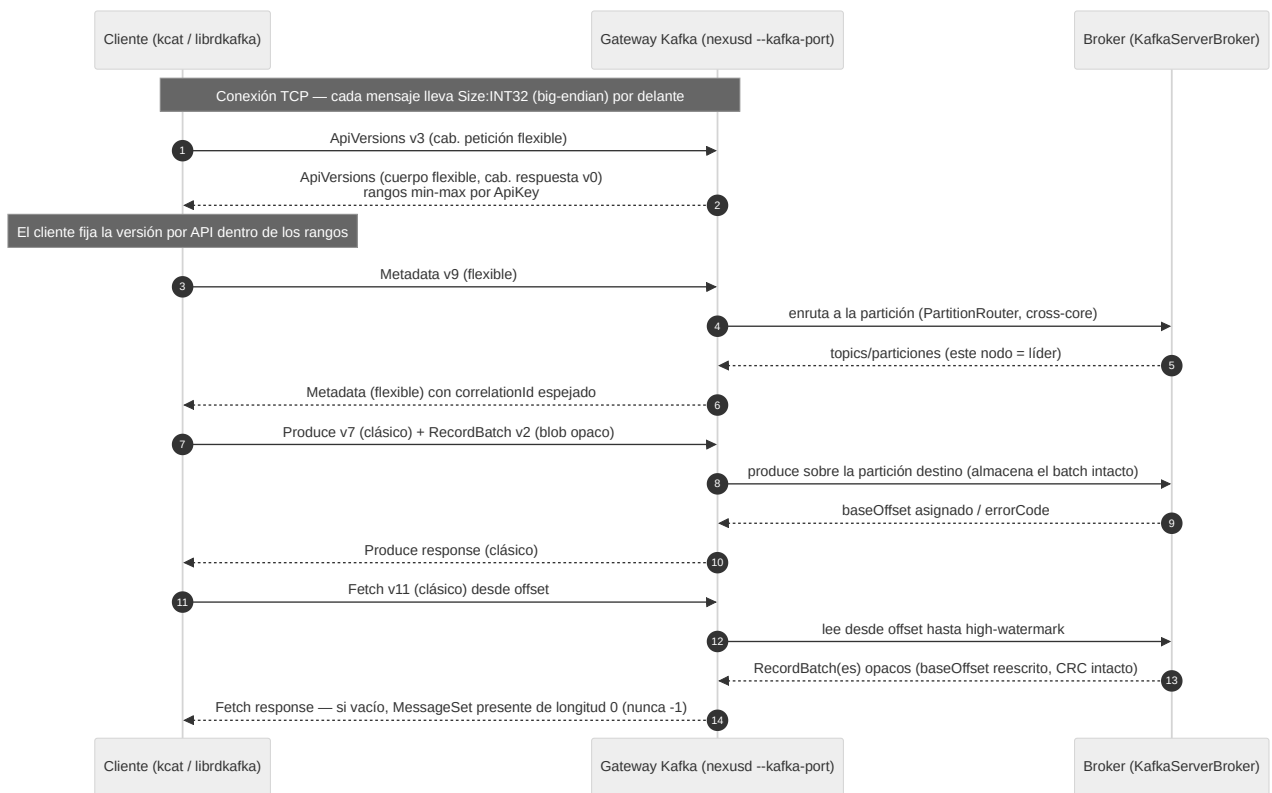
Clásico vs flexible (*tagged fields*)

El codec elige el formato **por API y versión** según el umbral `flexible_since` : una versión `>=` su umbral es **flexible** (arrays/strings *compact* con longitud en `UNSIGNED_VARINT` y *tagged fields*); por debajo es **clásica**.

API	flexible_since	Cabecera petición	Cabecera respuesta
Produce	9	v1 clásica / v2 flexible	v0 clásica / v1 flexible
Fetch	12	v1 clásica / v2 flexible	v0 clásica / v1 flexible
ListOffsets	6	v1 clásica / v2 flexible	v0 clásica / v1 flexible
Metadata	9	v1 clásica / v2 flexible	v0 clásica / v1 flexible
ApiVersions	3	v1 clásica / v2 flexible	siempre v0 (regla de Kafka)

Por eso las versiones que negocia `librdkafka 2.x` (`Produce v7`, `Fetch v11`) van en formato **clásico**, mientras que `Metadata v9`, `ListOffsets v7` y `ApiVersions v3` van en **flexible**. **Excepción:** `ApiVersions` responde **siempre** con cabecera v0 aunque su cuerpo sea flexible.

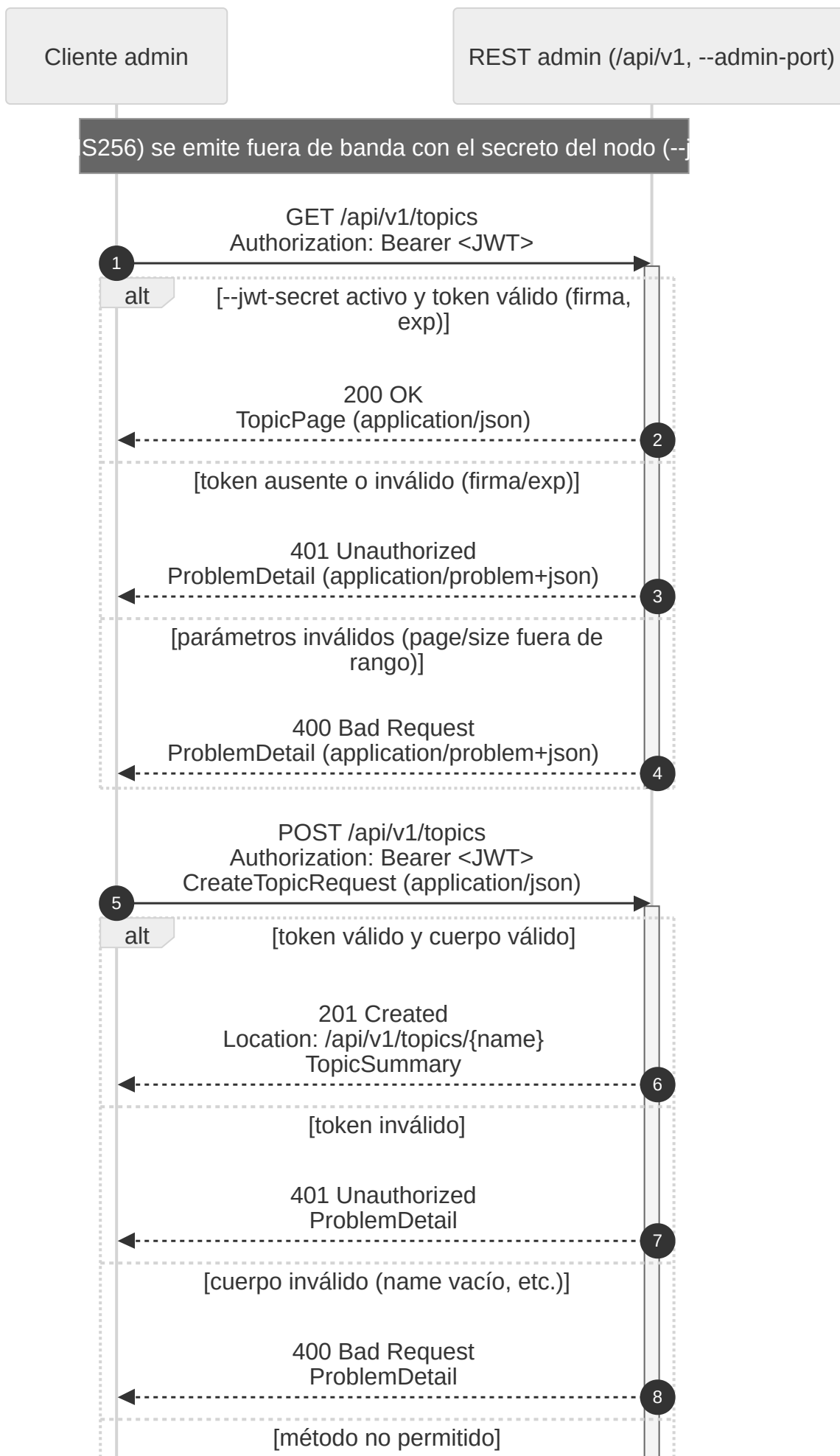
Intercambio cliente (`kcat /librdkafka`) ↔ gateway

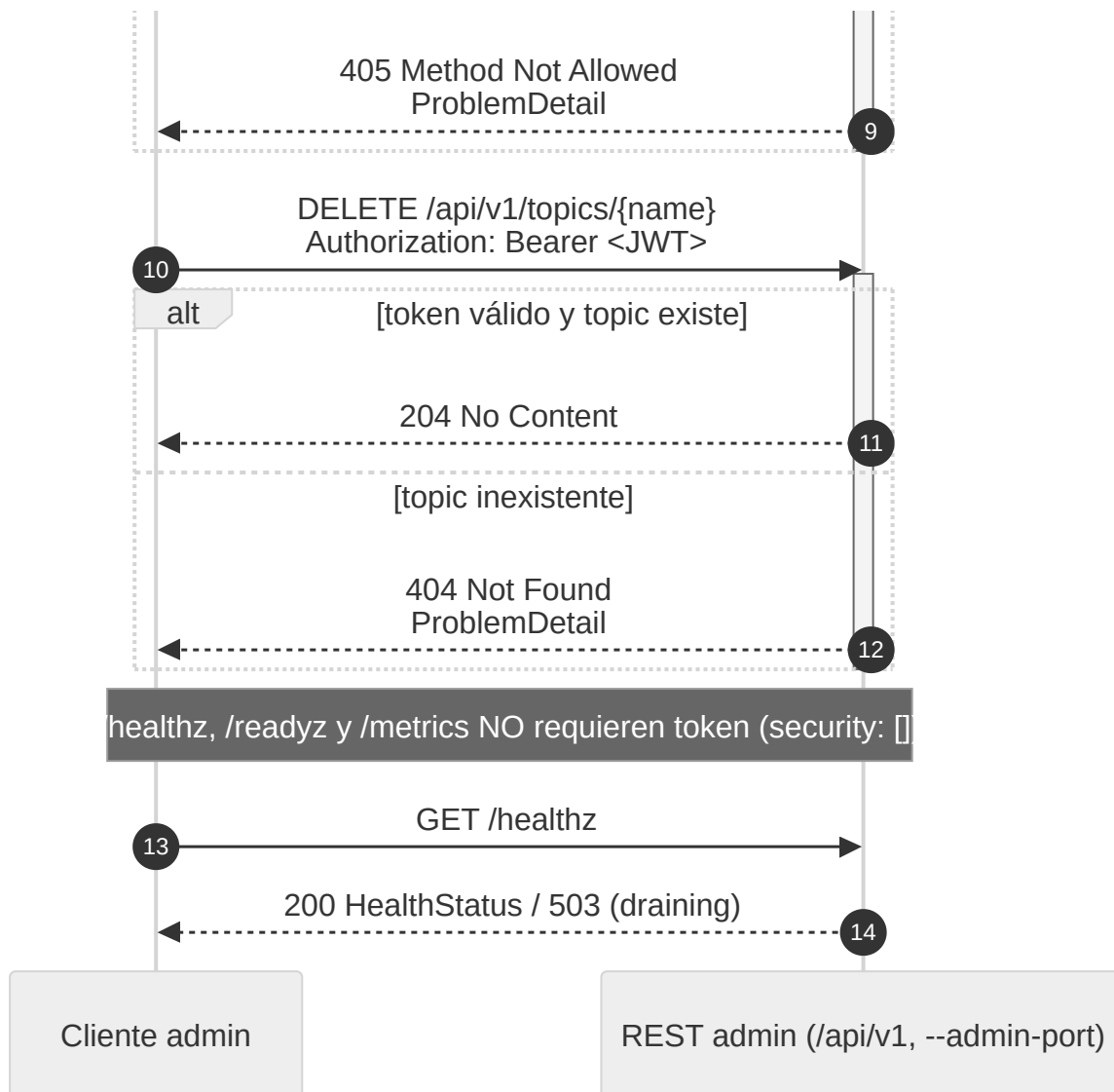


El `correlationId` espeja petición ↔ respuesta (igual que Kafka). En `Fetch` el blob se reconstruye reescribiendo solo el `baseoffset` : barato, porque el CRC32C de Kafka v2 cubre desde `attributes` en adelante, así que no hay que recalcarlo. Un `Fetch` sin datos devuelve un `MessageSet` **presente de longitud 0**, nunca `null` (-1), que librdkafka rechazaría como corrupto.

Diagrama 18: Flujo REST admin con JWT

El plano de administración de NexusMQ es una API REST bajo `/api/v1` servida en el **puerto de operación** (`nexusd --admin-port <N>` , sin valor por defecto), separada del plano de datos binario. Las rutas `/api/v1/*` se autentican con **Bearer JWT** (HS256, firmado con el secreto del nodo) **solo si** el nodo arrancó con `--jwt-secret` ; en ese caso devuelven `401` sin un token válido. Los errores siguen **RFC 7807** (`application/problem+json`). Los *endpoints* de salud/métricas (`/healthz` , `/readyz` , `/metrics`) van **sin autenticar**. Contrato: [../openapi.yaml](#) .





Notas del contrato

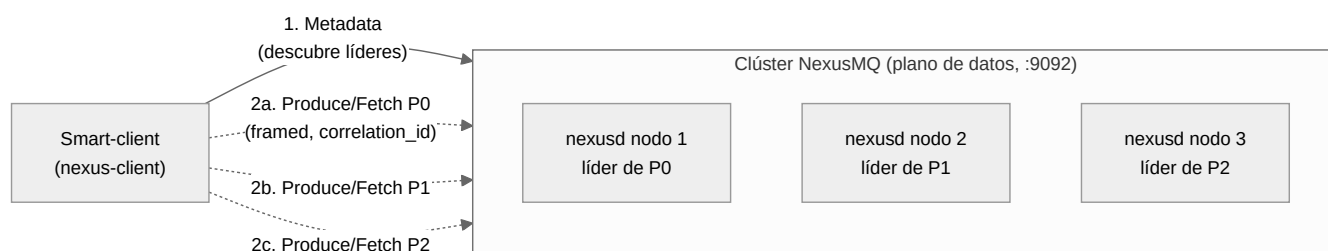
- **Validación del token en el borde:** la API comprueba la **firma** (HS256 con el secreto del nodo) y la **expiración** antes de servir cualquier ruta `/api/v1/*`. Si `--jwt-secret` no está activo, no se exige token.
- **Política de errores central (RFC 7807):** toda respuesta de error usa `ProblemDetail` (`application/problem+json`) con al menos `title` y `status`. El estándar contempla además `403` (no autorizado) para autorización por recurso, aunque el contrato actual detalla `400 / 401 / 404 / 405`.
- **Códigos por operación** (según `openapi.yaml`): `GET /api/v1/topics` → `200 / 400 / 401`; `POST /api/v1/topics` → `201 / 400 / 401 / 405`; `GET /api/v1/topics/{name}` → `200 / 401 / 404`; `DELETE /api/v1/topics/{name}` → `204 / 401 / 404 / 405`; `GET /api/v1/groups` → `200 / 400 / 401 / 405`.
- **Paginación:** las colecciones aceptan `page` (≥ 1 , def. 1) y `size` (1–100, def. 20).

Diagrama 19: Ingress en dos modos — nativo directo vs proxy

El *ingress* de NexusMQ soporta **dos modos** con una jerarquía explícita (ADR-0006): el **nativo directo** (primario) y el **proxy** (secundario, *opt-in*, cableado en el plano de datos por ADR-0027). El nativo prioriza el rendimiento; el proxy, la conveniencia para clientes "tontos" y HTTP, asumiendo a sabiendas un salto extra y la ruptura del *zero-copy*. Fuentes: `../adr/adr-0006-ingress-dos-modos.md`, `../adr/adr-0027-modo-proxy-upstream-pool.md`, `src/ingress/proxy.hpp`, `src/ingress/load_balancer.hpp`, `src/ingress/upstream_pool.hpp`.

Modo nativo directo (primario)

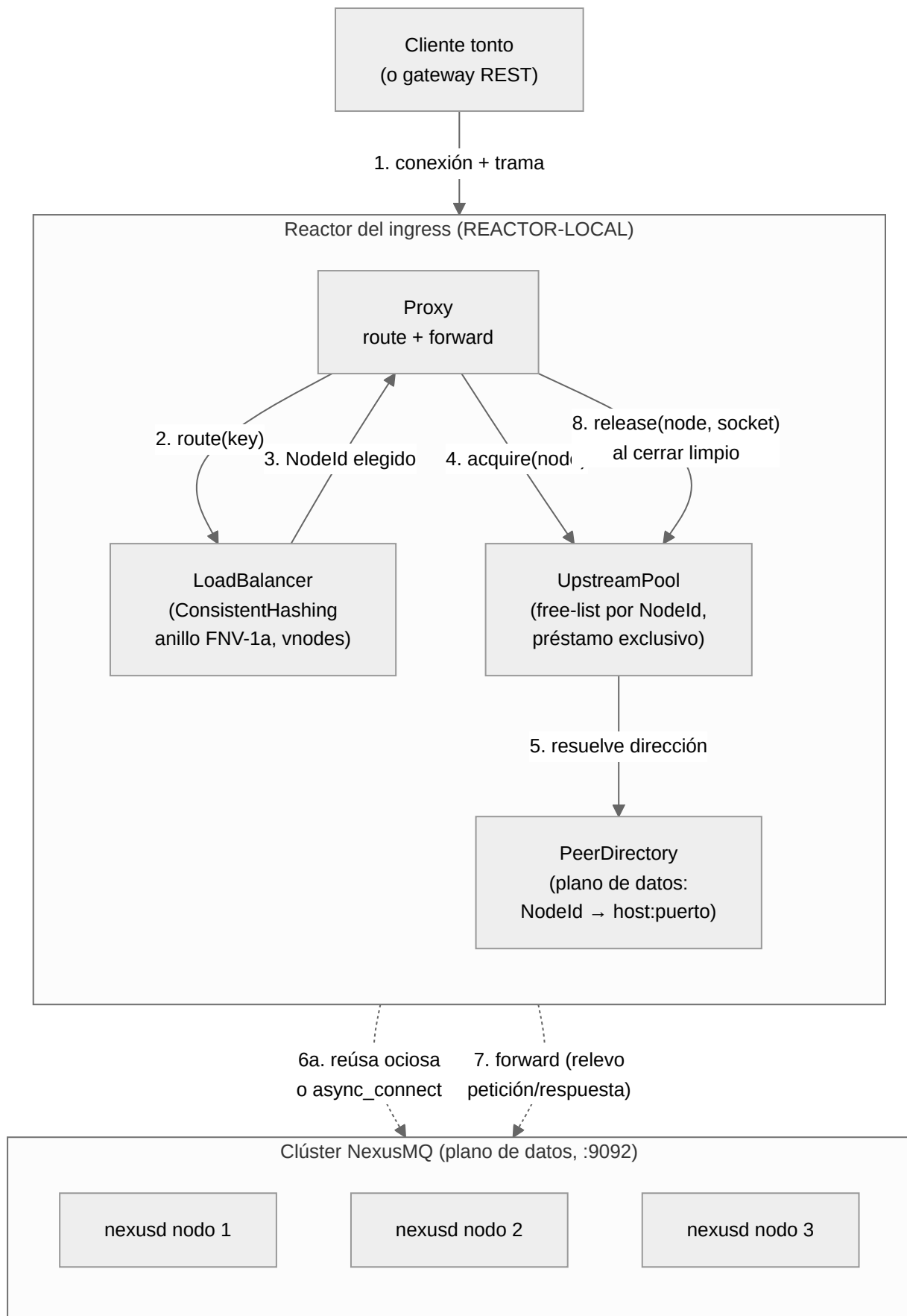
El *smart-client* conoce la topología: pide *metadata*, descubre el **líder** de cada partición y le habla **directamente** por el plano de datos. Sin proxy, sin salto extra, sin romper el *zero-copy*.



- El cliente **gestiona la metadata** y el descubrimiento de líder (coste asumido en el cliente).
- Camino caliente sin intermediarios: máxima latencia/throughput por defecto.

Modo proxy (secundario, opt-in)

El cliente "tonto" no conoce la topología: se conecta al *ingress*, que **enruta** su tramo por *consistent-hashing* (`LoadBalancer`, anillo FNV-1a con *vnodes*) y **releva** sus tramas (petición/respuesta a nivel de trama, `Proxy::forward`) a una conexión obtenida del `UpstreamPool`. Cada reactor tiene su propio *pool* (`REACTOR-LOCAL`, sin locks, *shared-nothing*).



- **route** : `Proxy::route(key)` delega en `LoadBalancer::pick` con *consistent-hashing* (mínima perturbación al añadir/quitar nodos); devuelve `null` o `opt` si el anillo está vacío.

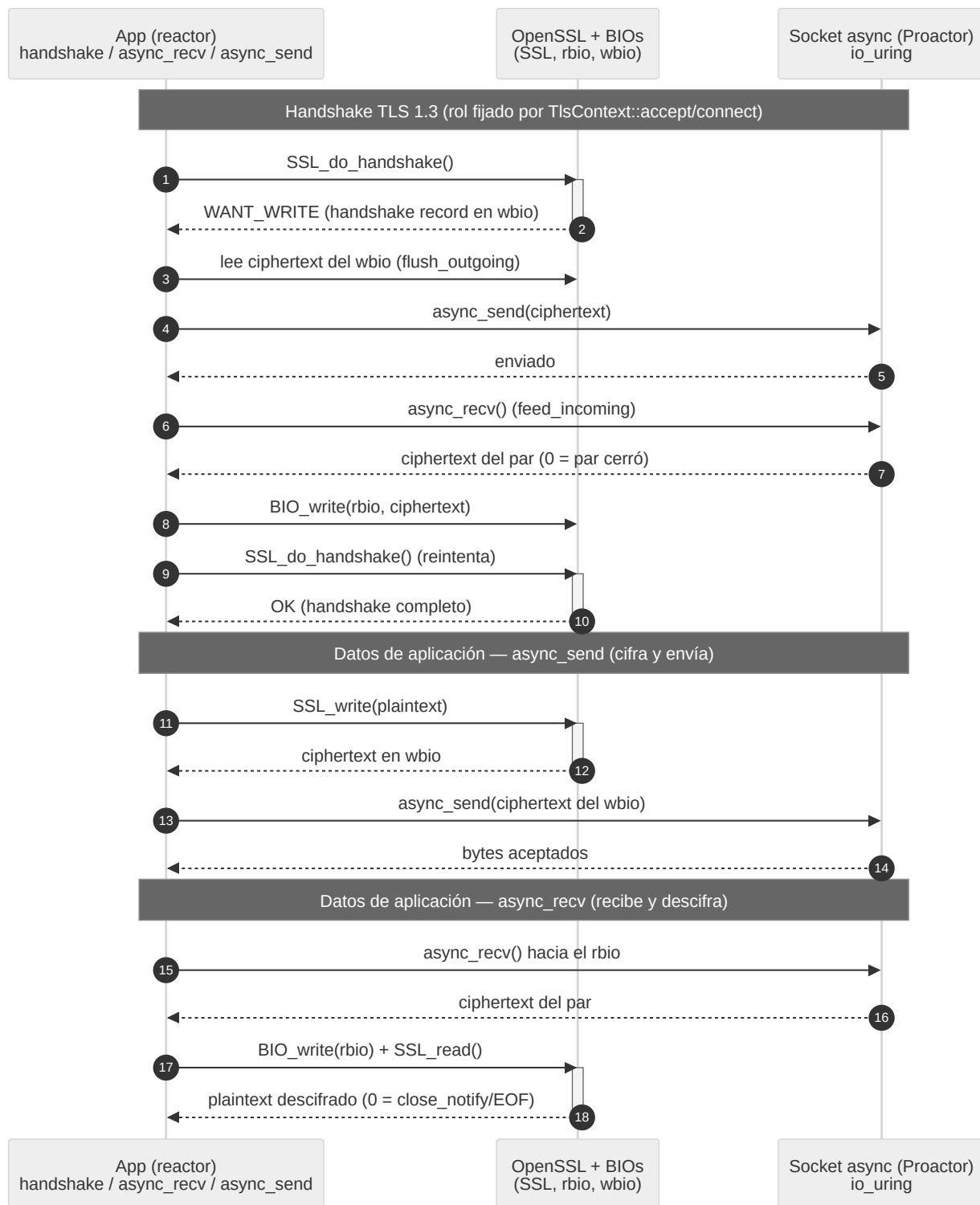
- **acquire / release** : el `UpstreamPool` entrega una conexión **en préstamo exclusivo** (reúsa una ociosa de la *free-list* del nodo o **diala** una nueva con `Socket::async_connect` , sin congelar el reactor) y la recupera al cerrar el cliente; la *free-list* está **acotada** (`max_idle_per_node` , def. 8) para no fugar descriptores.
- **Préstamo exclusivo**: garantiza que **no se intercalan** tramas de dos clientes en la misma conexión aguas arriba (corrección del relevo).
- **Dos PeerDirectory distintos**: el del plano de datos (este) es una **instancia separada** del de Raft (ADR-0025); mismo tipo, distinta instancia.

Trade-offs (por qué dos modos, no uno)

Aspecto	Nativo directo	Proxy
Camino	cliente → líder	cliente → ingress → nodo
Saltos extra	0	1 (asumido)
<i>Zero-copy</i>	sí	se rompe (copia en el relevo)
Cliente	<i>smart</i> (gestiona metadata)	<i>tonto</i> / HTTP
Activación	por defecto	<i>opt-in</i>

Diagrama 20: Puente TLS — OpenSSL síncrono sobre el proactor asíncrono

NexusMQ termina **TLS 1.3** (y, intra-clúster, **mTLS**) en el *ingress* (§7.9). OpenSSL es la librería elegida, pero su E/S es **síncrona/bloqueante**, mientras que el broker es *thread-per-core* **asíncrono** sobre un `Proactor` (`io_uring`). La solución (ADR-0019) es un **puente de BIOs de memoria**: OpenSSL nunca toca el descriptor; cifra/descifra contra dos `BIO` en memoria (`rbio` / `wbio`) y el transporte se hace de forma asíncrona vaciando/alimentando esos BIOs por el `Proactor` . Así la criptografía corre en línea, pero el socket es no bloqueante. Fuentes: `../adr/adr-0019-tls-opcional-openssl-bios.md` , `src/ingress/tls.hpp` (`TlsContext` / `TlsConnection`).



Cómo funciona el bucle (qué hace cada pieza)

- **TlsContext** (THREAD-SAFE, RAII sobre `SSL_CTX`): fábricas `server` / `client` cargan el material PEM (cadena de certificado, clave privada) y, si se da una CA, configuran **mTLS** (`SSL_VERIFY_PEER` | `FAIL_IF_NO_PEER_CERT`). `accept` / `connect` abren una `TlsConnection` fijando el rol del *handshake*.
- **TlsConnection** (REACTOR-LOCAL, *solo movable*): posee el `SSL*` y el `Socket`. Las primitivas OpenSSL (`SSL_do_handshake` / `SSL_read` / `SSL_write`) operan **solo contra los BIOs de memoria**:
 - ante `WANT_WRITE`, `flush_outgoing` **vacía** el `wbio` al socket (`async_send`);

- ante `WANT_READ`, `feed_incoming` **alimenta** el `rbio` con lo leído del socket (`async_recv`); un retorno de `0` byte significa que el par cerró (EOF / `close_notify`).
- `peer_principal()` : extrae el **CN** del certificado del par para *authz* (mTLS).

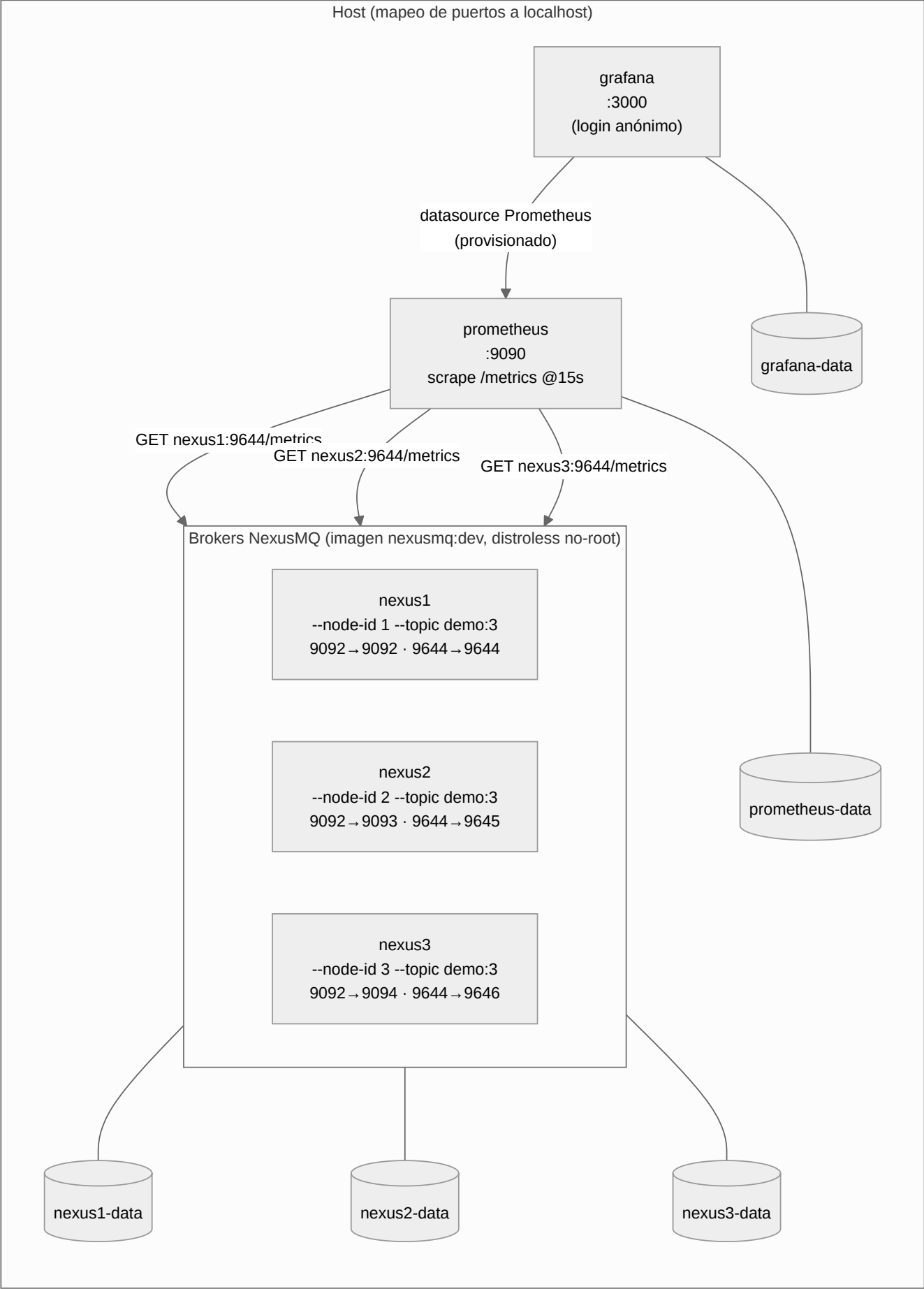
Acoplamiento opcional (coste cero, ADR-0008)

El build usa `find_package(OpenSSL)` . Si está presente, compila `ingress/tls.{hpp,cpp}` y define `NEXUS_HAVE_OPENSSL` (toda la cabecera vive bajo `#ifdef NEXUS_HAVE_OPENSSL`); si no, el plano TLS se **omite por completo** y el broker arranca **en claro**. El CI instala `libssl-dev` para ejercer el *path* TLS (compilación, tests, sanitizers y clang-tidy).

Coste asumido: el puente de BIOs añade una copia intermedia *ciphertext* ↔ *socket*; es aceptable porque el throughput TLS lo domina la cifra, no esa copia.

Diagrama 21: Despliegue local con Docker Compose

El `docker compose` de `deploy/` levanta **3 nodos** `nexusd` + **Prometheus** + **Grafana** para validar el empaquetado, la observabilidad y los *probes* de salud sobre varias instancias. En Fase 3 los tres nodos son **brokers independientes** (el runtime de clúster multi-nodo —descubrimiento de líder, replicación entre nodos— llega en fases posteriores). Cada `nexusd` expone dos puertos: el **plano de datos** (`9092` , protocolo binario *framed* con `correlation_id`) y el **puerto de operación** (`9644` : `/healthz` , `/readyz` , `/metrics` , `/api/v1/...`). Fuente: `../../../../deploy/docker-compose.yml` , `../../../../deploy/prometheus.yml` .



Detalles del despliegue (fieles al compose)

- **Imagen** `nexusmq:dev` : build multi-stage (Ubuntu para compilar → **distroless** `cc` , no-root) desde la raíz del repo (`deploy/Dockerfile`). El `HEALTHCHECK` reutiliza `nexus-cli diagnostics` (distroless no trae `shell` ni `curl`): el contenedor pasa a *healthy* solo cuando `/healthz` y `/readyz` responden OK.
`restart: unless-stopped` en todos los servicios.
- **Mapeo de puertos en** `localhost` : `nexus1 9092/9644` , `nexus2 9093/9645` , `nexus3 9094/9646` (el puerto interno es `9092/9644` en los tres; cambia el lado del host).
- **Persistencia**: un volumen de datos por nodo (`nexusN-data` → `/var/lib/nexusmq`), más `prometheus-data` y `grafana-data` .
- **Prometheus** (`:9090`): raspa `/metrics` (`metrics_path: /metrics`) de los tres por su **nombre de servicio** en compose (`nexus1:9644` , `nexus2:9644` , `nexus3:9644`), `scrape_interval: 15s` , etiqueta `cluster: local` ; `depends_on` de los tres nodos.
- **Grafana** (`:3000`): login anónimo (rol `Admin`), *datasource* Prometheus **provisionado** desde `grafana/datasources.yml` ; `depends_on` de Prometheus.

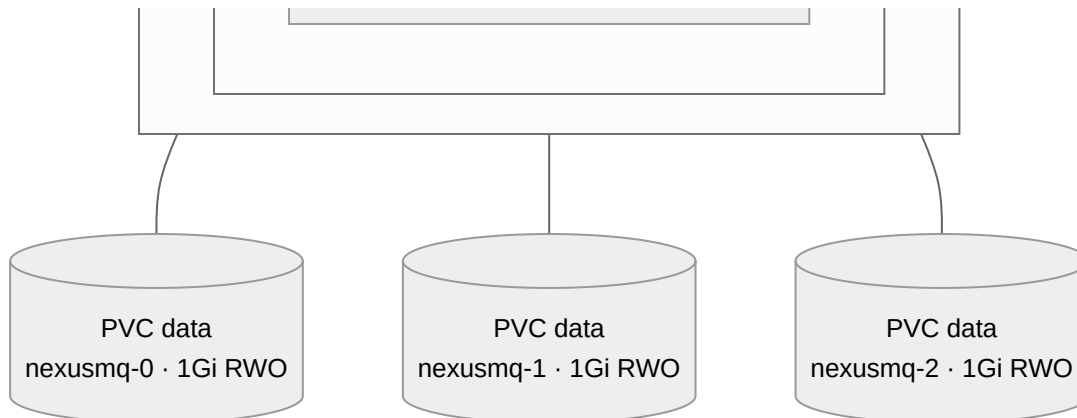
Requisito del host: el kernel debe soportar **io_uring** (el broker usa el backend `io_uring` directo, ADR-0012).

Ver el flujo de scrape end-to-end en [23-pipeline-observabilidad.md](#) y el despliegue a escala en [22-despliegue-kubernetes.md](#) .

Diagrama 22: Despliegue en Kubernetes — StatefulSet + Service headless

El broker tiene **estado persistente** (logs de partición), de ahí el **StatefulSet** (3 réplicas) con **volumeClaimTemplates** (un **PVC por réplica**) y un **Service headless** (`clusterIP: None`) que da DNS estable a cada réplica (`nexusmq-0.nexusmq`, `nexusmq-1.nexusmq`, ...). Los *probes* van sobre el **puerto de operación** (`ops`, 9644): **liveness** `/healthz`, **readiness** `/readyz`. Fuentes: `../../deploy/k8s/statefulset.yaml`, `../../deploy/k8s/service.yaml`.





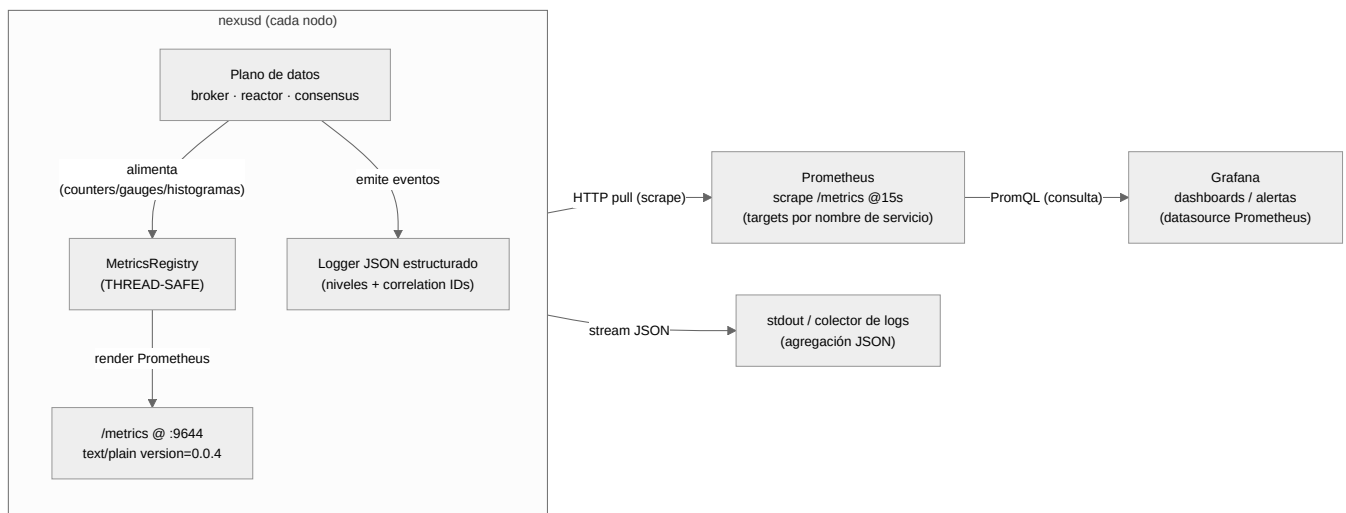
Detalles del manifiesto (fieles a `deploy/k8s/`)

- **StatefulSet** (`replicas: 3`, `serviceName: nexusmq`): identidad y almacenamiento estables por réplica; `volumeClaimTemplates` crea un **PVC data de 1Gi (ReadWriteOnce)** por pod, montado en `/var/lib/nexusmq`.
- **Service headless** (`clusterIP: None`): no balancea por VIP; expone DNS por pod y agrupa los puertos `data` (9092) y `ops` (9644). El líder de partición se descubre por *metadata* (no por el Service), coherente con el modo nativo directo (ver [19-modo-nativo-vs-proxy.md](#)).
- **Probes sobre el puerto ops**:
 - **liveness** `httpGet /healthz (initialDelay 5s, period 10s, failureThreshold 3)`: ¿sigue vivo el proceso? Un fallo justifica reiniciar el pod.
 - **readiness** `httpGet /readyz (initialDelay 3s, period 5s, failureThreshold 3)`: ¿puede recibir tráfico? Pasa solo con el arranque terminado y los *probes* (disco, Raft, *lag*) sanos; un `fail` retira el pod del balanceo sin matarlo.
- **Contenedor endurecido (hardening)**: `runAsNonRoot (uid/gid 65532, fsGroup 65532)`, `readOnlyRootFilesystem: true`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`.
- **Recursos**: `requests cpu 250m / mem 128Mi`; `limits cpu 1 / mem 512Mi`.

La imagen `nexusmq:dev` debe estar disponible para el clúster (build + push a un registry, o `kind load docker-image nexusmq:dev`) y el nodo debe soportar **io_uring**. Aplicar con `kubectl apply -f deploy/k8s/` y seguir con `kubectl rollout status statefulset/nexusmq`.

Diagrama 23: Pipeline de observabilidad — métricas y logs

La observabilidad de NexusMQ es *self-hosted* (Prometheus + Grafana en Docker). El broker **alimenta** una `MetricsRegistry` (THREAD-SAFE, en `nexus-telemetry`) desde las capas del plano de datos (broker/reactor: tasa de produce/fetch, *lag*, `commit_index` /term de Raft, latencias) y la **expone** en `/metrics` por el **puerto de operación** (9644), en **formato de exposición Prometheus** (`text/plain; version=0.0.4`). Prometheus **raspa** ese endpoint periódicamente y Grafana lo consulta como *datasource*. En paralelo, el proceso emite **logs JSON** estructurados (con niveles y *correlation IDs*). Fuentes: [capítulo 12 \(Observabilidad\)](#), `src/telemetry/metrics.hpp`, `../..../deploy/prometheus.yml`, `../..../deploy/docker-compose.yml`.



Cómo fluye (fiel al código y al deploy)

- **Origen de las métricas** (§7.6, §9 ADR-0017): la `MetricsRegistry` vive en `nexus-telemetry` (no en `nexus-server`, un exe no enlazable, ni en `nexus-ingress`, que crearía el ciclo `broker` → `ingress`). Las capas inferiores **registran** métricas; el gateway REST y el servidor las **exponen** en `/metrics`.
- **Modelo pull**: Prometheus **raspa** (no recibe *push*); en el compose, `scrape_interval: 15s`, `metrics_path: /metrics`, *targets* por nombre de servicio (`nexus1:9644`, `nexus2:9644`, `nexus3:9644`) con etiqueta `cluster: local` (ver [21-despliegue-docker-compose.md](#)).
- **Endpoint sin autenticar**: `/metrics`, `/healthz` y `/readyz` van **sin token** (`security: []`), a diferencia de `/api/v1/*` que usa Bearer JWT (ver [18-flujo-rest-jwt.md](#)).
- **Grafana**: consulta Prometheus por **PromQL** y pinta *dashboards* / dispara alertas; *datasource* provisionado desde `grafana/datasources.yml`.
- **Logs JSON**: estructurados, con niveles y *correlation IDs* para *tracing* de peticiones; emitidos a la salida estándar para que el orquestador/colector los agregue.

Señales y SLOs (cómo se usan)

Las métricas materializan las **cuatro señales de oro** (latencia, tráfico, errores, saturación) y el método **USE**, base de los SLOs en percentiles (**p95/p99/p999**, nunca solo la media). Las cifras reales de rendimiento viven en [../benchmarks.md](#) (no se inventan aquí).

Registro de decisiones de arquitectura (ADR)

Cada decisión de arquitectura de NexusMQ vive en un fichero `adr-NNNN-titulo-corto.md`, con las secciones **Contexto** · **Decisión** · **Consecuencias** · **Alternativas consideradas** (plantilla en `../../BibliotecaDocumentacion/patrones/plantilla-adr.md`).

Una decisión **aceptado** **no se edita**: si cambia, se **reemplaza** por un ADR nuevo que la marca como *reemplazada por adr-NNNN*. La historia de decisiones es inmutable. El recorrido narrado de estas decisiones vive en el capítulo [28 de la documentación técnica](#).

ADR	Título	Estado	Fecha
0001	Plataforma de desarrollo y <i>target</i> (Linux primero, Windows después)	aceptado	2026-06-07
0002	Modelo de I/O asíncrona (proactor; <code>io_uring</code> primero, IOCP después)	aceptado	2026-06-07
0003	Modelo de replicación (Raft por partición)	aceptado	2026-06-07
0004	Protocolo del plano de datos (binario propio + gateway REST)	aceptado	2026-06-07
0005	Arquitectura de concurrencia (<i>shared-nothing thread-per-core</i>)	aceptado	2026-06-07
0006	Rol del <i>ingress</i> (dos modos: nativo directo + proxy/REST)	aceptado	2026-06-07
0007	Postura de consistencia (CP / PACELC PC-EC)	aceptado	2026-06-07
0008	Viabilidad de coste cero (desarrollo y test gratuitos)	aceptado	2026-06-07
0009	Política de manejo de errores por capa	aceptado	2026-06-10
0010	Entorno de desarrollo (VS Code sobre WSL)	aceptado	2026-06-11
0011	Estándar C++23 + libcpp en Clang (para <code>std::expected</code>)	aceptado	2026-06-11
0012	Backend <code>io_uring</code> directo sobre el uapi del kernel (sin liburing)	aceptado	2026-06-12
0013	Capa <code>nexus-wire</code> para el framing sobre conexión	aceptado	2026-06-13
0014	Modelo del log de Raft (índice = ordinal de entrada; término en sidecar)	aceptado	2026-06-14
0015	RaftNode como máquina de estados síncrona sin E/S	aceptado	2026-06-14
0016	<code>ReplicatedPartition</code> como tipo paralelo a <code>Partition</code>	aceptado	2026-06-15
0017	Target <code>nexus-telemetry</code> para observabilidad	aceptado	2026-06-17
0018	REST admin por puerto/adaptador (<code>AdminService</code> / <code>AdminApi</code>)	aceptado	2026-06-18
0019	TLS opcional vía OpenSSL con puente de BIOs de memoria	aceptado	2026-06-19
0020	<i>Binding</i> de Python vía ABI C estable (<code>nexus-ffi</code> + <code>ctypes</code>)	aceptado	2026-06-20

ADR	Título	Estado	Fecha
0021	Backend IOCP (Windows) — diseño fijado, implementación diferida	reemplazado por adr-0022	2026-06-20
0022	Backend IOCP (Windows) — implementado, compile-verificado con MinGW	reemplazado por adr-0023	2026-06-20
0023	Backend IOCP (Windows) — verificado en runtime con MSVC	aceptado	2026-06-20
0024	Compactación del log de Raft por snapshot	aceptado	2026-06-20
0025	Activación en producción del stack Raft + multi-reactor (transporte inter-nodo)	aceptado	2026-06-21
0026	Sharding por núcleo del plano de datos (partición → núcleo, grupos por hash)	aceptado	2026-06-21
0027	Cableado del modo proxy — <i>pool</i> de conexiones aguas arriba por reactor	aceptado	2026-06-25
0028	Port completo de <code>nexusrd</code> a Windows (backend, afinidad y señales)	aceptado	2026-06-27
0029	Adaptador Kafka asíncrono cross-core sobre el broker vivo	aceptado	2026-06-28

ADR-0001: Plataforma de desarrollo y *target* (Linux primero, Windows después)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

El autor desarrolla en Windows con Visual Studio, pero el dominio del proyecto —un broker de mensajería— se despliega de forma natural en Linux y contenedores. Además, el ecosistema de *tooling* de alto rendimiento (perf, flamegraphs, io_uring, hugepages, NUMA) es Linux. Existe, por tanto, una tensión entre el entorno de trabajo preferido (Visual Studio) y la plataforma natural del dominio (Linux), y se desea no renunciar a ninguno de los dos.

Decisión

Se adopta **Linux** (x86-64, Docker) como *target* primario, desarrollado **desde Visual Studio vía WSL2** (workload "Linux development with C++"), con **CMake + vcpkg** como sistema de build y de dependencias portables. El *target* **Windows nativo** queda como un **objetivo posterior comprometido** —no descartado—, que se aborda tras consolidar la plataforma Linux.

Consecuencias

- (+) Encaje con el dominio y con el *tooling* esperado por los perfiles de sistemas/HFT, conservando a la vez el IDE preferido.
- (+) La portabilidad se diseña desde el inicio sin pagar por adelantado su coste completo.
- (+) La estructura de **solución única** (un único árbol CMake) **no bloquea** el *target* Windows posterior: el mismo árbol cambia de *preset* a MSVC y solo añade el adaptador **IOCP** en `src/io/`, sin reestructurar (ver §5.2).
- (–) Se asume la fricción de WSL2 y la de mantener un *toolchain* multiplataforma.

Alternativas consideradas

- **Windows nativo (IOCP/RIO) como primario:** descartado como primario por encaje de dominio y de *tooling*, aunque sería un diferenciador; se mantiene como fase posterior.
- **Cross-platform total desde el día 1:** descartado por el coste de mantener dos backends de I/O y de fichero antes de tener producto.

ADR-0002: Modelo de I/O asíncrona (proactor; io_uring primero, IOCP después)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

El plano de datos del broker es I/O intensiva, por lo que el modelo de E/S asíncrona es una decisión central. Los dos *targets* previstos (Linux y Windows) ofrecen modelos asíncronos: io_uring e IOCP, **ambos basados en completions**. En cambio, epoll es un modelo *readiness-based*, conceptualmente distinto.

Decisión

Modelar la capa de I/O como **proactor**: la interfaz consiste en enviar una operación asíncrona y recibir una *completion*. El **backend será io_uring ahora e IOCP después**. Las **coroutines de C++20** se asientan sobre este proactor.

El storage engine arranca con I/O **bloqueante** (Fase 1) y adopta io_uring como **optimización medida**. Bajo el reactor *thread-per-core*, **todo** el I/O —incluido `fsync`— pasa por io_uring, porque nada bloqueante puede vivir en el reactor (matiz R6). El *direct I/O* con caché propia queda como profundidad opcional de la Fase 4.

Consecuencias

- (+) Una sola forma de I/O para ambos *targets*; coroutines naturales; un anillo por núcleo encaja con la arquitectura *shared-nothing*.
- (+) La portabilidad queda como capacidad documentada, sin necesidad de doble implementación inicial.
- (–) Un proactor es más complejo que un *reactor* epoll directo; si hiciera falta epoll, se emularía como *completion*.
- (–) Prohibir bloqueos en el reactor exige una disciplina asíncrona total (allocators por núcleo, `fsync` asíncrono).

Alternativas consideradas

- **Reactor epoll (*readiness*):** descartado como forma canónica por no mapear bien a IOCP (Windows).
- **Librería asíncrona externa (ASIO/Seastar):** descartada para el núcleo por ser una pieza de aprendizaje central; Seastar se valora solo como referencia conceptual.

ADR-0003: Modelo de replicación (Raft por partición)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Provisional revisable: se reconfirma con los *benchmarks* de la Fase 1; si cambia, se **reemplaza** por un ADR nuevo (no se edita).

Contexto

La Fase 2 requiere replicar particiones y tolerar caídas de líder. Existen dos arquitecturas de referencia:

- **Raft por partición** (modelo Redpanda): cada partición es su propio grupo Raft, y el log de Raft es el log de la partición.
- **ISR estilo Kafka:** el consenso se usa solo para metadatos/controller, y los datos se replican por primario-backup con *high-watermark*.

Decisión

Adoptar **Raft por partición**: un **único mecanismo** de ordenación, replicación y elección por partición. El *high-watermark* se deriva del `commitIndex` de Raft, y el modelo de `acks` (§4.3) se asienta sobre su *commit*.

Se prefiere por ser **conceptualmente uniforme, formalmente especificado y testeable de forma determinista** —lo que sirve al objetivo de *correctitud demostrable*— y porque compone con la arquitectura *shared-nothing* (un grupo Raft anclado a un núcleo).

Consecuencias

- (+) Un solo mecanismo que razonar y probar; *failover* sin pérdida de datos *committed*; encaje con *thread-per-core*.
- (+) Base natural para el modelo de ACK y la postura CP (ver ADR-0007).
- (–) Coste de **quórum en cada escritura** (latencia de mayoría) en el camino caliente.
- (–) Muchas particiones implican muchos grupos Raft y, por tanto, mucho *heartbeat*; no es problemático a escala de portfolio, pero sí relevante a escala de miles de particiones.

Alternativas consideradas

- **ISR estilo Kafka:** ofrece mayor throughput potencial, pero introduce más piezas (controller, gestión de ISR, *high-watermark* separado) y más superficie de error; descartado por complejidad y por ser el diseño "de hace 14 años".
- **Raft por partición:** elegido (ver Decisión).

ADR-0004: Protocolo del plano de datos (binario propio + gateway REST)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

El plano de datos necesita un protocolo. Hay dos caminos en tensión:

- Un **binario propio** maximiza el aprendizaje y el control (framing, varint, *multiplexing/correlation IDs*, versionado, control de flujo) y compone sin *impedance mismatch* con la arquitectura *shared-nothing*.
- La **compatibilidad con Kafka** daría ecosistema (kcat, consolas, clientes), pero obliga a implementar una especificación ajena y enorme (más de 70 *API keys* versionadas, *record batch v2*, *consumer group protocol...*), atando el diseño y *front-loading* un esfuerzo ingrato.

Decisión

Implementar un **protocolo binario propio** para las fases nucleares, acompañado de un **gateway REST** para interoperabilidad temprana (cubre el "¿funciona con herramientas reales?" vía HTTP/curl/Postman).

Se difiere un **subconjunto Kafka-compatible** (*ApiVersions* / *Metadata* / *Produce* / *Fetch*) a la **Fase 4** como *stretch*: basta para la demo "kcat habla con mi broker" sin apostar el proyecto a la especificación completa. El titular "Kafka-compatible" queda así **recuperable más tarde** sin pagar su coste por adelantado.

Consecuencias

- (+) Control total del protocolo; aprendizaje de diseño de protocolos (lo que el proyecto quiere demostrar); libertad para ajustarlo a *shared-nothing* y al modelo de créditos.
- (+) Interoperabilidad inmediata vía REST.
- (–) Sin ecosistema Kafka "gratis" hasta la Fase 4.
- (–) El cliente nativo hay que escribirlo (ya estaba en alcance, OE-2).

Alternativas consideradas

- **Compat Kafka desde el inicio:** ofrece ecosistema instantáneo, pero a costa de un esfuerzo enorme e ingrato, riesgo de *uncanny valley* (compatibilidad parcial que casi funciona) y un diseño atado antes de existir; descartado para el núcleo y diferido como subconjunto de la Fase 4.
- **Binario propio + REST:** elegido (ver Decisión).

ADR-0005: Arquitectura de concurrencia (*shared-nothing thread-per-core* con reactor propio)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Provisional revisable tras los *benchmarks* de la Fase 1; si cambia, se **reemplaza** por un ADR nuevo.

Contexto

El plano de datos exige un modelo de concurrencia. Hay dos familias en juego:

- **Thread-pool con estado compartido** (modelo Kafka clásico): *locks* sobre estructuras compartidas.
- **Shared-nothing thread-per-core** (modelo Redpanda/Seastar): un reactor por núcleo, estado por núcleo y paso de mensajes.

El proyecto prioriza la profundidad de sistemas, la *correctitud demostrable* y estar en el estado del arte.

Decisión

Adoptar **shared-nothing thread-per-core: un reactor por núcleo (*pinned*)**, cada uno dueño de su anillo **io_uring**, su **allocator** y su subconjunto de particiones; la comunicación entre reactores se hace **solo por colas SPSC cross-core** (estilo `submit_to`).

Se **construye un reactor mínimo propio** sobre `io_uring` + corutinas C++20 y **no** se usa Seastar (dependencia pesada, Linux-only; "construir" demuestra más que "usar" y encaja con la política §3.4 de hacer el núcleo a mano). Seastar queda como **referencia conceptual**. La decisión es **provisional revisable** tras los *benchmarks* de la Fase 1.

Consecuencias

- (+) Elimina la clase de bug más cara (carreras sobre estado compartido); cachés y NUMA locales; sin *locks* en el camino caliente; estado del arte.
- (+) Compone con Raft por partición (un grupo anclado a un núcleo) y con `io_uring` (un anillo por núcleo).
- (–) **Disciplina asíncrona total:** nada bloqueante en el reactor (un `fsync`, `malloc` o *lock* que entre al kernel congela el núcleo), lo que exige `fsync` asíncrono y *allocators* por núcleo.
- (–) Coordinación *cross-core* explícita (SPSC + *wakeup*).
- (–) Si se adopta *direct I/O*, hay que implementar caché/*readahead* propios (no se usa el *page cache*).

Alternativas consideradas

- **Thread-pool con work-stealing / estado compartido:** más simple de arrancar, pero demuestra el diseño de hace 14 años, introduce contención y la clase de bug que queremos eliminar; descartado como primario.

- **Usar Seastar:** ahorra construir el reactor, pero demuestra menos, es Linux-only y pesado; descartado como dependencia, retenido como referencia.

ADR-0006: Rol del *ingress* (dos modos: nativo directo + proxy/REST)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

En un plano de datos de alto rendimiento, interponer un proxy en el camino caliente añade un salto y rompe el *zero-copy*. Pero exigir siempre un *smart-client* excluye a los clientes simples y a HTTP. Existe, por tanto, una tensión explícita entre rendimiento y conveniencia.

Decisión

Soportar **dos modos** con una jerarquía explícita:

1. **Nativo directo** (primario): un *smart-client* que va al **líder** de la partición vía *metadata*, sin proxy.
2. **Proxy/REST** (secundario, *opt-in*): el *ingress* enruta clientes "tontos" mediante *consistent hashing* y expone un **gateway REST**, asumiendo el salto extra a sabiendas.

Se documenta como **dos modos con *trade-offs***, no como "un *ingress* que lo hace todo".

Consecuencias

- (+) Máximo rendimiento por defecto y conveniencia/interoperabilidad bajo demanda; demuestra **criterio** en vez de incoherencia.
- (–) Dos caminos que mantener y documentar; el cliente nativo debe gestionar *metadata* y el descubrimiento de líder.

Alternativas consideradas

- **Solo proxy:** simple para el cliente, pero penaliza el camino caliente; descartado como único.
- **Solo smart-client:** óptimo en rendimiento, pero excluye a clientes simples y a HTTP; descartado como único.

ADR-0007: Postura de consistencia (CP / PACELC PC-EC)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

Todo sistema replicado debe elegir, ante una partición de red, entre **consistencia** y **disponibilidad** (CAP); y, en operación normal, entre **latencia** y **consistencia** (PACELC). El proyecto prioriza la correctitud y se apoya en Raft, que requiere quórum para progresar.

Decisión

NexusMQ es un sistema **CP**: ante una partición de red, una partición de datos que se quede sin **quórum** de su grupo Raft **deja de aceptar escrituras** y, por tanto, no diverge.

En términos de PACELC es **PC/EC**: ante una **Partición** elige **Consistencia**, y en condiciones normales (**E**lse) también prioriza **Consistencia** sobre **Latencia** (las escrituras esperan al quórum cuando se usa `acks=quorum`).

La lectura por defecto se sirve desde el **líder**, hasta el *high-watermark*. Las lecturas *stale* desde *followers* son **opt-in** y quedan documentadas.

Consecuencias

- (+) No hay divergencia ni pérdida de datos *committed*; el comportamiento es coherente con Raft y con el objetivo de *correctitud demostrable*.
- (+) Modelo mental simple para el usuario.
- (–) Una partición minoritaria queda **no disponible** para escritura durante un *split*; se acepta como precio de la consistencia.

Alternativas consideradas

- **AP (alta disponibilidad, escrituras en minoría con reconciliación posterior)**: ofrece mayor disponibilidad pero abre la puerta a la divergencia y a los conflictos; es incoherente con el modelo de log ordenado y con Raft. Descartada.

ADR-0008: Viabilidad de coste cero (desarrollo y test gratuitos; cloud como *target* opcional)

- **Estado:** aceptado
- **Fecha:** 2026-06-07

Contexto

El proyecto se **orienta** a despliegue en Azure/AWS, pero su valor formativo y de portfolio no debe depender de gasto recurrente en cloud. Conviene fijar explícitamente que todo el ciclo de trabajo se puede realizar de forma gratuita.

Decisión

Realizar **todo** el desarrollo, el testing (unitario, de integración y *crash/chaos*) y el *benchmarking* **en local**, **sin coste**:

- *Toolchain* y dependencias **open source**.
- Cluster de 3 nodos mediante **Docker Compose**.
- *Chaos* con `tc netem` y `cgroups`.
- CI en **GitHub Actions** (gratis en repositorios públicos / *free tier* en privados).
- Observabilidad **Prometheus + Grafana self-hosted**.

El despliegue en **cloud** es un **objetivo de diseño** (imagen Docker, principios 12-factor) y, como mucho, una **demo opcional** cubrible con *free tiers* (AWS/Azure/Oracle Always Free), nunca un requisito de gasto. Los *benchmarks* se miden **en local**, ya que los *free tiers* no sirven para obtener cifras serias.

Consecuencias

- (+) Barrera de entrada económica nula y reproducibilidad del ciclo completo.
- (+) El diseño *cloud-ready* permite una demo desplegada si se desea.
- (–) Una demo en cloud real con *hardware* serio sí costaría dinero (egress, almacenamiento, instancias encendidas); se mitiga con IaC bajo demanda.
- (–) Los *free tiers* no permiten *benchmarks* representativos; se asume y por eso se miden en local.

Alternativas consideradas

- **Depender de cloud para test/bench:** descartada por el coste y por la peor reproducibilidad de los *benchmarks*.

ADR-0009: Política de manejo de errores por capa (excepciones / `std::expected` / códigos de wire)

- **Estado:** aceptado
- **Fecha:** 2026-06-10

Contexto

`principios/manejo-errores.md` prescribe **excepciones**, no **códigos de error** para la **lógica de negocio de aplicación**, mientras que `stacks/cpp` avala `std::expected / error_code` en **código de sistemas y baja latencia**. El broker tiene **tres superficies de error** distintas: el **protocolo de red** (contrato), el **núcleo de sistemas** (camino caliente) y el **plano de control** (admin/REST). Hay que fijar qué mecanismo rige en cada una para no contradecir la normativa ni pagar el coste de las excepciones en el *hot path*.

Decisión

Política por capa, alineada con la biblioteca:

- **Protocolo / wire (§7.2.2):** **códigos de error numéricos** (`errorCode:i16`) como **contrato**; se **traducen al modelo interno en el borde**, no se propagan crudos por el núcleo (`manejo-errores: "los códigos de error de protocolo... son legítimos"`).
- **Núcleo de sistemas / camino caliente** (storage, reactor, consensus): `std::expected<T,E>` (o `std::error_code / Result`) — el error es un **valor**, de coste predecible; **sin excepciones en el hot path**. Se evaluará `-fno-exceptions` solo con **justificación medida** (coste de *unwinding* / latencia de cola).
- **Plano de control / aplicación** (admin, configuración): **excepciones para lo excepcional** más `std::optional` para la ausencia de valor; en el **borde REST**, traducción a **ProblemDetail (RFC 7807)** (§7.6).
- **Invariantes transversales (no cambian):** aportar **contexto** en el error; **validar en el borde** (*fail-fast*, §7.9); mantener **una sola política central**; **nunca tragar** un error; y **envolver** los errores de librerías de terceros (liburing/OpenSSL) en tipos propios.

Consecuencias

- (+) Coherente con `principios/manejo-errores.md` y con `stacks/cpp` ; sin coste de excepciones en el camino caliente; modelo de error de cara al cliente estándar (códigos en binario, RFC 7807 en REST).
- (–) Hay que **propagar expected a mano** (mitigado con los monádicos `and_then / or_else`) y mantener la **traducción en los bordes** entre los tres modelos.

Alternativas consideradas

- **Excepciones en todo (incluido el hot path):** contradice la baja latencia (*unwinding* no determinista) y la guía de sistemas de `stacks/cpp` ; descartada para el núcleo.
- **Códigos de error en todo (incluida la aplicación):** contradice `manejo-errores.md` para la lógica de aplicación; descartada fuera del núcleo/contrato.

ADR-0010: Entorno de desarrollo (VS Code sobre WSL; reemplaza el IDE de ADR-0001)

- **Estado:** aceptado (reemplaza la elección de IDE de ADR-0001)
- **Fecha:** 2026-06-11

Reemplaza la elección de IDE de ADR-0001. El resto de ADR-0001 (*target* Linux primero, WSL2, CMake + vcpkg, Windows después) **sigue vigente**; ADR-0001 no se edita (un ADR aceptado se reemplaza, no se modifica).

Contexto

ADR-0001 fijó desarrollar **desde Visual Studio** (workload "Linux development with C++") sobre WSL2. El código del proyecto vive **dentro del sistema de ficheros de WSL** (`/home/...` , visible desde Windows como `\\wsl.localhost\...`).

El modelo CMake+WSL de Visual Studio **asume que el código reside en el FS de Windows** y lo sincroniza a WSL con `rsync` ; al abrir una carpeta que ya vive en WSL sobre el *share* `\\wsl.localhost` , VS **no activa la integración CMake** (no genera caché, no aparecen *targets* ni "Elemento de inicio"), con cuelgues al cerrar. Se verificó con **VS 2026 (18.7)**: presets válidos (`cmake --list-presets` OK), `enableCMake: true` y componentes Linux/WSL instalados — el bloqueo es de **diseño de VS**, no de configuración.

Decisión

Migrar el IDE a **VS Code conectado a WSL** (extensiones **Remote-WSL** + **CMake Tools** + **C/C++**), abriendo el proyecto en su ruta Linux real (`code .` desde WSL). CMake Tools consume el **mismo CMakePresets.json** y permite configurar, compilar, ejecutar y **depurar (gdb) nativo** en WSL, sin copias ni `rsync` . El código permanece en WSL (mejor I/O que `/mnt/c`).

Se **estandariza** en los presets `linux-gcc/clang/asan` (Ninja, los mismos que el CI) y se eliminan los presets `wsl-ubuntu*` (Makefiles) y los *vendor maps* `microsoft.com/VisualStudioSettings` introducidos para VS.

Consecuencias

- (+) El IDE trabaja donde vive el código: sin `rsync` , sin cuelgues, **un único CMakePresets.json** para IDE, CI y terminal, con idéntico *toolchain* (gcc/gdb/cmake).
- (+) Experiencia gráfica equivalente (configurar/compilar/ejecutar/depurar).
- (+) Menos superficie de configuración (un solo conjunto de presets).
- (–) Se renuncia a Visual Studio como IDE (preferencia previa del autor) para este proyecto.
- (–) Recuperar VS exigiría mover el repo al FS de Windows — descartado por el flujo *Linux-first*.

Alternativas consideradas

- **Mover el repo a `c:\` y seguir en Visual Studio (modelo `rsync` de VS):** funcional, pero el origen pasa a Windows, con peor rendimiento de I/O en WSL2 y fricción de *line endings*/permisos; contradice *Linux-first*. Descartado.
- **Forzar VS sobre `\\wsl.localhost` :** no es un flujo soportado; es la causa de la no-activación y de los cuelgues. Descartado.
- **VS Code + Remote-WSL:** elegido (ver Decisión).

ADR-0011: Estándar C++23 + libc++ en Clang (para

`std::expected`)

- **Estado:** aceptado (refina el mecanismo de ADR-0009, no lo reemplaza)
- **Fecha:** 2026-06-11

Refina el mecanismo de ADR-0009 (no lo reemplaza). ADR-0009 fija `expected<T> = std::expected<T, Error>` en el núcleo; este ADR registra las **consecuencias de toolchain** de esa decisión, descubiertas al implementar el modelo de errores en M2.

Contexto

`std::expected` es **C++23**, pero el proyecto se fijó en C++20 (`cxx_std_20`). Al implementar `error.hpp` se constató, además, que **Clang 18 + libstdc++ no compila `<expected>`** ni siquiera en C++23: el `<expected>` de libstdc++ exige `__cpp_concepts >= 202002L` y Clang 18 reporta `201907L`. GCC (libstdc++) sí lo compila (`__cpp_concepts = 202002L`).

Sin solución, el *lane* de Clang del CI —y **clang-tidy**, que parsea con el frontend de Clang— quedarían rotos en todo fichero que use el modelo de errores.

Decisión

Subir el estándar a `cxx_std_23`. GCC compila con libstdc++ (por defecto). Clang compila con libc++ (`-stdlib=libc++`), fijado **globalmente** en el preset `linux-clang` y en el CI (build + lint), de modo que también las dependencias traídas por FetchContent (GoogleTest/Benchmark) usen libc++ y no haya choque de ABI.

clang-tidy consume el `compile_commands.json` de ese preset, así que parsea con libc++. Se documenta en `CLAUDE.md` (hechos del proyecto) y se requiere `libc++-dev` / `libc++abi-dev` donde se use Clang.

Consecuencias

- (+) `std::expected` disponible y **probado en ambos compiladores** (GCC/libstdc++ y Clang/libc++); se conserva la cobertura de Clang y de clang-tidy.
- (+) Modelo de error de cara al cliente estándar (sin coste de excepciones en el camino caliente, ADR-0009).
- (–) Conviven **dos** librerías estándar (libstdc++ con GCC, libc++ con Clang): hay que instalar libc++ donde se compile con Clang; algo más de superficie de toolchain.
- (–) El estándar mínimo sube de C++20 a C++23 (el toolchain objetivo lo soporta de sobra).

Alternativas consideradas

- **expected propio (C++20) sobre `std::variant`**: portable sin libc++ y sin subir de estándar, pero se aparta del mecanismo elegido en ADR-0009 (`std::expected`); descartado por preferencia explícita de

mantener el tipo estándar.

- `std::expected` **solo con GCC (sin lane Clang)**: simple, pero pierde la cobertura de Clang y deja **clang-tidy** sin poder parsear el modelo de errores; descartado.

ADR-0012: Backend `io_uring` directo sobre el uapi del kernel (sin liburing)

- **Estado:** aceptado (precisa el diseño detallado de `nexus-io`, no cambia el diseño)
- **Fecha:** 2026-06-12

Precisa el diseño detallado de `nexus-io` (no cambia el diseño): el diseño detallado anotaba el backend del `Proactor` como «`io_uring (liburing)`». Este ADR registra que la **implementación** habla con `io_uring` **directamente** por el uapi del kernel, sin la biblioteca `liburing`. El **puerto `Proactor` y sus contratos no cambian**; solo el backend.

Contexto

El backend `io_uring` (R5) debe cumplir la **puerta de calidad**: compilar y testear **en local y en CI**, en GCC y Clang, bajo sanitizers. En el entorno de desarrollo no hay `sudo` para instalar `liburing-dev` y `vcpkg` no está *bootstrapeado* (se cae a `FetchContent`); `liburing` usa *autotools* (no CMake), así que vendorizarla de forma reproducible es frágil.

En cambio, el **uapi del kernel** (`<linux/io_uring.h>`) está presente tanto en local (WSL2, kernel 6.18) como en el *runner* de CI (ubuntu-24.04), y `io_uring` funciona en tiempo de ejecución en local (probado: `io_uring_setup` OK, `IORING_FEAT_SINGLE_MMAP`).

Decisión

Implementar `IoUringBackend` **directamente sobre el uapi**: `io_uring_setup` / `io_uring_enter` vía `syscall`, anillos SQ/CQ mapeados con `mmap` (se exige `IORING_FEAT_SINGLE_MMAP`), barreras *acquire/release* sobre los índices compartidos con el kernel y SQEs gestionadas a mano. **Cero dependencias externas**: el mismo binario compila en todas partes solo con cabeceras del kernel.

CMake compila el backend solo donde existe `<linux/io_uring.h>` (define `NEXUS_HAVE_IOURING`); el *smoke-test* hace E/S real y se **omite en ejecución** (`GTEST_SKIP`) si `io_uring` no está disponible (kernel viejo, *seccomp* del CI). Toda la gestión cruda (fd, mmap, ops en vuelo) queda confinada en un *pimpl* RAII, sin filtrar el uapi al resto del árbol.

Consecuencias

- (+) Sin dependencias ni instalación: build idéntica en local y CI, sin `sudo /vcpkg/autotools`.
- (+) Control total del anillo (afín al *shared-nothing* y al objetivo de aprender sistemas) y *hot-path* sin biblioteca intermedia.
- (+) Validado en local en los cuatro *lanes* (GCC/Clang/ASan/TSan) porque `io_uring` corre aquí.
- (–) Reimplementamos lo que `liburing` ofrece hecho (setup de anillos, *helpers* de SQE): más código propio que mantener y más superficie de error de bajo nivel.
- (–) Acceso a uniones del `io_uring_sqe` (ABI del kernel): obliga a desactivar `cppcoreguidelines-pro-type-union-access` (igual criterio que `pro-bounds-*` / `pro-type-vararg` para código de sistemas).

- (–) `wake()` queda como *no-op* hasta R6 (requiere registrar un `eventfd` en el anillo).

Alternativas consideradas

- **liburing vía `liburing-dev` (`apt`):** lo estándar, pero requiere `sudo` (no disponible) y añade una dependencia de sistema a instalar en cada entorno; descartado por la restricción del entorno.
- **liburing vendorizada (`FetchContent/ExternalProject`):** reproducible sin `sudo`, pero liburing usa *autotools* (genera cabeceras con `./configure`): wrapper frágil en CMake y más lento en CI; descartado por complejidad frente a la opción sin dependencias.

ADR-0013: Capa `nexus-wire` para el framing sobre conexión

(`FrameReader` / `FrameWriter`)

- **Estado:** aceptado
- **Fecha:** 2026-06-13

Contexto

Este ADR **refina el diseño detallado** (no cambia el diseño del protocolo): el diseño detallado original ubicaba `FrameReader` / `FrameWriter` en `protocol/frame.hpp`, pero el grafo de dependencias fija `nexus-protocol` → `common` y «protocolo puro (encode/decode, sin E/S ni async)». La decisión resuelve ese conflicto entre dos fuentes de verdad.

`FrameReader` / `FrameWriter` leen y escriben tramas longitud-prefijo (§7.2) sobre un `Socket` mediante el `Proactor`: necesitan `nexus-io` (`Socket`, `Proactor`) y corrutinas (`task<expected<T>>`). Colocarlos en `nexus-protocol` obligaría a que el protocolo —hoy puro encode/decode, testable y fuzzeable sin E/S— dependiera de `nexus-io` y de la maquinaria `async`, contradiciendo el grafo (`protocol` → `common`) y el principio de capas limpias. Los consumidores del framing-sobre-conexión (`broker`, `client`, `ingress`, `server`) ya dependen tanto de `io` como de `protocol`.

Decisión

Se crea un target nuevo, `nexus-wire` (`src/wire/`), que depende de **common + io + protocol** y aloja `Frame`, `FrameReader` y `FrameWriter`. `nexus-protocol` se mantiene **puro** (codec, `FrameHeader`, mensajes, códigos de error: solo → `common`, sin E/S ni `async`, independientemente testeable). `broker/client/ingress/server` dependen de `nexus-wire` para el transporte de tramas. El `Buffer` de `nexus-common` gana `extend` / `truncate` (cola mutable para `recv` sin copia intermedia), que usa `FrameReader`.

Consecuencias

- (+) `nexus-protocol` queda independiente de E/S: la (de)serialización se prueba y fuzzea sin tocar sockets ni `io_uring`.
- (+) Separación de responsabilidades clara (protocolo = bytes; wire = tramas sobre conexión) y reutilizable por todos los consumidores.
- (+) Grafo acíclico y descendente (`wire` → {`common`, `io`, `protocol`}).
- (–) Un target más en el árbol (15 en total) y una desviación respecto a la ubicación prevista en el diseño detallado original.
- (–) `read_frame` expone el payload como vista **dentro del búfer del lector** (válida hasta la siguiente lectura): zero-copy a cambio de una invariante de vida documentada.

Alternativas consideradas

- `nexus-protocol` depende de `io` (`FrameReader` en `protocol/frame_io.hpp`): sigue el diseño detallado original y evita un target nuevo, pero rompe la pureza del protocolo (lo acopla a `io_uring/async`) y el grafo `protocol` → `common`; descartado por el autor a favor de capas limpias.

- `FrameReader` **genérico en `nexus-io` (sin conocer `FrameHeader`)**: mantiene `io` → `common`, pero parte el concepto de «trama» en dos capas y se aparta del `read_frame()` → `Frame` del diseño detallado original; descartado.

ADR-0014: Modelo del log de Raft (índice = ordinal de entrada; término en sidecar)

- **Estado:** aceptado
- **Fecha:** 2026-06-14

Contexto

Este ADR **refina el diseño detallado** (no cambia ADR-0003): concreta *cómo* `RaftLog` es «una vista (`term`, `index`) sobre el `PartitionLog`» cuando el formato de `RecordBatch` (§5.4) no lleva campo de término y los offsets del log son **por record** (un batch abarca varios offsets).

ADR-0003 fija que «el log de Raft **es** el log de la partición». Al implementarlo surgen dos fricciones:

1. El `RecordBatch` (la unidad de replicación, §5.4) **no tiene** campo de término, y añadirse lo cambiaría el formato en disco y su CRC (toca todo `nexus-storage`).
2. El offset de partición es **por record** (un batch de N records ocupa N offsets), mientras que Raft razona con un **índice por entrada**. Igualar «índice = offset» obligaría a entradas de un solo record (mata el *batching*, contrario al §5.4) o a partir batches al truncar.

Decisión

Una **entrada de Raft** ↔ un `RecordBatch`. El **índice de Raft es el ordinal de la entrada** (1-based, +1 por entrada, contiguo), **espacio distinto** del offset de partición (por record). `RaftLog` envuelve un `PartitionLog` (que almacena los bytes de cada entrada y asigna sus offsets) y **posee el mapeo** `índice` → `(term, base_offset, last_offset, type)`.

El **término** (y el resto de metadatos por entrada) se persiste en un **sidecar** de registros de tamaño fijo (`raft-meta`, 25 B/entrada: `term:i64 | base_offset:i64 | last_offset:i64 | type:u8`), append-only y truncable; así el log de partición conserva `RecordBatch` **intactos** (el *fetch* del consumidor los lee sin desenvolver) y la recuperación del término no exige tocar el formato del batch. El *high-watermark* visible (espacio de offsets) = `last_offset` de la entrada en `commit_index`. La resolución de conflictos (`truncate_from(index)`) → `PartitionLog::truncate_to(base_offset(index))` (frontera de batch, ADR/C3) + truncado del sidecar.

Consecuencias

- (+) `RecordBatch` y `nexus-storage` quedan **sin cambios** (formato y CRC estables); el *fetch* sirve batches tal cual.
- (+) El algoritmo de Raft opera sobre índices contiguos +1 (formulación estándar del *paper*; sin aritmética de offsets).
- (+) El sidecar de tamaño fijo hace `term_at / last_term` /recuperación O(1)/lineal triviales.
- (–) Dos espacios de coordenadas (índice de entrada vs offset de record) que la capa de partición reconcilia (el *high-watermark* traduce de uno a otro).
- (–) Un fichero sidecar por partición además del `.log` / `.index`.

- (–) El mapeo se persiste por duplicado parcial (`base / last` son derivables del log escaneándolo), a cambio de un `open` simple sin re-escanear el log.

Alternativas consideradas

- **Añadir `leader_epoch / term` a la cabecera del `RecordBatch`** (estilo Kafka v2): fiel a «el log es el log», pero cambia el formato en disco y el CRC, e impacta todo `nexus-storage` y sus tests; descartado para no reabrir un formato ya estabilizado en Fase 1.
- **Índice = `offset+1` con entradas de un solo record**: iguala ambos espacios, pero elimina el *record batch* como unidad (contradice §5.4 y el modelo Kafka); descartado.
- **Anidar el `RecordBatch` del cliente dentro de un marco Raft** (`term/type` + `payload` como records del batch del log): término durable sin sidecar, pero cambia la semántica del `offset` a *por batch* (rompe el modelo por record de la Fase 1b) y obliga al *fetch* a desenvolverse; descartado.

ADR-0015: RaftNode como máquina de estados síncrona sin E/S (entradas → salidas)

- **Estado:** aceptado
- **Fecha:** 2026-06-14

Contexto

Este ADR **refina el diseño detallado** (no cambia ADR-0003 ni la mecánica de Raft): el diseño detallado original modela `RaftNode` con `propose / replicate_to` como **corrutinas** y un puerto `RaftTransport` (`task<expected<...>> send_append/send_vote`). La decisión reorganiza esa frontera sin alterar el algoritmo.

El objetivo nº1 de Fase 2 es la **correctitud demostrable** de Raft mediante **simulación determinista** con reloj y red virtuales (§8.1). Si `RaftNode` hace E/S por dentro (corrutinas que `co_await` un transporte), el reloj y la red quedan **dentro** del nodo y la prueba determinista exige inyectar un proactor/transport asíncrono y conducir corrutinas — friccionando con FIRST y con la reproducibilidad. El patrón estándar de implementaciones de Raft probadas (etcd/raft, TigerBeetle, ongaro) es un **núcleo sin E/S**: el nodo consume *entradas* (ticks de reloj, RPC recibidos) y produce *salidas* (mensajes a enviar, entradas a aplicar); el tiempo, la red y el disco viven **fuera**.

Decisión

`RaftNode` (`src/consensus/raft_node.hpp/.cpp`) es una **máquina de estados síncrona y sin E/S**: sin `RaftTransport`, sin corrutinas, sin reloj propio.

- **Entradas:** `tick(now)`, `on_request_vote(now, args) → reply`, `on_append_entries(now, args) → reply`, `on_request_vote_reply(now, from, reply)`, `on_append_entries_reply(now, from, reply)`, `propose(batch)` (C6).
- **Salidas:** una cola de mensajes salientes proactivos (`RaftMessage{from, to, payload}` con `payload = variant` de los RPC) drenable con `take_messages()`; las entradas confirmadas se exponen vía `commit_index()` (el broker lee el log hasta ahí).

El **reloj se inyecta** como `MonoTime now` en cada entrada (los *timeouts* de elección/heartbeat son aritmética pura sobre `now`); la **aleatorización** del *election timeout* usa un RNG sembrado (`random_seed + self`), reproducible. En producción, un adaptador del reactor traduce: temporizador → `tick`, RPC recibido → `on_*`, mensajes de `take_messages()` → envíos por `nexus-wire` (se cablea en la integración con el broker, C9). Respecto al diseño detallado original: se **añade** `now` a las firmas de los manejadores (determinismo) y se **sustituye** el par `propose: task<...> + RaftTransport` por `propose` síncrono + cola de salida (el adaptador asíncrono vive fuera del núcleo).

Consecuencias

- (+) Simulación **determinista** trivial: el arnés (C8) tiene un reloj virtual y una red virtual (cola de mensajes con retardos/particiones programables) y dirige N `RaftNode` reproduciblemente; cero hilos, cero E/S real en el test del algoritmo.
- (+) El núcleo de Raft se razona y prueba aislado de `io_uring/corrutinas`.

- (+) Encaja con *shared-nothing*: un `RaftNode` por partición, dirigido por su reactor.
- (–) Hace falta un **adaptador** (reactor ↔ nodo) que el diseño detallado original no nombraba (se añade en C9).
- (–) El emisor debe drenar `take_messages()` tras cada entrada (contrato explícito).

Alternativas consideradas

- `RaftNode` con corrutinas + `RaftTransport` (**diseño detallado literal**): menos piezas en producción, pero mete reloj/red dentro del nodo y complica la prueba determinista (el objetivo nº1); descartado.
- **Núcleo sin E/S con *callbacks* de envío** (en vez de cola drenable): equivalente, pero los *callbacks* reintroducen acoplamiento y dificultan inspeccionar las salidas en el test; se prefiere la cola explícita.

ADR-0016: `ReplicatedPartition` como tipo paralelo a `Partition` (composición de la pila Raft)

- **Estado:** aceptado
- **Fecha:** 2026-06-15

Contexto

Este ADR **refina el diseño detallado** (no cambia ADR-0003/0015): el diseño detallado original preveía **mutar** `Partition` para intercalar `RaftNode` y convertir `produce / fetch` en `task<expected<...>>`. La decisión mantiene `Partition` intacta y añade un tipo nuevo, sin alterar el algoritmo ni la frontera de E/S de ADR-0015.

En C9 hay que dar a la partición respaldo Raft (`acks=quorum` , `high-watermark = commit_index` , ADR-0003). La `Partition` de la Fase 1b funciona con `ack` local y sirve al broker e2e que ya pasa. Dos fricciones al mutarla in situ:

1. La pila de consenso es **autorreferencial** — `PartitionLog` $\leftarrow$ `RaftLog` (ref) $\leftarrow$ `RaftNode` (ref)—, así que un tipo con esos miembros por valor no sería **movible** sin invalidar referencias internas.
2. `RaftNode` es una **FSM sin E/S** (ADR-0015) que un portador externo debe conducir (`tick / on_* / take_messages`), contrato que aún no existe en el broker (llega con el *routing* multi-reactor, C11).

Mutar `Partition` ahora mezclaría ambos modelos (`ack` local vs `quorum`) en un tipo en uso y arriesgaría el camino e2e verde.

Decisión

Se añade **`ReplicatedPartition`** (`src/broker/replicated_partition.{hpp,cpp}`) como **tipo paralelo** a `Partition` , sin tocar esta última. Compone la pila por **`unique_ptr`** (`PartitionLog` + `RaftLog` + `RaftNode`): las direcciones de heap son estables, por lo que las referencias internas siguen válidas y el objeto es **movible** (move por defecto, copia borrada).

`produce` (solo líder; `Unsupported` si no) aplica la idempotencia por productor (§5.9, reutilizada de `Partition`) y **propone** la entrada al `RaftNode` , traduciendo el índice de Raft asignado a su último offset de partición vía `RaftLog::offsets_at` ; `high_watermark()` = offset (exclusivo) de la entrada en `commit_index` (`0` $\Rightarrow$ `log_start_offset`). La FSM **no se conduce sola**: `raft()` expone la superficie para que el portador (reactor o arnés) la dirija; en C9 se valida con **enrutado directo/simulado** (test con reloj y red virtuales que comprueba que una escritura no es visible hasta el `quorum`). El **cambio en caliente** del broker a `ReplicatedPartition` (con transporte real) se difiere a **C11**.

Consecuencias

- (+) `Partition` y el broker e2e quedan **intactos** (sin riesgo en el camino verde); la unidad de partición replicada queda lista y probada aislada.
- (+) Tipo **movible** pese a la pila autorreferencial (los `unique_ptr` fijan las direcciones).

- (+) Reutiliza idempotencia y `PartitionLog` / `RaftLog` sin duplicar lógica.
- (+) `produce` síncrono (la FSM no hace E/S, ADR-0015); cuando llegue el reactor, el portador drena `take_messages()` y espera a `commit_index`.
- (-) **Dos** tipos de partición conviviendo hasta C11 (deuda temporal explícita).
- (-) El cliente de `ReplicatedPartition` debe conducir `raft()` (contrato externo, como en ADR-0015).

Alternativas consideradas

- **Mutar `Partition` (diseño detallado literal)**: un solo tipo, pero rompe su movilidad por la pila autorreferencial, mezcla ack local/quorum en un tipo en uso y arriesga el e2e verde antes de tener el portador (C11); descartado.
- **`RaftNode` por valor dentro de la partición**: evita una indirección, pero el tipo deja de ser movable (referencias internas colgantes al mover) y complica almacenarlo en contenedores del broker; descartado a favor de los `unique_ptr`.
- **Fusionar ambos tras C11** (un tipo con modo local/quorum): posible evolución futura; hoy se prefiere separar para no acoplar la Fase 1b a la 2.

ADR-0017: Target `nexus-telemetry` para observabilidad (métricas/logs) bajo el broker

- **Estado:** aceptado
- **Fecha:** 2026-06-17

Contexto

Este ADR **refina el diseño detallado** (no cambia ninguna decisión de arquitectura previa): el diseño detallado original ubicaba `metrics.{hpp,cpp}` dentro de `nexus-server`, que es el **ejecutable** del broker. La decisión mueve la observabilidad a una **biblioteca** propia para respetar el grafo de dependencias.

La `MetricsRegistry` es **THREAD-SAFE** y la **alimentan** capas del plano de datos (broker/reactor: tasa de produce/fetch, *lag*, `commit_index` /term de Raft, latencias) mientras que la **exponen** el gateway REST (`nexus-ingress`, que tiene un `MetricsRegistry`) y el servidor (`/metrics`). Si viviera en `nexus-server` (un exe, no enlazable como dependencia) ninguna biblioteca inferior podría registrar métricas; y colocarla en `nexus-ingress` crearía una dependencia ascendente `broker → ingress` (ciclo de capas). Lo mismo aplica al logger JSON estructurado (lo usa todo el árbol). `nexus-common` se reserva para vocabulario mínimo (tipos, errores, bytes); una registry con mutex, familias y render Prometheus no encaja ahí.

Decisión

Se crea el target `nexus-telemetry` (`src/telemetry/`), que **depende solo de** `nexus-common` y se sitúa **bajo** broker/ingress/server en el grafo. Aloja `MetricsRegistry` (contadores/gauges/histogramas + exposición Prometheus) y, más adelante, el logger JSON estructurado. La registry es THREAD-SAFE con un **mutex que protege solo la estructura** (alta de series y recorrido en `render`); las **actualizaciones de valor** (`inc / set / observe`) son **atómicas y sin candado** (lock-free en el camino caliente). Cualquier capa (broker, reactor, ingress, server) enlaza `nexus-telemetry` para registrar o exponer.

Consecuencias

- (+) Grafo de dependencias **acíclico**: el plano de datos registra sin depender de capas superiores.
- (+) Observabilidad **testable en aislamiento** (sin red ni servidor).
- (+) `nexus-common` se mantiene mínimo.
- (+) Registro de valores **lock-free**; solo el alta de una serie nueva toma el mutex.
- (–) Un target más que mantener.
- (–) El *sharding* por núcleo de los contadores (cero contención total) queda como optimización futura medida; hoy basta con atómicos.

Alternativas consideradas

- **metrics en nexus-server (diseño detallado literal)**: imposible que las bibliotecas inferiores registren métricas (un exe no es dependencia); descartado.
- **metrics en nexus-ingress**: crea el ciclo de capas `broker → ingress`; descartado.

- **metrics en nexus-common** : mete infraestructura con estado (mutex, familias, render) en la capa de vocabulario mínimo; descartado por cohesión.

ADR-0018: REST admin por puerto/adaptador (`AdminService` en ingress, `AdminApi` en server)

- **Estado:** aceptado
- **Fecha:** 2026-06-18

Contexto

Este ADR **refina el diseño detallado** (no cambia ninguna decisión de arquitectura previa): el diseño detallado original ubicaba `admin_api.{hpp,cpp}` dentro de `nexus-server` y, a la vez, hacía que `RestGateway` (en `nexus-ingress`) tuviera un `AdminApi&`. Eso crea un **ciclo de capas** `ingress → server` (la pasarela usa la API) y `server → ingress` (el `Server` posee la `Ingress`). La decisión rompe el ciclo con **inversión de dependencias**, igual que ADR-0017 hizo con la telemetría.

El `RestGateway` (plano de ingress) traduce HTTP ↔ dominio y necesita ejecutar operaciones de administración (crear/borrar/describir/listar topics y grupos). La lógica de esas operaciones vive sobre `TopicManager / GroupCoordinator`, que son del **broker** (capa inferior a server, pero `ingress` **no** depende de broker en el grafo: `ingress → common, io, protocol, wire`). Si el gateway dependiera de un `AdminApi` concreto alojado en `nexus-server`, `ingress` dependería de `server` (que ya depende de `ingress`): ciclo.

Decisión

Se define el **puerto** `AdminService` (interfaz abstracta) y sus **DTOs** (`CreateTopicSpec`, `TopicSummary`, `PartitionInfo`, `TopicDescription`, `GroupSummary`) en `nexus-ingress` (`ingress/admin_service.hpp`), como tipos de datos planos sin dependencia del broker. El `RestGateway` depende solo de `AdminService&`. El **adaptador** concreto `AdminApi` (en `nexus-server`, `server/admin_api.{hpp,cpp}`) **implementa** `AdminService` sobre `TopicManager&` y un *group lister* inyectado (`std::function`), traduciendo los tipos del broker a los DTOs del puerto. La enumeración de grupos es **reactor-local** (cada `GroupCoordinator` vive en su reactor), así que se inyecta como función desde el cableado del server (I14), no se acopla al puerto. Se añade a `TopicManager` un accesor de observabilidad `list_metadata()` (control-plane).

Consecuencias

- (+) Grafo **acíclico**: `ingress` define el puerto (sin nuevas dependencias), `server` implementa el adaptador (ya depende de `ingress` y `broker`).
- (+) `RestGateway` es **testable** con un doble de `AdminService` sin levantar broker.
- (+) Los DTOs del REST quedan **desacoplados** de los tipos internos (ADR-0009: traducción en el borde).
- (–) Una capa de traducción DTO ↔ dominio en el adaptador.
- (–) El listado de grupos cross-core real se materializa en el cableado (I14), no en el puerto.

Alternativas consideradas

- **AdminApi concreto en nexus-server , referenciado por ingress (diseño detallado literal):** crea el ciclo `ingress ↔ server` ; descartado.
- **AdminApi en nexus-broker :** tendría que implementar la interfaz `AdminService` de `ingress` ⇒ `broker → ingress` (dependencia **ascendente**, prohibida); descartado.
- **RestGateway como router genérico de *handlers* (`std::function`):** rompe el ciclo, pero diluye el contrato de administración en *callbacks* sin tipado; el puerto explícito documenta mejor la API y es más fiel al diseño detallado original.

ADR-0019: TLS opcional vía OpenSSL con puente de BIOs de memoria sobre el `Proactor`

- **Estado:** aceptado
- **Fecha:** 2026-06-19

Refina el diseño detallado (no cambia ninguna decisión previa): el diseño detallado original ya situaba `TlsContext / TlsConnection` sobre OpenSSL en `nexus-ingress`. Este ADR fija **cómo** se integra OpenSSL (síncrono por diseño) con el modelo `proactor` asíncrono y **qué grado de dependencia** es OpenSSL en el build.

Contexto

El plano de `ingress` (ADR-0006) termina TLS 1.3 y, intra-clúster, mTLS. La librería elegida es OpenSSL (madura, ubicua), pero su API de E/S es **bloqueante/síncrona** y NexusMQ es **thread-per-core asíncrono** sobre un `Proactor` (`io_uring`, ADR-0005/0002). Además, OpenSSL es una dependencia de sistema pesada que no siempre está presente (por ejemplo, entornos mínimos de CI o builds de desarrollo), y el broker debe poder **arrancar en claro** para pruebas locales y despliegues sin TLS.

Decisión

Se toman tres decisiones coordinadas:

1. **Acoplamiento opcional.** El build usa `find_package(OpenSSL)`; si está presente, compila `ingress/tls.{hpp,cpp}` y define `NEXUS_HAVE_OPENSSL` (público). Si no, el plano TLS se **omite por completo** (cabecera y `.cpp` guardados con `#ifdef`, tests con `GTEST_SKIP`) y el broker funciona en claro. El CI instala `libssl-dev` para **sí** ejercer el path TLS (compilación, tests, sanitizers y clang-tidy).
2. **Puente de BIOs de memoria.** `TlsConnection` no deja que OpenSSL toque el descriptor; usa dos BIO de memoria (`rbio / wbio`). El bucle de `handshake / async_recv / async_send` llama a la primitiva OpenSSL y, ante `WANT_READ / WANT_WRITE`, **vacía** el `wbio` al socket y **alimenta** el `rbio` desde el socket mediante el `Proactor` asíncrono. Así la criptografía corre en línea pero el transporte es no bloqueante.
3. **Fábricas en el contexto.** `TlsContext::server / client` cargan material PEM y configuran la verificación (mTLS = `SSL_VERIFY_PEER|FAIL_IF_NO_PEER_CERT + CA`); `accept / connect` crean la `TlsConnection` fijando el rol del handshake. `peer_principal()` extrae el CN del certificado del par para authz.

Consecuencias

- (+) El núcleo asíncrono **no se contamina** con E/S bloqueante; TLS encaja tras el mismo `Proactor` que el resto.
- (+) Build y despliegue **sin OpenSSL** siguen siendo válidos (degradación a claro), cumpliendo coste-cero (ADR-0008).

- (+) `TlsContext` / `TlsConnection` se prueban por loopback con certificados autofirmados generados en el test (una vía, mTLS con principal, y fallo de verificación).
- (-) El puente BIO añade una copia intermedia ciphertext ↔ socket (acceptable; el throughput TLS lo domina la cifra, no la copia).
- (-) La compilación condicional obliga a `#ifdef` en cabecera, `.cpp` y tests.

Alternativas consideradas

- **BIO propio sobre el Proactor (custom BIO method):** evita una copia, pero exige implementar un `BIO_METHOD` con semántica de reintento correcta y es mucho más frágil; el puente de BIOs de memoria es el patrón canónico y suficiente.
- **OpenSSL como dependencia obligatoria (vcpkg/sistema):** simplifica el build, pero rompe coste-cero y el arranque en claro para desarrollo; descartado a favor del acoplamiento opcional.
- **Otra librería (BearSSL/mbedtls/rustls-ffi):** menor huella, pero peor ergonomía/cobertura y menos familiar; OpenSSL ya estaba fijado en el diseño detallado original.

ADR-0020: *Binding* de Python vía ABI C estable (`nexus-ffi` + `ctypes`) en lugar de pybind11

- **Estado:** aceptado
- **Fecha:** 2026-06-20

Ajusta el alcance previsto (la fase F9 contemplaba «*binding* Python (pybind11) si el entorno lo soporta»): fija **cómo** se expone NexusMQ a Python cuando el entorno **no** soporta construir extensiones de CPython.

Contexto

F9 (Fase 4) pide un *binding* de Python. La opción por defecto (pybind11) compila una **extensión nativa de CPython**, que necesita las **cabeceras de desarrollo de Python** (`Python.h` , `pyconfig.h`) y una *toolchain* de extensiones. El entorno de desarrollo y los *runners* de CI **no** las tienen (solo el intérprete; sin `python3-dev` , sin `pip` , sin `sudo` para instalarlas), de modo que una extensión pybind11 **no se podría compilar ni verificar** aquí — chocaría con la puerta de calidad (TDD, «nunca pushear en rojo», verificar en local en los dos compiladores).

Decisión

Exponer un subconjunto **puro y transversal** del núcleo tras una **frontera C** `extern "C"` en un target nuevo `nexus-ffi` , compilado como **librería compartida** (`libnexus-ffi.so`), y consumirla desde Python con `ctypes` (módulo `bindings/python/nexusmq.py`). La ABI (`src/ffi/nexus_ffi.h`) cubre: versión, **CRC32C** (integridad de records) y el codec de **contexto de traza W3C traceparent** (F8). Las ventajas son tres:

1. Se compila con el **propio compilador de C++** (sin cabeceras de Python), así que entra en la puerta de calidad (GCC/Clang/ASan + clang-tidy sobre `src/ffi/nexus_ffi.cpp`).
2. Se verifica con **cualquier** intérprete de Python presente (prueba de humo `bindings/python/smoke_test.py` con `ctypes`).
3. La ABI C es **estable** y sirve a otros lenguajes (Rust/Go/Node FFI), no solo a Python.

Para enlazar los archivos estáticos del núcleo dentro de la `.so` , `nexus-common` y `nexus-telemetry` se marcan `POSITION_INDEPENDENT_CODE` .

Consecuencias

- (+) *Binding* **realmente verificado** en este entorno (ABI por GoogleTest en la puerta de calidad; lado Python por la prueba de humo), no código sin compilar.
- (+) Sin dependencias nuevas de build (ni pybind11 ni `python3-dev`); CI intacto.
- (+) La frontera C es reutilizable por otros lenguajes.
- (–) Ergonomía más baja que pybind11 (gestión manual de tipos/ `ctypes` , sin objetos Python ricos automáticos) y superficie acotada a funciones puras (no expone aún el cliente productor/consumidor).
- (–) Introduce la primera librería **compartida** del árbol (y PIC en dos targets base).

Alternativas consideradas

- **pybind11 (extensión de CPython):** mejor ergonomía, pero **no construible ni verificable** sin `python3-dev` / `toolchain` en este entorno/CI; quedaría como código sin compilar, contra la normativa.
Reconsiderable si el entorno gana las cabeceras de desarrollo.
- **Cabeceras de Python vendoreadas a mano (prefijo local, como OpenSSL en ADR-0019):**
`pyconfig.h` es **generado por la build de CPython** y específico de plataforma; replicarlo a mano es frágil y los *runners* de CI tampoco lo tendrían. Descartado.
- **Servicio REST + cliente HTTP en Python:** ya existe el gateway REST (Fase 3), pero eso es interoperabilidad de red, no un *binding* en proceso; complementario, no sustituto.

ADR-0021: Backend IOCP (Windows) — diseño fijado, implementación diferida

- **Estado:** reemplazado por adr-0022
- **Fecha:** 2026-06-20

Cierra F10 (Fase 4, *stretch*, marcada «[limitado] — no verificable en este entorno»): decide **cómo** encajaría IOCP en la arquitectura y **por qué** la implementación se difiere.

Contexto

F10 pide un backend de E/S para **Windows** (IOCP) en `nexus-io` y un preset `windows-msvc`. IOCP es, como `io_uring`, un modelo **proactor** (se inicia la operación y el SO notifica al completarse), así que conceptualmente encaja en el puerto `Proactor` (ADR-0002/0005) sin tocar el bucle del reactor. Pero hay dos obstáculos duros:

1. **No es verificable en este entorno:** el desarrollo y el CI son **solo Linux** (WSL), sin *toolchain* MSVC ni Windows SDK; no se puede compilar ni probar nada de Win32 aquí, y la normativa prohíbe *pushear* código no verificado como si funcionara.
2. **Alcance real:** NexusMQ es **Linux-nativo de arriba abajo** — `io_uring`, `O_DIRECT`, `eventfd`, `sockets` POSIX, `fcntl`, `file descriptors` `int` en el propio puerto `Proactor`. Un backend IOCP útil exige **portar también File y Socket** a Win32 (`HANDLE / SOCKET`, `ReadFile / WriteFile` con `OVERLAPPED`, `Winsock`, `AcceptEx`), un esfuerzo transversal desproporcionado para un ítem *stretch*.

Decisión

Fijar el diseño y diferir la implementación.

1. Se documenta el mapeo IOCP → `Proactor` operación a operación en `src/io/iocp_backend.hpp` (`IocpBackend : Proactor`, *pimpl* que oculta `<windows.h>` / `<winsock2.h>`):
`CreateIoCompletionPort` para el puerto y asociación de cada `HANDLE / SOCKET`;
`ReadFile / WriteFile` (con `Offset / OffsetHigh`), `WSARecv / WSASend`, `AcceptEx`,
`FlushFileBuffers`, *waitable timers*; `GetQueuedCompletionStatusEx` para drenar *completions* en **lote** (espera con *timeout* hasta el *deadline*); `PostQueuedCompletionStatus` para *wake*; traducción de `dwNumberOfBytesTransferred / GetLastError` al `result` estilo `io_uring` del puerto.
2. Todo el encabezado va bajo `#ifdef _WIN32` y **sin .cpp**: en Linux es vacío, no entra en la build ni en la puerta de calidad (no rompe nada).
3. Se añade el preset `windows-msvc` como **andamio**, oculto fuera de Windows por `condition`.

Consecuencias

- (+) El diseño Windows queda **registrado y concreto** (declaración + mapeo), demostrando que el puerto `Proactor` es portable a otro proactor sin cambiar el reactor.
- (+) Cero impacto en el árbol Linux y en CI (encabezado vacío bajo `#ifdef`, preset condicionado).

- (+) Honestidad de verificación: no se publica un IOCP «de mentira» sin compilar.
- (–) F10 **no** entrega un backend funcional: queda como diseño + andamio.
- (–) El port real requerirá, antes que `IocpBackend`, abstraer `File / Socket` del POSIX (trabajo futuro fuera de Fase 4).

Alternativas consideradas

- **Implementar `IocpBackend` completo (con `.cpp`) ahora:** imposible de compilar/verificar aquí; sería código no probado contra la normativa (TDD, «nunca en rojo»). Descartado.
- **Job de CI en *runner* Windows:** GitHub ofrece `windows-latest`, pero el árbol entero no compila en MSVC (POSIX por doquier); habilitar un build Windows real es el *port* completo, no F10. Descartado por alcance.
- **Capa de compatibilidad POSIX en Windows (Cygwin/MSYS2 o Winsock+wepoll):** permitiría reusar el código POSIX, pero no es IOCP nativo (objetivo de F10) y añade dependencias; descartado.

ADR-0022: Backend IOCP (Windows) — implementado y compile-verificado con MinGW (reemplaza ADR-0021)

- **Estado:** reemplazado por adr-0023
- **Fecha:** 2026-06-20

Reemplaza ADR-0021. Aquel difirió la implementación IOCP por considerarla *no verificable en este entorno* (sin MSVC/SDK; CI solo Linux). Esa premisa **ya no se sostiene**: el entorno **sí** dispone (vía `apt`) del cross-compiler **MinGW-w64** con headers Win32 reales, que permite **compilar y enlazar** el código Windows sin una máquina Windows. Decisión del autor: llevar Windows al mismo nivel de desarrollo que Linux.

Contexto

ADR-0021 dejó `iocp_backend.hpp` como diseño bajo `#ifdef _WIN32` y `File / Socket` como POSIX puro, con `int fd` en el puerto `Proactor`. El obstáculo declarado era la verificación. Con MinGW-w64 disponible, el código Win32 se puede compilar con `-Wall -Wextra -Wpedantic -Werror` contra los headers reales (`<windows.h>`, `<winsock2.h>`, `<mswsock.h>`) y enlazar contra `ws2_32 / mswsock` — cubriendo la inmensa mayoría de errores (firmas, uso de API, tipos). Solo queda fuera la verificación en **runtime** (requiere Windows real), que se asume como deuda explícita acotada.

Decisión

Implementar el port Windows de la capa de E/S y verificarlo por compilación+enlace con MinGW.

1. `NativeHandle` (= `int` en POSIX, `uintptr_t` en Windows) sustituye a `int fd` en el puerto `Proactor`, `File` y `Socket`; el `IoResult` del `Completion` se ensancha a `intptr_t` para alojar un `SOCKET / HANDLE` aceptado (en Linux, sin cambio de comportamiento).
2. `File` y `Socket / Listener` ganan su rama `#if defined(_WIN32)` (Win32: `CreateFileA / ReadFile / WriteFile` CON `OVERLAPPED`, `FlushFileBuffers`, `SetEndOfFile`, `GetFileSizeEx`, `FILE_FLAG_NO_BUFFERING` como `O_DIRECT`; `Winsock`: `socket / connect / bind / closesocket`, `WSAStartup` `RAII`).
3. `iocp_backend.cpp` implementa `IocpBackend` sobre `CreateIoCompletionPort / GetQueuedCompletionStatusEx / PostQueuedCompletionStatus`, con `AcceptEx`, temporizadores por `timeout` de la espera y la maquinaria Win32 confinada en un `pimpl Port`.
4. CMake compila `iocp_backend.cpp` y enlaza `Winsock` en `WIN32`.
5. `tools/verify-windows-io.sh` reproduce la verificación cruzada.

Consecuencias

- (+) `nexus-io` (la capa que aísla la plataforma) es **Windows-capaz y compile/enlace-verificada**, demostrando que el puerto `Proactor` es portable a otro proactor sin tocar el reactor.

- (+) Cero impacto en Linux: el cambio `NativeHandle / IoResult` es *type-identical* (alias de `int / intptr_t`), 599 tests verdes en GCC y Clang/libc++.
- (+) La verificación es **honesta**: compila y enlaza con toolchain Windows real, no es código «a ciegas».
- (–) **Runtime no verificado** en Windows (sin máquina/CI Windows): deuda explícita acotada — un job `windows-latest` lo cerraría al reactivar el CI.
- (–) El **resto del servidor** (reactor *pinning* con `pthread_setaffinity_np`, `nexusd` con señales POSIX/eventfd) sigue siendo Linux-nativo: un `nexusd` Windows completo es trabajo futuro fuera de este alcance; lo entregado es la **capa de E/S** portable.

Alternativas consideradas

- **Mantener ADR-0021 (solo diseño)**: desaprovecha que MinGW está disponible; deja Windows sin avanzar pese a poder verificarlo. Descartado por decisión del autor.
- **Solo refactor portable (sin los `.cpp Win32`)**: verificable en Linux pero no entrega el backend. Descartado a favor del port completo.
- **Job de CI en `windows-latest` para runtime**: cerraría la deuda de runtime, pero el CI está desactivado por cuota (se reactiva al publicar) y el árbol completo aún no compila en Windows (POSIX fuera de `nexus-io`). Aplazado.

ADR-0023: Backend IOCP (Windows) — verificado en runtime sobre Windows con MSVC (reemplaza ADR-0022)

- **Estado:** aceptado
- **Fecha:** 2026-06-20

Reemplaza ADR-0022. Aquel **implementó y compile-verificó** el backend IOCP con MinGW-w64, pero dejó como deuda explícita acotada la verificación en **runtime** («Runtime no verificado en Windows ... un job `windows-latest` lo cerraría»). Esa deuda **ya no aplica**: el código se ha **ejecutado** en una máquina Windows real con MSVC y funciona. El diseño no cambia; ADR-0023 lo eleva de *compile-verificado* a *runtime-verificado* (mismo patrón que ADR-0021 → 0022).

Contexto

ADR-0022 cerró con `nexus-io` compile/enlace-verificado contra headers Win32 reales vía cross-compiler, pero **nunca ejecutado** (sin MSVC ni máquina Windows). MinGW (GCC) no ve las divergencias de MSVC (por ejemplo, `__builtin_*`, atributos `[[gnu::*]]`) ni cubre el **comportamiento en ejecución** (asociación de handles/sockets al puerto, `AcceptEx` + `SO_UPDATE_ACCEPT_CONTEXT`, `OVERLAPPED` en vuelo, drenado de `GetQueuedCompletionStatusEx`, temporizadores por *timeout*). El autor pasa a trabajar en **Windows real** con **Visual Studio 2026** (MSVC 19.51, CMake/Ninja *bundled*), lo que permite cerrar la deuda.

Decisión

Cerrar la deuda de runtime ejecutando `nexus-io` en Windows real con MSVC, mediante un arnés de verificación dedicado (en lugar de un job de CI: el CI sigue desactivado por cuota y el árbol completo no compila en Windows).

1. El preset `windows-msvc` con `-D NEXUS_BUILD_TESTS=OFF` compila **solo** `nexus-io` (+ `nexus-common`) con MSVC `/W4 /WX /permissive-`.
2. `tools/wincheck` es un ejecutable **Windows-only** (`CMakeLists` bajo `if(WIN32)`, **sin** `ctest` ni `ReactorPool` —que es POSIX—) enlazado a `nexus::io` que **ejercita** la capa: `File` bloqueante (round-trip `write_at / read_at`, `size / sync / truncate`) y `File` con E/S directa (`FILE_FLAG_NO_BUFFERING`, búfer alineado a 4 KiB); **eco IOCP por loopback** end-to-end (`Listener::bind / Socket::connect` + `AcceptEx / WSAREcv / WSASend`) con cierre limpio (EOF), bombeando `wait_completions / run_completions` a mano con un **driver de corrutinas** propio (`sync_wait` no sirve: la *task* se suspende en E/S asíncrona); y `submit_timer` (deadline corto que vence y reanuda).
3. **Resultado: los 4 casos PASAN** con MSVC 19.51 (VS 2026).

Consecuencias

- (+) `nexus-io` deja de tener deuda de runtime: la portabilidad del puerto `Proactor` a otro proactor (IOCP) queda demostrada **ejecutándose**, no solo compilando.
- (+) MSVC `/W4 /WX` validó lo que MinGW no veía y reveló **una sola** divergencia real, de portabilidad de detección de *CPU features* en `nexus-common` (`crc32c.cpp` : `__builtin_cpu_supports / [[gnu:target("sse4.2")]]` de GCC → `__cpuid` + intrínsecos SSE4.2 incondicionales en MSVC, tras `#if defined(_MSC_VER)`); el camino GCC/Clang queda intacto.
- (+) El **diseño de ADR-0022 se confirma sin cambios**: ningún ajuste en el código IOCP/ `File` / `Socket` fue necesario para que funcionara en runtime (las rutas `Win32/Winsock` estaban correctas).
- (–) La verificación es **manual** (ejecutar `tools/wincheck`), no automática: al reactivar el CI, un job `windows-latest` debería ejecutarlo para evitar regresiones (tarea de cierre).
- (–) Sigue siendo **solo** `nexus-io` : el resto del servidor (reactor *pinning* con `pthread_setaffinity_np` , `nexusrd` con señales POSIX/eventfd, backend `io_uring`) es Linux-nativo; un `nexusrd` Windows completo queda fuera de alcance.

Alternativas consideradas

- **Mantener ADR-0022 (runtime como deuda)**: ya no se sostiene —hay máquina Windows con MSVC disponible—; dejar la capa sin ejecutar nunca era el único hueco de F10. Descartado.
- **Job de CI en `windows-latest` en vez de arnés local**: cerraría la regresión de forma automática, pero el CI está desactivado por cuota y el árbol completo no compila en Windows; el arnés local es **ejecutable hoy**. No es excluyente: ese job ejecutaría precisamente `tools/wincheck` . Aplazado.
- **Convertir `wincheck` en test de ctest (GoogleTest)**: mezclaría la verificación Windows con la suite Linux (que no compila en Windows) y arrastraría el *toolchain* de tests; un **ejecutable autónomo** es más simple, aislado y no depende de `NEXUS_BUILD_TESTS` . Descartado.

ADR-0024: Compactación del log de Raft por snapshot (base de snapshot en `RaftLog`)

- **Estado:** aceptado
- **Fecha:** 2026-06-20

Contexto

Bajo Raft por partición (ADR-0003), el **log replicado es el WAL** (glosario): crece sin techo aunque sus entradas más antiguas ya estén *committed* y aplicadas. Sin compactación, (1) el log de cada partición crece indefinidamente y (2) un seguidor que se reincorpora muy rezagado obligaría al líder a re-replicar desde el índice 1. El paper de Raft resuelve ambos con **snapshots** (§7): se toma una imagen del estado hasta `last_included_index`, se descarta el prefijo del log cubierto por ella y un seguidor demasiado atrás se pone al día con `InstallSnapshot` en vez de `AppendEntries`. El RPC `InstallSnapshot` y el tipo `Snapshot` ya existían (ADR-0014/Fase 2), pero `RaftLog` era **denso desde el índice 1** (índice `i` ↔ `entries_[i-1]`), sin forma de descartar un prefijo ni de recuperar `term_at(last_included_index)` tras hacerlo.

Decisión

Dotar a `RaftLog` de una **base de snapshot** (`last_included_index`, `last_included_term`, `last_included_offset`) y compactar contra ella en tres planos coherentes:

1. **Modelo lógico (exacto).** `RaftLog` recuerda `snapshot_index_/snapshot_term_`; el índice `i` vive en `entries_[i - snapshot_index_ - 1]`, `last_index() = snapshot_index_ + entries_.size()`, `term_at(snapshot_index_) = snapshot_term_` y `term_at(i < snapshot_index_)` es `Compacted`. `compact_to(index)` descarta el prefijo aplicado **exactamente** en `index` (el llamante garantiza `index ≤ commit_index`).
2. **Persistencia.** La base de snapshot se guarda en un **sidecar dedicado** `raft-snapshot` (un único registro de tamaño fijo con **CRC32C**, mismo patrón que `RaftStateStore` de D1), separado del sidecar de metadatos por entrada (`raft-meta`). Al abrir, `RaftLog::open` lee la base y reconstruye el espacio de índices; la comprobación de coherencia con el `PartitionLog` se generaliza para admitir un `log_start_offset` ya recortado.
3. **Reclamación física (best-effort).** `PartitionLog::truncate_prefix_to(offset)` borra los segmentos **sellados enteros** cuyo rango queda por debajo de `offset` (nunca el activo, nunca a media trama), avanzando `log_start_offset`. La compactación lógica es exacta en el índice; la física libera disco por segmentos completos (puede quedar algo por encima de `log_start_offset` hasta que el segmento rote). Es la pieza simétrica de `enforce_retention`, pero recortando a un **offset preciso**.

La **puesta al día de un seguidor** muy rezagado (cuyo `next_index ≤ snapshot_index_` del líder) se hace con `InstallSnapshot`: el seguidor adopta la base de snapshot del líder y reanuda la replicación normal desde ahí. Coherente con ADR-0015 (FSM sin E/S): instalar un snapshot es una transición síncrona que reposiciona la base del log; el portador hace la E/S.

Alcance / FSM del broker. En NexusMQ la "máquina de estados" aplicada **es el propio log de particiones** (los `RecordBatch`); el `state` del `Snapshot` no es una materialización aparte, sino la **base** (`index`, `term`,

offset) que mantiene consistente el espacio de offsets entre réplicas. La visibilidad de los **datos** antiguos para consumidores la gobierna la **retención** (ADR de storage), ortogonal a la compactación del WAL de Raft.

Consecuencias

- (+) El log de Raft deja de crecer sin techo: una réplica acota su tamaño compactando el prefijo aplicado, y la base sobrevive a reinicios.
- (+) Reaprovecha el patrón de durabilidad de D1 (registro fijo + CRC32C + `fsync`) y la maquinaria de borrado de segmentos del `PartitionLog`.
- (+) El RPC `InstallSnapshot` ya existente encuentra por fin un modelo donde encajar.
- (–) La reclamación física es por segmentos enteros (no exacta al byte): aceptable y estándar.
- (–) La puesta al día por `InstallSnapshot` en el seguidor exige reposicionar el `PartitionLog` a una base vacía: se implementa de forma incremental (D2b) y la compactación **no se dispara automáticamente** hasta cablearla con seguridad en el servidor vivo (D3), de modo que el hueco queda latente, no activo.

Alternativas consideradas

- **No compactar (retención del WAL como único freno):** la retención borra datos antiguos, pero no resuelve el espacio de índices de Raft ni la puesta al día de un seguidor rezagado por debajo del punto de borrado. Insuficiente. Descartada.
- **Snapshot como materialización separada del estado (fichero de imagen):** clásico del paper, pero en un broker el estado aplicado es el propio log; duplicarlo en una imagen aparte sería redundante y costoso. Se adopta la base `(index, term, offset)` + datos en el `PartitionLog`. Descartada.
- **Recorte físico exacto al byte (partir segmentos):** complicaría el `PartitionLog` (segmentos inmutables una vez sellados) por una ganancia marginal. Se recorta por segmentos enteros. Descartada.

ADR-0025: Activación en producción del stack Raft + multi-reactor con transporte inter-nodo real

- **Estado:** aceptado
- **Fecha:** 2026-06-21

Contexto

Las fases previas dejaron listas, pero **sin enchufar al servidor vivo**, varias piezas: el log y la FSM de Raft (ADR-0014/0015), la `ReplicatedPartition` (ADR-0016), el `routing cross-core` (`PartitionRouter / call_on`), el `ReactorPool thread-per-core` (ADR-0005) y la persistencia (D1) y compactación (D2) del estado de Raft.

El binario `nexusrd` corre hoy con **un único Reactor** y `Partition` simples (`ack local`); los RPC de Raft solo circulan por la **red virtual** de los tests (simulación determinista); nadie persiste el estado de Raft ni dispara la compactación en producción. D3 cierra esa deuda diferida por *fasing*: **activar** el stack en `nexusrd` con **transporte real** entre nodos. No reescribe nada; es integración más inyección de un transporte de red.

Decisión

Cuatro piezas coherentes, implementadas de forma incremental (D3.1..D3.7), siempre verde:

1. **Plano inter-nodo separado del plano de cliente.** Los RPC de Raft viajan por su **propio plano** (su listener, sus conexiones persistentes entre nodos), distinto del plano de datos del cliente (ADR-0004/0013). Un **sobre de wire de Raft** rutea cada `RaftMessage` y el sobre identifica la **réplica de partición destino** (`topic`, `partition`) para entregar el RPC al `RaftNode` correcto del nodo receptor.
2. **Portador por partición, afinado al reactor dueño.** Cada réplica la conduce un **portador** (`RaftCarrier`) que vive en el reactor dueño de la partición (`PartitionRouter: core = partition % cores`): un `tick` periódico **local** avanza la FSM, drena `take_messages()` y entrega cada mensaje al transporte; los RPC entrantes se enrutan al reactor dueño (`call_on`) y se aplican con `on_*`. El camino del `tick` es local al reactor de la partición (sin cross-core), respetando shared-nothing: no hay estado de Raft compartido entre núcleos.
3. **Persistencia y compactación en el lazo del portador.** Al abrir la réplica se carga `RaftStateStore::load()` → `restore_persistent_state` (D1); **antes** de transportar `take_messages()`, si `persistent_state_dirty()`, se hace `save()` con `fsync` (regla §5 de Raft: persistir término/voto antes de responder al RPC). La compactación (`compact_to`, D2) se dispara por **política** (umbral de entradas aplicadas) desde el portador, ya con seguridad.
4. **Multi-reactor en el server.** El `Server` pasa de un `Reactor` único a un `ReactorPool` (uno por núcleo, *pinned*); el listener acepta en el reactor 0 y el `RequestRouter` enruta cada petición a la partición por su reactor dueño (`call_on`), volviéndose **asíncrono** (`task<expected<...>>`). `ReplicatedPartition` (ADR-0016) sustituye a `Partition` cuando `replication_factor > 1`; con factor 1 (mono-nodo, desarrollo) sigue `Partition`.

Sobre de wire de Raft

El formato del sobre es `topic | partition | from | to | type:u8 | payload`, con prefijo de longitud para *streaming* sobre TCP. El campo `type` es el discriminante del `variant` del RPC y el `payload` reutiliza el `encode / decode` por RPC ya existente (ADR-0014). De este modo el contrato de wire de Raft reaprovecha los codecs de los RPC sin duplicarlos.

Consecuencias

- (+) D1/D2 dejan de ser mecanismo latente: el consenso se replica de verdad por la red y sobrevive a reinicios; la compactación acota el log en el servidor vivo.
- (+) El plano inter-nodo separado evita mezclar el contrato de cliente con el de réplica: evolucionan por separado (ADR-0013) y tienen SLAs distintos.
- (+) El portador por reactor respeta shared-nothing (ADR-0005).
- (+) La red virtual determinista se **conserva** para probar la lógica de la FSM; el transporte real se prueba aparte (sockets de loopback más inyección de caos).
- (–) Aparece un segundo listener/puerto (inter-nodo) y direccionamiento de peers por configuración.
- (–) El `RequestRouter` cambia de firma a asíncrono y el camino de despacho se refactoriza; se hace incremental para no romper el árbol.
- (–) El transporte real introduce timeouts/reintentos/reordenamiento que la red virtual no tenía: se asumen como parte del plano de red (`fundamentos/redes/`).

Alternativas consideradas

- **Reutilizar el plano de cliente para los RPC de Raft (un puerto, `Apikey` s nuevas):** acopla el contrato de cliente al de réplica, mezcla flujos con SLAs distintos y complica el versionado. Se separa el plano. Descartada.
- **Portador global único (un hilo conduce todas las particiones):** concentra el trabajo, rompe shared-nothing y serializa el consenso de particiones independientes. Se hace por partición/reactor. Descartada.
- **Persistir el estado de Raft en cada `tick` (no solo si *dirty*):** `fsync` innecesarios en el camino caliente. Se persiste solo ante cambio (`dirty`), antes de enviar. Descartada.
- **`co_await cross-core` para el *reply* del RPC (estilo `call_on`):** un RPC a un nodo remoto **no vuelve** al completarse; su respuesta entra después como **otro** RPC (notificación asíncrona). El transporte inter-nodo es *fire-and-forget* con *reply* diferido, no petición/respuesta acoplada (a diferencia del `call_on` cross-core, que sí es local y síncrono entre reactores). Descartada para el plano de red.

ADR-0026: Sharding por núcleo del plano de datos — partición → núcleo y coordinación de grupos por hash

- **Estado:** aceptado
- **Fecha:** 2026-06-21

Contexto

ADR-0025 (punto 4) decidió mover el `Server` a un `ReactorPool` y volver asíncrono el `RequestRouter`, pero dejó a **alto nivel** cómo se reparte el estado entre núcleos. Tras D3.4a (`dispatch` asíncrono) y D3.4b (`Server` sobre `ReactorPool` con el plano de datos **confinado al núcleo 0**, sin *data races*), toca decidir el **sharding** concreto.

El modelo es *shared-nothing thread-per-core* (ADR-0005): ningún estado mutable se comparte entre hilos; el único canal entre núcleos es el paso de mensajes (`PartitionRouter` / `call_on`, ya existentes y probados). Hay tres estados que ubicar:

- (a) el **log/estado de cada partición** (`Partition` / `ReplicatedPartition`, hoy todos en un `TopicManager` único),
- (b) los **metadatos de topics** (qué topics existen y con cuántas particiones),
- (c) la **coordinación de grupos de consumidores** y sus **offsets confirmados** (`GroupCoordinator` / `OffsetManager`, hoy una instancia REACTOR-LOCAL por router).

Decisión

Tres reglas de ubicación, coherentes con shared-nothing:

1. **Partición → núcleo dueño = $\text{partition} \% N$** (ya lo fija `PartitionRouter`). El **estado de cada partición** (su log, su pila Raft) vive **solo** en el reactor dueño. El `TopicManager` se **fragmenta**: cada núcleo abre e instancia únicamente las particiones que le tocan; las demás no existen en su memoria. Las operaciones de partición (Produce/Fetch) se enrutan al núcleo dueño con `call_on` (cross-core local y síncrono entre reactores; ADR-0025).
2. **Metadatos de topics = inmutables tras crear, replicados por valor a cada núcleo.** El conjunto `{nombre topic → nº particiones}` es plano de control que cambia raras veces (Create/DeleteTopic). Cada núcleo guarda **su propia copia** (INMUTABLE entre cambios), de modo que `Metadata` y la validación ("¿existe la partición P?") se responden **localmente** sin cross-core. Un cambio de metadatos se propaga publicando la nueva copia a cada núcleo vía `submit_to` (control, no *hot path*). No hay un `TopicManager` compartido con lock: se elige réplica inmutable por núcleo frente a estado compartido sincronizado.
3. **Grupo de consumidores → núcleo coordinador = $\text{hash}(\text{group_id}) \% N$** (estilo Kafka *group coordinator*). La membresía del grupo (`GroupCoordinator`) y sus **offsets confirmados** (`OffsetManager`) viven en **un solo** núcleo —el coordinador del grupo—, no replicados. Las operaciones de grupo (JoinGroup/SyncGroup/Heartbeat/OffsetCommit/OffsetFetch) se enrutan a ese núcleo con `call_on`. Así cada grupo tiene un **dueño único** (linealizabile sin locks) y la carga de grupos

se reparte entre núcleos. El listado de grupos del puerto admin agrega con `call_on` sobre todos los núcleos.

Función de hash de ubicación

`hash(group_id)` usa una función estable y documentada (FNV-1a sobre los bytes del id), parte del **contrato interno** de ubicación; no viaja por wire.

Consecuencias

- (+) Cero estado mutable compartido entre hilos: el modelo sigue siendo shared-nothing puro (ADR-0005); la corrección no depende de locks sino del confinamiento por reactor.
- (+) Produce/Fetch y las operaciones de grupo escalan con los núcleos; cada partición y cada grupo tienen un dueño único que serializa sus operaciones sin contención global.
- (+) Metadata /validación son locales (sin cross-core en el *hot path* de cada petición).
- (–) Una petición que toca una partición o un grupo de **otro** núcleo paga un salto cross-core (`call_on` : encolar en el buzón del dueño más reanudar); es el coste inherente del sharding y se mide en D3.4.
- (–) Los metadatos de topics se **duplican** por núcleo: aceptable por ser pequeños e inmutables entre cambios, pero la propagación de un Create/DeleteTopic debe alcanzar a todos los núcleos antes de servir el nuevo estado.
- (–) El `RequestRouter` pasa a tener **una instancia por núcleo** y a depender del `PartitionRouter / call_on` ; su modelo de test cambia (de `sync_wait` sin reactor a conducir un reactor real o un doble).

Alternativas consideradas

- **TopicManager único compartido con lock (RW-mutex)**: rompe shared-nothing, introduce contención en el *hot path* y *cache ping-pong* en el lock; contradice ADR-0005 y la normativa de concurrencia ("la mejor región crítica es la que no existe"). Descartada.
- **Coordinador de grupos global en el núcleo 0**: concentra toda la carga de grupos en un núcleo (punto caliente) y serializa grupos independientes. Se reparte por hash del `group_id` . Descartada.
- **Replicar también offsets/membresía de grupo a todos los núcleos**: exigiría sincronizar escrituras entre núcleos (consenso o locks) para mantener un único valor confirmado por grupo; rompe el dueño único. Se confina cada grupo a su coordinador. Descartada.
- **Sharding de particiones por *hash(topic,partition)* en vez de `partition % N`** : repartiría mejor topics con pocas particiones, pero rompe la igualdad con `PartitionRouter` (ya probado) y el mapeo directo partición → Raft → núcleo. Se mantiene `partition % N` . Descartada.

ADR-0027: Cableado del modo proxy en el plano de datos — *pool* de conexiones aguas arriba por reactor

- **Estado:** aceptado
- **Fecha:** 2026-06-25

Contexto

ADR-0006 definió el *ingress* en **dos modos**: nativo directo (primario, *smart-client* al líder) y **proxy** (secundario, *opt-in*: enruta clientes "tontos" por *consistent-hashing* y releva sus tramas a un nodo). I18 implementó la lógica enrutable y testeable del proxy (`Proxy::route` sobre el anillo del `LoadBalancer` y `Proxy::forward` , relevo petición/respuesta **a nivel de trama** entre dos `Socket` ya conectados), pero **difirió** al cableado de servidor (D4-3) el *dialado* del nodo aguas arriba y el *pool*/reúso de esas conexiones.

Faltan dos piezas para activar el modo en el plano de datos: (a) **establecer** la conexión al plano de datos del nodo elegido **sin bloquear** el reactor, y (b) **reutilizarla** (un *handshake* TCP por petición cuesta RTTs; la normativa de redes exige *connection pooling*). Además, el `PeerDirectory` (ADR-0025) resuelve solo el **plano inter-nodo (Raft)**, en un puerto **separado** del plano de datos: el proxy no puede reusarlo para dialar el puerto de cliente.

Decisión

1. **UpstreamPool por reactor, REACTOR-LOCAL.** Cada reactor mantiene su propio *pool* —sin locks, coherente con shared-nothing (ADR-0005)—. `acquire(proactor, node)` devuelve una conexión **viva y en préstamo exclusivo** al nodo: reúsa una ociosa de la *free-list* del nodo o **diala una nueva** de forma asíncrona (`Socket::async_connect`) si no hay. `release(node, socket)` la devuelve a la *free-list* para reúso. El préstamo es **exclusivo** mientras dura el relevo de una conexión de cliente (el relevo es petición/respuesta a nivel de trama, I18): así nunca se **intercalan** tramas de dos clientes en la misma conexión aguas arriba (que corrompería el flujo). La *free-list* por nodo está **acotada** (cierra el excedente) para no fugar descriptores ni crecer sin límite (colas acotadas, normativa de concurrencia/datos-distribuidos).
2. **Directorio de direcciones del plano de datos, distinto del de Raft.** El `UpstreamPool` resuelve `NodeId → host:puerto` del **plano de cliente** mediante un `PeerDirectory` poblado con direcciones del plano de datos (instancia **separada** de la del plano Raft de ADR-0025). Se **reutiliza el tipo** `PeerDirectory / PeerAddress` (estructura idéntica: `NodeId → host:puerto` , inmutable, thread-safe de solo lectura) en vez de duplicar un tipo casi igual (minimiza entidades, Kent Beck 4); la distinción Raft vs datos es por **instancia y composición**, no por tipo. El *composition root* construye y posee ambos directorios.
3. **Activación opt-in.** El modo proxy del plano de datos es **opt-in** (ADR-0006): sin configurarlo, el servidor atiende en modo nativo directo (cada conexión la sirve `serve_connection` localmente). Con el modo activo, la conexión de cliente se enruta (`Proxy::route`) a un nodo y se releva (`Proxy::forward`) sobre una conexión obtenida del `UpstreamPool` , devuelta al *pool* al cerrar limpiamente el cliente.

Consecuencias

- (+) El proxy reutiliza conexiones TCP aguas arriba (ahorra *handshakes*/RTTs), cumple la normativa de redes y mantiene shared-nothing (un *pool* por reactor, sin contención).
- (+) El dialado es **asíncrono** (no congela el reactor).
- (+) Préstamo exclusivo $\Rightarrow$ sin intercalado de tramas $\Rightarrow$ corrección del relevo.
- (–) Una conexión ociosa reusada puede haber sido cerrada por el par mientras esperaba; el primer uso fallará y el llamante la descarta (un *health-check*/reintento de un salto queda como **mejora futura**, no en este corte).
- (–) Otro estado por reactor, que vive y se destruye en orden con el *pool* de reactores.
- (–) El proxy añade un salto y rompe el *zero-copy* (coste asumido y documentado en ADR-0006).

Alternativas consideradas

- **Pool global compartido entre reactores (con lock):** rompe shared-nothing (ADR-0005), introduce contención y *cache ping-pong* en el *hot path*; contradice la normativa de concurrencia. Descartada.
- **Dialar una conexión nueva por cada conexión de cliente, sin pool:** simple, pero paga un *handshake* TCP por cliente y no reusa nada; contradice la normativa de redes (*connection pooling*). Se conserva como **caso degenerado** (free-list vacía), no como diseño. Descartada como único.
- **Reúso concurrente de una conexión aguas arriba entre varios clientes (multiplexado por `correlation_id`):** maximizaría el reúso, pero exige **remapear** `correlation_id` y casar respuestas fuera de orden; mucho más complejo y propenso a errores. El relevo de I18 es por conexión; se mantiene el préstamo exclusivo. Descartada (posible evolución futura).
- **Añadir el puerto de datos al `PeerDirectory` de Raft (un solo directorio con dos puertos):** acopla dos planos con ciclos de vida y semánticas distintas en un mismo tipo; se prefieren **dos instancias** del mismo tipo. Descartada.

ADR-0028: Port completo de `nexusd` a Windows — backend, afinidad y señales por plataforma

- **Estado:** aceptado
- **Fecha:** 2026-06-27

Contexto

ADR-0023 dejó `nexus-io` (File/Socket/Listener + `IoctlBackend`) **verificado en runtime** en Windows, pero ese alcance se limitaba a `nexus-io`: el resto del árbol seguía Linux-nativo y **no compilaba** en Windows. Este ADR amplía aquel alcance "solo `nexus-io`" al daemon completo (W3). Los puntos que bloqueaban la compilación del árbol entero eran:

- la **factoría de proactor** del `Server` construía siempre `IoUringBackend` (declarado en todas las plataformas pero **definido solo en Linux** → fallo de enlace),
- el *pinning* del `ReactorPool` usaba `pthread_setaffinity_np / cpu_set_t`,
- `nexusd` registraba señales POSIX,
- quedaban fugas POSIX en hojas (cliente/CLI bloqueantes con sockets BSD, `gmtime_r`, `std::filesystem::path::c_str()` que es `wchar_t*` en Windows y choca con las APIs `const char*` de OpenSSL).

El valor de portabilidad —que el puerto `Proactor` admite otro proactor— ya estaba demostrado por `nexus-io`; faltaba **extenderlo al daemon entero** (W3).

Decisión

Portar todo el árbol a Windows confinando la divergencia tras la abstracción `nexus-io` (`Proactor`) y guardas `#if defined(_WIN32)`, dejando el camino POSIX byte-idéntico. Cuatro piezas:

- Factoría de proactor (compilación).** `Server` elige el backend por defecto en compilación: `IoctlBackend` en Windows, `IoUringBackend` en Linux (`make_default_proactor`, antes `make_io_uring_proactor`). Ambos son **proactores sobre el mismo puerto `Proactor`** (ADR-0021/0023), así que el bucle del reactor, el `RequestRouter` y las corrutinas de servicio **no cambian**. La DIP sigue intacta: los tests inyectan su doble.
- Afinidad de hilo.** `pin_thread_to_core` usa `SetThreadAffinityMask(handle, mask)` en Win32 y `pthread_setaffinity_np / cpu_set_t` en POSIX, con el **mismo reparto** `core % cpus`. Mejor esfuerzo en ambos; en Windows la máscara cubre un grupo de procesadores (≤ 64 CPUs), suficiente para el objetivo.
- Señales / parada.** `nexusd` registra `SetConsoleCtrlHandler` en Windows (capta Ctrl-C, Ctrl-Break, cierre de consola, *logoff* y apagado, en un hilo aparte) y `std::signal(SIGINT/SIGTERM)` en POSIX. Ambos manejadores solo **despiertan** el servidor: `Server::stop()` marca un flag y llama `Reactor::stop()` → `Proactor::wake()`, que en IOCP es `PostQueuedCompletionStatus` (thread-safe). El "despertar por `eventfd`" era un detalle del backend `io_uring`; **la abstracción `wake()` ya era portable**, así que el plano de control no necesitó rediseño, solo el registro del manejador.

4. **Hojas POSIX.** Sockets bloqueantes del cliente nativo y del CLI → Winsock (`socket / closesocket` , firma y signo de `send / recv` , sin `MSG_NOSIGNAL` , `WSAStartup` por proceso vía RAII como en `nexus-io`); `gmtime_r` → `gmtime_s` (argumentos invertidos); rutas a OpenSSL vía un `native_path` que en POSIX devuelve la propia `path` (coste cero) y en Windows materializa la string narrow. Y se alinea la **política de avisos** MSVC con la canónica de Linux desactivando dos avisos `/W4` benignos que el gate Linux no exige (`C4324` —padding por `alignas` *deliberado* en las colas SPSC/MPMC— y `C4456` —*shadowing* idiomático de variables de condición en cadenas `if/else-if` —).

Consecuencias

- (+) **Todo el árbol** (`nexusd` , `nexus-cli` , `nexus-loadgen` , librerías) **compila y enlaza en Windows** con MSVC `/W4 /WX` y con clang-cl (W2), no solo `nexus-io`: la portabilidad del puerto `Proactor` queda demostrada para el daemon completo.
- (+) Un solo binario, dos plataformas; comportamiento **idéntico en Linux** (todo guardado o inerte).
- (+) El diseño `Proactor` de ADR-0021/0023 se confirma: bastó **elegir** el backend, sin tocar el reactor.
- (–) El **runtime** del daemon en Windows (smoke produce/fetch por IOCP) y los **tests** (`ctest`) se verifican en etapas aparte (mismo patrón que ADR-0023).
- (–) La afinidad en máquinas de **> 64 CPUs** es aproximada en Windows (un solo grupo de procesadores); aceptable para el objetivo de *locality*.
- (–) El **gate canónico** (Linux: GCC/libstdc++ + Clang/libc++ + ASan/TSan) **no se ejecuta en Windows**: el cambio se entrega en la rama `windows-port` y el autor lo revalida y fusiona desde una sesión Linux.

Alternativas consideradas

- **Selección de backend por polimorfismo en runtime (un flag/registro)**: innecesaria —los backends son **excluyentes por plataforma**, conocida en compilación—; un `#if` es coste cero y no añade estado. Descartada.
- **Capa de abstracción de sockets bloqueantes compartida**: solo dos ficheros la usan (cliente nativo y CLI); una guarda local por fichero es más simple que un tipo nuevo. Aplazada (si surge un tercer consumidor).
- **Quedarse con `std::signal(SIGINT)` también en Windows**: compila, pero **no capta** cierre de consola/logoff/apagado y `SIGTERM` no se levanta en Windows; `SetConsoleCtrlHandler` es idiomático y completo. Descartada.
- **Renombrar las variables que disparan `C4456` o reordenar las colas para `C4324`**: churn en código **canónico** por avisos que la política de Linux no exige; se prefiere **alinear la política MSVC** (espejo de los checks apagados en `.clang-tidy`), dejando el código compartido intacto. Descartada.

ADR-0029: Adaptador Kafka asíncrono cross-core sobre el broker vivo + codec por versión (interop `kcat`)

- **Estado:** aceptado
- **Fecha:** 2026-06-28

Contexto

F7a–F7e entregaron el subconjunto Kafka como **librería de protocolo** (`src/kafka/` : codec big-endian, cabeceras, `ApiVersions / Metadata / Produce / Fetch` , `dispatcher` `KafkaGateway` con el puerto `KafkaBroker`), pero **sin cablear al servidor**: `nexusd` no escuchaba Kafka por ningún socket y no existía un adaptador real del puerto `KafkaBroker` . Faltaba cerrar el objetivo declarado de F7 —"habla con `kcat` "— con **interop verificada en vivo**.

Dos fuerzas condicionan el diseño:

1. el plano de datos es **shared-nothing thread-per-core** con **sharding partición → núcleo** (ADR-0026), así que una conexión Kafka aceptada en un reactor cualquiera debe poder tocar particiones que **posee otro núcleo**;
2. `kcat /librdkafka` **no negocia las versiones flexibles** que F7d/F7e asumían: usa **Produce v7** y **Fetch v11**, que son **clásicas** (longitudes `INT16 / INT32` , sin *tagged fields*), mientras que `Metadata v9 / ListOffsets v7 / ApiVersions v3` sí son flexibles.

Decisión

Un adaptador `KafkaServerBroker` (en `nexus-server`) que implementa el puerto `KafkaBroker` como corrutinas `task<...>` y enruta cada partición a su núcleo dueño vía `PartitionRouter::route` (ADR-0026), con un *listener* Kafka en puerto propio (`--kafka-port`) y codecs Produce/Fetch gobernados por la versión negociada. Cinco piezas:

1. **Puerto asíncrono.** Los métodos de `KafkaBroker` (`metadata / produce / fetch / list_offsets`) pasan a devolver `task<...>`: el `KafkaGateway` hace `co_await broker_.X(req)` . Así el adaptador puede **saltar al reactor dueño** de la partición y volver, en vez de restringir el servicio al núcleo 0. Es la elección **correcta** frente a un adaptador síncrono-solo-núcleo-0: respeta el *sharding* sin serializar todo en un núcleo.
2. **Cableado.** `Server` abre un *listener* en `--kafka-port` (separado del 9092 nativo); cada conexión la sirve `serve_kafka_connection` con el framing **Size:INT32** (big-endian) que alimenta `KafkaGateway::handle_request` . El adaptador se construye en el *composition root* del servidor (núcleo 0 como ancla de metadatos; `bind_cluster` le inyecta el `PartitionRouter` y los `TopicManager` por núcleo).
3. **Codec por versión.** `decode/encode` de Produce y Fetch reciben `api_version` y ramifican **clásico vs flexible** (`is_flexible(api_key, version)`), con *gates* de campo por versión (p. ej. `transactional_id v3+`, `last_fetched_epoch` **solo v12+** —el bug que rompía Fetch v11—). `ApiVersions` anuncia rangos que cubren lo que negocia `librdkafka`.

4. **Almacenamiento opaco con rebase de offset.** El `RecordBatch` v2 de Kafka se guarda **opaco** dentro del envoltorio interno del log (solo su `record_count` gobierna los offsets); en `Fetch` se reconstruye el blob y se **reescribe el `baseOffset`** autoritativo (el CRC de Kafka cubre desde `attributes`, así que parchear `baseOffset` **no** invalida el CRC).
5. **Records nunca `null`.** Un `Fetch` sin datos nuevos (consumidor al día con el *high-watermark*) devuelve un `MessageSet` **vacío** (longitud 0), **no `null`** (-1): `librdkafka` rechaza un `MessageSetSize` de -1 como trama corrupta.

Consecuencias

- (+) `kcat` **interopera en vivo**: `-L` (metadata), `-P` (produce) y `-C` (consume) funcionan, incluido **multi-partición cross-core** (mensajes servidos desde el núcleo dueño).
- (+) El puerto `KafkaBroker` queda **desacoplado** del broker (DIP): el protocolo no conoce `nexus-broker` y se testea con un `FakeBroker`; el adaptador con un broker local *unbound*.
- (+) El diseño de ADR-0026 se confirma: bastó `co_await route(...)` por partición.
- (-) El adaptador asíncrono **propaga `task<>`** a todo el puerto (más verboso que síncrono), justificado por la corrección cross-core.
- (-) Cobertura **parcial** del protocolo: solo las 5 APIs y las versiones que negocia `librdkafka`; sin grupos de consumidores (`JoinGroup` / `Heartbeat`) ni transacciones —fuera del alcance *stretch*.

Alternativas consideradas

- **Adaptador síncrono que sirve solo desde el núcleo 0:** *diff* menor, pero **viola el sharding** (toda partición tendría que vivir o copiarse al núcleo 0) o serializa el tráfico Kafka en un núcleo. Descartada: la corrección y la coherencia con ADR-0026 pesan más que el tamaño del cambio.
- **Asumir versiones flexibles (v9/v12) e ignorar las clásicas:** es lo que hacía F7d/F7e y **rompe con `kcat`** (Produce v7 / Fetch v11). El codec **debe** ramificar por versión. Descartada.
- **Traducir el `RecordBatch` de Kafka a/desde el formato interno (no opaco):** caro y frágil (reencode más recálculo de CRC en el *hot path*) sin beneficio: el log solo necesita contar records y asignar offsets. Descartada a favor del blob opaco más rebase de `baseOffset`.